

CS25302

COMPUTER ARCHITECTURE
(COMMON TO CSE, ADS, CSBS, M.Tech CSE)

L	P	T	C
3	0	0	3

OBJECTIVES

- ❖ To learn the basic structure and operations of a computer.
- ❖ To learn the arithmetic and logic unit and implementation of fixed-point and floating-point arithmetic unit.
- ❖ To learn the basics of pipelined execution.
- ❖ To understand parallelism and multi-core processors.
- ❖ To understand the memory hierarchies and the ways of communication with I/O devices.

UNIT I	BASIC STRUCTURE OF A COMPUTER SYSTEM	9
---------------	---	----------

Eight ideas-Functional Units – Basic Operational Concepts – Performance – Instructions: Language of the Computer – Operations, Operands – Instruction representation – Logical operations – decision making – MIPS Addressing.

CO1

UNIT II	DATA REPRESENTATION AND ARITHMETIC FOR COMPUTERS	9
----------------	---	----------

Signed number representation, Addition and Subtraction – Multiplication – Division – Fixed- and Floating-Point Representation – Floating Point Operations.

CO2

UNIT III	DATA PATH AND CONTROL UNIT	9
-----------------	-----------------------------------	----------

A Basic MIPS implementation – Building a Datapath – Control Implementation Scheme – Pipelining – Pipelined data path and control – Handling Data Hazards & Control Hazards – Exceptions.

CO3

UNIT IV	PARALLELISM	9
----------------	--------------------	----------

Parallel Processing challenges – Flynn's classification – SISD, MIMD, SIMD, SPMD, and Vector Architectures - Hardware multithreading – multi-core processors and other Shared Memory Multiprocessors - Introduction to Graphics Processing Units, Clusters, Warehouse Scale Computers and other Message-Passing Multiprocessors.

CO4

UNIT V	MEMORY AND PERIPHERAL DEVICES	9
---------------	--------------------------------------	----------

Memory Hierarchy - memory technologies – cache memory – measuring and improving cache performance – virtual memory, TLB's – Accessing I/O Devices – Interrupts – Direct Memory Access – Bus structure – Bus operation – Arbitration – Interface circuits – USB

CO5

TOTAL : 45 PERIODS

TEXT BOOKS

1. M. David A. Patterson and John L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, Fifth Edition, Morgan Kaufmann / Elsevier, 2014.
2. Carl Hamacher, Zvonko Vranesic, Safwat Zaky and Naraig Manjikian, Computer Organization and Embedded Systems, Sixth Edition, Tata McGraw Hill, 2012.

REFERENCE BOOKS

1. William Stallings, “Computer Organization and Architecture – Designing for Performance”, Tenth Edition, Pearson Education, 2016.
2. John L. Hennessey and David A. Patterson, Computer Architecture – A Quantitative Approach, Morgan Kaufmann / Elsevier Publishers, Fifth Edition, 2012.
3. John P. Hayes, Computer Architecture and Organization, Third Edition, Tata McGraw Hill, 2012.
4. Jim Ledin, “Modern Computer architecture and Organization”, Packet Publishing, 2020.
5. Douglas Comer, “Essentials of Computer Architecture”, Taylor and Francis Group 2020

COURSE OUTCOMES

Upon completion of the course, students will be able to

- CO1 Understand the basics structure of computers, operations and instructions.
- CO2 Design arithmetic and logic unit.
- CO3 Understand pipelined execution and design control unit.
- CO4 Understand parallel processing architectures.
- CO5 Understand the various memory systems and I/O communication

MAPPING OF COs WITH POs AND PSOs														
COs	PROGRAM OUTCOMES (POs)											PROGRAM SPECIFIC OUTCOMES (PSOs)		
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
CO1	2	2	2	1	1	1	1	1	1	0	2	2	2	1
CO2	2	2	2	1	1	1	1	1	1	0	2	2	2	1
CO3	2	2	2	2	2	1	1	1	1	1	2	2	2	1
CO4	2	2	2	2	2	1	1	1	1	1	2	2	2	1
CO5	2	2	2	2	2	1	1	1	1	1	2	2	2	1

UNIT I BASIC STRUCTURE OF A COMPUTER SYSTEM

Eight ideas-Functional Units – Basic Operational Concepts – Performance – Instructions: Language of the Computer – Operations, Operands – Instruction representation – Logical operations – Decision making – MIPS Addressing.

Computer Architecture

Computer architecture is the set of rules and methods that describe the functionality, organization and implementation of computer system. The architecture of a system refers to its structure in terms of separately specified components of that system and their inter-relationships.

Computer architecture has three main subcategories

- **Instruction set architecture (ISA):** This includes the functions and capabilities of the central processing unit. It is the embedded programming language and defines what programming it can perform or process. This part is the software that makes the computer run and it includes data formats and the programmed instruction set.
- **Microarchitecture:** also known as "computer organization", this describes how a particular processor will implement the ISA.
- **Systems design:** includes all hardware components of a computer including data processors, multi processors, memory controllers etc.

Evolution of Processors

There are two major approaches to processor architecture

CISC (Computer Instruction Set Computer): CISC have a single processing unit external memory and a small register set with hundreds of different instructions. These processors have a single instruction to perform a task. The advantage is fewer lines of code are needed to get the job done. This approach uses less memory but it can take longer time to complete instructions.

RISC (Reduced Instruction Set Computer): The RISC processors have a smaller set of instructions with few addressing modes. Execution of RISC instructions are faster and take one clock cycle per instruction.

Eight Great Ideas in Computer Architecture

The following are eight great ideas that computer architects have invented in the last 60 years of computer design.

- Design for Moore's Law.
- Use Abstraction to Simplify Design
- Make the common case fast
- Performance via parallelism
- Performance via pipelining
- Performance via prediction
- Hierarchy of memories
- Dependability via redundancy

1. Design for Moore's Law.

- The Number of Transistors in a integrated circuit doubles approximately every two years. (Gordon Moore, one of the founders of Intel.)
- As computer designs can take years, the resources available per chip can easily double or quadruple between the start and finish of the project.
- Computer architects must anticipate where the technology will be when the design finishes rather than design for where it starts.

2. Use Abstraction to Simplify Design

- In computer architecture, a computer system is usually represented as consisting of five abstraction levels: hardware, firmware, assembler, operating system and processes.
- In computer science, an abstraction level is a generalization of a model or algorithm.
- The simplification provided by a good abstraction layer facilitates easy reuse.
- Both computer architects and programmers had to invent techniques to make themselves more productive, for otherwise design time would lengthen as dramatically as resources grew by Moore's Law.
- A major productivity technique for hardware and software is to use abstractions to represent the design at different levels of representation; lower-level details are hidden to offer a simpler model at higher levels.

3. Make the common case fast

- Making the common case fast will tend to enhance performance better than optimizing the rare case.
 - It implies that you know what the common case is, which is only possible with careful experimentation and measurement.

4. Performance via parallelism

- Computer architects have offered designs that get more performance by performing operations in parallel.
- Parallel computing is a form of computation in which many calculations are carried out simultaneously, operating on the principle that large problems can often be divided into smaller ones which are then solved concurrently. Parallel.
 - Parallelism has been employed for many years, mainly in high performance computing.

5. Performance via pipelining

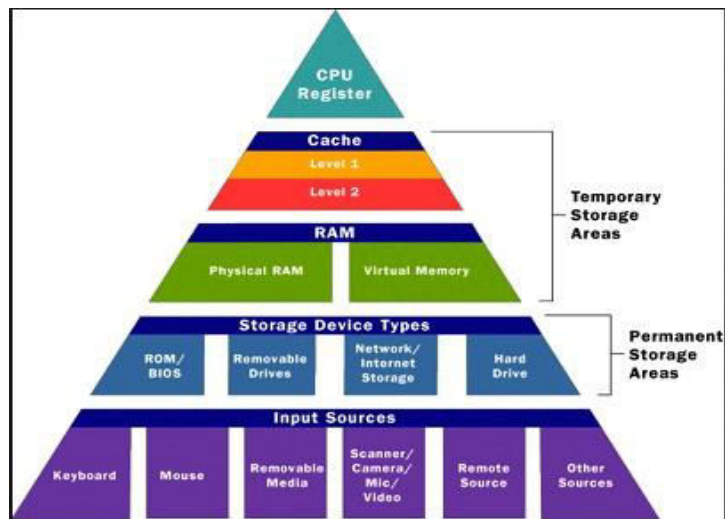
- Pipelining is a technique used in the design of computers to increase the instruction throughput (the number of instructions that can be executed in a unit of time).
- The basic instruction cycle is broken up into a series of pipeline stages.
- Rather than processing each instruction sequentially (one at a time, finishing one instruction before starting the next), each instruction is split up into a sequence of steps so that different steps can be executed concurrently (by different circuitry) and in parallel (at the same time).
- Pipelining increases instruction throughput by performing multiple operations at the same time (in parallel), but does not reduce instruction latency (the time to complete a single instruction from start to finish) as it still must go through all steps.

6. Performance via prediction

- To improve the flow and throughput in a instruction pipeline, Branch predictors play a critical role in achieving high effective performance in many modern pipelined microprocessor architectures.
- Without branch prediction, the processor would have to wait until the conditional jump instruction has passed the execute stage before the next instruction can enter the fetch stage in the pipeline.
- The branch predictor attempts to avoid this waste of time by trying to guess whether the conditional jump is most likely to be taken or not taken.

7. Hierarchy of memories

- Programmers want memory to be fast, large, and cheap, as memory speed often shapes performance, capacity limits the size of problems that can be solved, and the cost of memory today is often the majority of computer cost.
- Architects have found that they can address these conflicting demands with a hierarchy of memories, with the fastest, smallest, and most expensive memory per bit at the top of the hierarchy and the slowest, largest, and cheapest per bit at the bottom.
- Caches give the programmer the illusion that main memory is nearly as fast as the top of the hierarchy and nearly as big and cheap as the bottom of the hierarchy.

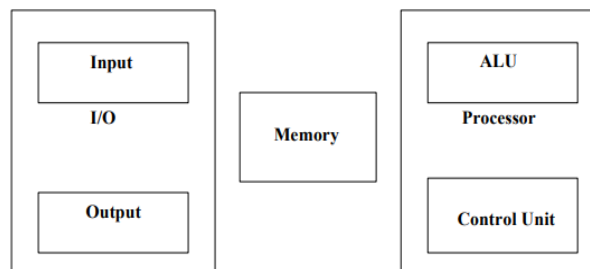


8. Dependability via redundancy

- Computers not only need to be fast; they need to be dependable.
- Since any physical device can fail, we make systems dependable by including redundant components that can take over when a failure occurs and to help detect failures.
- Examples: Systems designers usually provide failover capability in servers, systems or networks requiring continuous availability -- the used term is high availability -- and a high degree of reliability.

FUNCTIONAL UNITS OF COMPUTER SYSTEM

A computer consists of five functionally independent main parts Input, Memory, Arithmetic and Logic unit (ALU), Output and Control unit.



Input device accepts the coded information as source program i.e. high level language. This is either stored in the memory or immediately used by the processor to perform the desired operations. The program stored in the memory determines the processing steps. Basically the computer converts one source program to an object program. i.e. into machine language.

Finally the results are sent to the outside world through output device. All of these actions are coordinated by the control unit.

Input unit: - The source program/high level language program/coded information/simply data is fed to a computer through input devices keyboard is a most common type. Whenever a key is pressed, one corresponding word or number is translated into its equivalent binary code over a cable & fed either to memory or processor.

Example:- Joysticks, trackballs, mouse, scanners etc are other input devices.

Memory unit: - Its function into store programs and data. It is basically to two types

1. Primary memory
2. Secondary memory

1. Primary memory: - It Is the one exclusively associated with the processor and operates at the electronics speeds programs must be stored in this memory while they are being executed. The memory contains a large

number of semiconductors storage cells. Each capable of storing one bit of information. These are processed in a group of fixed size called word.

To provide easy access to a word in memory, a distinct address is associated with each word location. Addresses are numbers that identify memory location.

Number of bits in each word is called word length of the computer. Programs must reside in the memory during execution. Instructions and data can be written into the memory or read out under the control of processor. Memory in which any location can be reached in a short and fixed amount of time after specifying its address is called random-access memory (RAM).

The time required to access one word is called memory access time. Memory which is only readable by the user and contents of which can't be altered is called read only memory (ROM) it contains operating system. Caches are the small fast RAM units, which are coupled with the processor and are often contained on the same IC chip to achieve high performance. Although primary storage is essential it tends to be expensive.

2 Secondary memory: - Is used where large amounts of data & programs have to be stored, particularly information that is accessed infrequently.

Examples: - Magnetic disks & tapes, optical disks (ie CD-ROM's), floppies etc.,

Arithmetic logic unit (ALU):-

Most of the computer operators are executed in ALU of the processor like addition, subtraction, division, multiplication, etc. the operands are brought into the ALU from memory and stored in high speed storage elements called register. Then according to the instructions the operation is performed in the required sequence.

The control and the ALU are many times faster than other devices connected to a computer system. This enables a single processor to control a number of external devices such as key boards, displays, magnetic and optical disks, sensors and other mechanical controllers.

Output unit:-

These actually are the counterparts of input unit. Its basic function is to send the processed results to the outside world.

Examples:- Printer, speakers, monitor etc.

Control unit:-

It effectively is the nerve center that sends signals to other units and senses their states. The actual timing signals that govern the transfer of data between input unit, processor, memory and output unit are generated by the control unit.

BASIC OPERATIONAL CONCEPTS

To perform a given task, an appropriate program consisting of a list of instructions is stored in the memory. Individual instructions are brought from the memory into the processor, which executes the specified operations.

Examples: - Add \$S0,\$S1,\$S2

This instruction adds the operands at \$S1 and \$S2 and places the sum into \$S0. This instruction requires the performance of several steps,

1. First the instruction is fetched from the memory into the processor
2. The operands are fetched and added the contents
3. Finally the resulting sum is stored in the register \$S0
4. In some other type of computers, these two types of operations are performed by separate instructions for performance reasons.

lw \$t0,32(\$s3)

add \$s2,\$t0,\$s1

Transfers between the memory and the processor are started by sending the address of the memory location to be accessed to the memory unit and issuing the appropriate control signals. The data are then transferred to or from the memory.

The fig shows how memory and the processor can be connected. In addition to the ALU and the control circuitry, the processor contains a number of registers used for several different purposes.

Instruction register (IR):- Holds the instruction that is currently being executed. Its output is available for the control circuits, which generates the timing signals that control the various processing

elements in one execution of instruction.

Program counter PC:- This is another specialized register that keeps track of execution of a program. It contains the memory address of the next instruction to be fetched and executed.

Besides IR and PC, there are n-general purpose registers R0 through Rn-1.

The other two registers which facilitate communication with memory are: -

1. MAR – (Memory Address Register):- It holds the address of the location to be accessed.
2. MDR – (Memory Data Register):- It contains the data to be written into or read out of the address location.

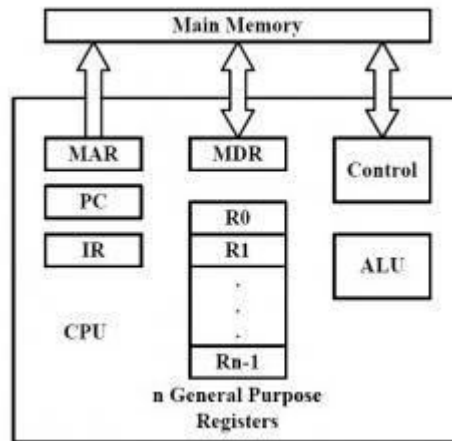


Fig : Connections between the processor and the memory

Operating steps are

1. Programs reside in the memory & usually get these through the I/P unit.
2. Execution of the program starts when the PC is set to point at the first instruction of the program.
3. Contents of PC are transferred to MAR and a Read Control Signal is sent to the memory.
4. After the time required to access the memory elapses, the address word is read out of the memory and loaded into the MDR.
5. Now contents of MDR are transferred to the IR & now the instruction is ready to be decoded and executed.
6. If the instruction involves an operation by the ALU, it is necessary to obtain the required operands.
7. An operand in the memory is fetched by sending its address to MAR & Initiating a read cycle.
8. When the operand has been read from the memory to the MDR, it is transferred from MDR to the ALU.
9. After one or two such repeated cycles, the ALU can perform the desired operation.
10. If the result of this operation is to be stored in the memory, the result is sent to MDR.
11. Address of location where the result is stored is sent to MAR & a write cycle is initiated.
12. The contents of PC are incremented so that PC points to the next instruction that is to be executed.

Normal execution of a program may be preempted (temporarily interrupted) if some devices require urgent servicing, to do this one device raises an Interrupt signal.

An interrupt is a request signal from an I/O device for service by the processor. The processor provides the requested service by executing an appropriate interrupt service routine.

The Diversion may change the internal stage of the processor its state must be saved in the memory location before interruption. When the interrupt-routine service is completed the state of the processor is restored so that the interrupted program may continue.

PERFORMANCE

Response time or execution time: The total time required for the computer to complete a task, including disk accesses, memory accesses, I/O activities, operating system overhead, CPU execution time, and so on.

Throughput: The total amount of work done in a given time.

Bandwidth: The amount of data that can be carried from one point to another in a given time period (usually a second). This kind of bandwidth is usually expressed in bits (of data) per second (bps). Occasionally, it's expressed as bytes per second (Bps).

Clock cycles per instruction (CPI): Average number of clock cycles per instruction for a program or program fragment.

To maximize performance, we want to minimize response time or execution time for some task. Thus, we can relate performance and execution time for a computer X:

$$Performance_X = \frac{1}{Execution\ Time_X}$$

$$Performance_X > Performance_Y$$

$$\frac{1}{Execution\ Time_X} = \frac{1}{Execution\ Time_Y}$$

$$Execution_Y > Execution_X$$

That is, the execution time on Y is longer than that on X, if X is faster than Y. Relate the performance of two different computers quantitatively.

$$\frac{Performance_X}{Performance_Y} = n$$

X is n times faster than Y.

$$\frac{Performance_X}{Performance_Y} = \frac{Execution\ Time_Y}{Execution\ Time_X} = n$$

If X is n times as fast as Y, then the execution time on Y is n times as long as it is on X.

Problem 1:

If computer A runs a program in 10 seconds and computer B runs the same program in 15 seconds, how much faster is A than B?

We know that A is n times as fast as B

$$\frac{Performance_A}{Performance_B} = \frac{Execution\ Time_B}{Execution\ Time_A} = n$$

$$Performance\ ratio = \frac{15}{10} = 1.5$$

A is therefore 1.5 times as fast as B.

CPU performance

The time CPU spends computing for the task and does not include time spent waiting for I/O or running other programs.

Clock :Processor circuits are controlled by a timing signal called clock.It determines when events takes place in the hardware.

Clock cycle:The clock defines regular time intervals called Clock cycle.

Clock period: The length of each clock cycle.

Clock rate(R): Clock rate is defined as $R=1/P$ measured in cycles/sec.

A simple formula relates the most basic metrics (clock cycles and clock cycle time) to CPU time:

CPU execution time for a program = *CPU clock cycles for a program* * *Clock cycle time*

$$CPU\ cycle\ time = \frac{1}{Clock\ Rate}$$

$$CPU\ execution\ time\ for\ a\ program = \frac{CPU\ clock\ cycles\ for\ a\ program}{Clock\ Rate}$$

Problem 2:

Our favorite program runs in 10 seconds on computer A, which has a 2 GHz clock. We are trying to help a computer designer build a computer, B, which will run this program in 6 seconds. The designer has determined that a substantial increase in the clock rate is possible, but this increase will affect the rest of the CPU design, causing computer B to require 1.2 times as many clock cycles as computer A for this program. What clock rate should we tell the designer to target?

Given:

$$Clock\ rate_A = 2\ GHz = 2 \times 10^9\ cycles/second$$

$$CPU\ time_A = 10\ seconds$$

$$CPU\ Clock\ Cycle_B = 1.2 \times CPU\ Clock\ Cycle_A$$

$$CPU\ time_B = 6\ seconds$$

$$CPU\ time_A = \frac{CPU\ Clock\ cycles_A}{Clock\ rate_A}$$

$$CPU\ time_A = \frac{CPU\ Clock\ cycles_A}{Clock\ rate_A}$$

$$CPU\ clock\ cycles = 20 \times 10^9\ cycles$$

$$CPU\ time_B = \frac{CPU\ Clock\ cycles_B}{Clock\ rate_B}$$

$$CPU\ time_B = \frac{1.2 \times CPU\ Clock\ cycles_A}{Clock\ rate_B}$$

$$6\ seconds = \frac{1.2 \times 20 \times 10^9\ cycles}{Clock\ rate_B}$$

$$Clock\ rate_B = \frac{1.2 \times 20 \times 10^9\ cycles}{6\ seconds} = 4\ GHz$$

Therefore, to run the program in 6 seconds, B must have twice the clock rate of A.

INSTRUCTION PERFORMANCE

$$CPU\ clock\ cycles = Instructions\ for\ a\ program * Average\ clock\ cycles\ per\ instruction(CPI)$$

$$CPU \text{ Clock cycles} = \sum_1^n CPI_i \times I_i$$

The performance equation can be written in terms of instruction count, CPI and clock cycle time

$$CPU \text{ time} = CPU \text{ clock cycle} * Clock \text{ Cycle Time}$$

$$CPU \text{ time} = Instruction \text{ count} * CPI * Clock \text{ Cycle Time}$$

$$CPU \text{ time} = \frac{Instruction \text{ count} * CPI}{Clock \text{ rate}}$$

Problem 3:

Let us assume that two computers use same instruction set architecture. Computer A has a clock cycle time of 250ps and a CPI of 2.0 for some program and computer B has a clock cycle time of 500 ps and a CPI of 1.2 for the same program. Which computer is faster for this program and by how much?

Each computer executes the same number of instructions for the program call this number I.

Given:

$$CPI_A = 2.0$$

$$CPI_B = 1.2$$

$$Clock \text{ cycle time}_A = 250 \text{ ps}$$

$$Clock \text{ cycle time}_B = 500 \text{ ps}$$

First find number of processor clock cycles for each computer:

$$CPU \text{ clock cycles} = Instructions \text{ for a program} * Average \text{ clock cycles per instructions.}$$

$$CPU \text{ clock cycles} = I \times CPI$$

$$CPU \text{ clock cycle A} = I * 2.0$$

$$CPU \text{ clock cycle B} = I * 1.2$$

Compute the CPU time for each computers

$$CPU \text{ time}_A = CPU \text{ clock cycle}_A * clock \text{ cycle time}_A = I * 2.0 * 250 \text{ ps} = 500 \times I \text{ ps}$$

$$Cpu \text{ time}_B = CPU \text{ clock cycle}_B * clock \text{ cycle time}_B = I * 1.2 * 500 \text{ ps} = 600 \times I \text{ ps}$$

Computer A is faster.

$$Performance \text{ Raio} = \frac{Performance_A}{Performance_B} = \frac{Execution \text{ Time}_B}{Execution \text{ Time}_A} = \frac{600 \times I \text{ ps}}{500 \times I \text{ ps}} = 1.2$$

Computer A is 1.2 times faster than computer B for this program.

The Classic CPU Performance Equation.

We can now write this basic performance equation in terms of instruction count (the number of instructions executed by the program), CPI, and clock cycle time:

$$CPU \text{ time} = Instruction \text{ count} \times CPI \times Clock \text{ cycle time}$$

or, since the clock rate is the inverse of clock cycle time:

$$CPU \text{ time} = \frac{Instruction \text{ count} \times CPI}{Clock \text{ rate}}$$

These formulas are particularly useful because they separate the three key factors that affect performance.

The performance of a program depends on the algorithm, the language, the compiler, the architecture, and the actual hardware. The following table summarizes how these components affect the factors in the CPU performance equation.

Hardware or software component	Affects what?	How?
Algorithm	Instruction count, possibly CPI	The algorithm determines the number of source program instructions executed and hence the number of processor instructions executed. The algorithm may also affect the CPI, by favoring slower or faster instructions. For example, if the algorithm uses more floating-point operations, it will tend to have a higher CPI.
Programming language	Instruction count, CPI	The programming language certainly affects the instruction count, since statements in the language are translated to processor instructions, which determine instruction count. The language may also affect the CPI because of its features; for example, a language with heavy support for data abstraction (e.g., Java) will require indirect calls, which will use higher CPI instructions.
Compiler	Instruction count, CPI	The efficiency of the compiler affects both the instruction count and average cycles per instruction, since the compiler determines the translation of the source language instructions into computer instructions. The compiler's role can be very complex and affect the CPI in complex ways.
Instruction set architecture	Instruction count, clock rate, CPI	The instruction set architecture affects all three aspects of CPU performance, since it affects the instructions needed for a function, the cost in cycles of each instruction, and the overall clock rate of the processor.

Problem 4:

The table shows the two-code sequence with number of instructions of different instruction classes within each code sequence respectively. The instruction are classified as A, B and C according to the CPI as shown in table.

CPI	A	B	C
	1	2	3

Code sequence Instruction counts for each instruction class is Code sequence Instruction counts for each instruction class

Code sequence	Instruction Count		
	A	B	C
1	2	1	2
2	4	1	1

- Determine which code sequence executes the most instructions.
- Determine which code sequence will execute quickly.
- Determine the CPI for each code sequence

(i) Code sequence 1 executes $2+1+2 = 5$ instructions

Code sequence 1 executes $4+1+1 = 6$ instructions

Code sequence 1 executes fewer instructions

(ii) $CPU \text{ Clock cycles} = \sum_1^n CPI_i x I_i$

$CPU \text{ clock cycles}_1 = (1 * 2) + (2 * 1) + (3 * 2) = 10 \text{ cycles}$

$CPU \text{ clock cycles}_3 = (1 * 4) + (2 * 1) + (3 * 1) = 9 \text{ cycles}$

So code sequence 2 is faster even though it executes more instructions.

(iii)

$$CPI = \frac{CPU \text{ clock cycles}}{Instruction \text{ count}}$$

$$CPI_1 = \frac{CPU \text{ Clock cycles}_1}{Instruction \text{ count}_1} = \frac{10}{5} = 2.0$$

$$CPI_2 = \frac{CPU \text{ Clock cycles}_2}{Instruction \text{ count}_2} = \frac{9}{6} = 1.5$$

INSTRUCTIONS: LANGUAGE OF THE COMPUTER – OPERATIONS AND OPERANDS

Operations and Operands

The words of a computer's language are called instructions, and its vocabulary is called an instruction set. We have chosen MIPS Technologies, which is an elegant example of the instruction set designed since 1980s.

Operations of the Computer Hardware

Every computer must be able to perform arithmetic. The MIPS assembly language notation
add a, b, c

instructs a computer to add the two variables b and c and to put their sum in a. For example, suppose we want to place the sum of four variables b, c, d, and e into variable a.

The following sequence of instructions adds the four variables:

add a, b, c # The sum of b and c is placed in a

add a, a, d # The sum of b, c, and d is now in a

add a, a, e # The sum of b, c, d, and e is now in a

Instructions are kept in the computer as a series of high and low electronic signals and may be represented as numbers.

Operands

In MIPS, data must be in registers to perform arithmetic operations

32 Registers

Symbolic Name	Number	Usage
\$zero	0	Constant 0
\$at	1	Reserved for the assembler
\$v0 - \$v1	2-3	Result Registers
\$a0 - \$a3	4 - 7	Argument Registers 1 . . . 4.
\$t0 - \$t9	8 - 15, 24 - 25	Temporary Registers 0 . . . 9.
\$s0 - \$s7	16 - 23	Saved Registers 0 . . . 7.
\$k0 - \$k1	26 - 27	Kernel Registers 0 . . . 1.
\$gp	28	Global Data Pointer.
\$sp	29	Stack Pointer.
\$fp	30	Frame Pointer.
\$ra	31	Return Address.

1. Register Operands

- Arithmetic instructions use register operands
- MIPS has a 32 × 32-bit register file
- Use for frequently accessed data
- Numbered 0 to 31
- 32-bit data called a “word”

- Assembler names
- \$t0, \$t1, ..., \$t9 for temporary values
- \$s0, \$s1, ..., \$s7 for saved variables

Example 1:

C code for the following instruction:

```
f = (g + h) - (i + j);
```

```
f, ..., j in $s0, ..., $s4
```

Compiled MIPS code:

```
add $t0, $s1, $s2
```

```
add $t1, $s3, $s4
```

```
sub $s0, $t0, $t1
```

2. Memory Operands

- Main memory used for composite data
- Arrays, structures, dynamic data
- To apply arithmetic operations
- Load values from memory into registers
- Store result from register to memory
- Memory is byte addressed
- Each address identifies an 8-bit byte
- Words are aligned in memory
- Address must be a multiple of 4
- MIPS is Big Endian
- Most-significant byte at least address of a word
- c.f. Little Endian: least-significant byte at least address

3. Constant or Immediate Operands

- Constant data specified in an instruction
- `addi $s3, $s3, 4`
- No subtract immediate instruction
- Just use a negative constant

```
addi $s2, $s1, -1
```

Operations

1. Arithmetic Operations

Add and Subtract instruction use three Operands - Two Sources and one Destination.

Example `add $t0, $s1, $s2`

2. Data Transfer Operations

These operations help in moving the data between memory and registers.

Example 1:	Example 2:
C code: <pre>g = h + A[8];</pre> • g in \$s1, h in \$s2, base address of A in \$s3 Compiled MIPS code: • Index 8 requires offset of 32 • 4 bytes per word <pre>lw \$t0, 32(\$s3) # load word add \$s1, \$s2, \$t0</pre> offset base register	C code: <pre>A[12] = h + A[8];</pre> • h in \$s2, base address of A in \$s3 Compiled MIPS code: • Index 8 requires offset of 32 <pre>lw \$t0, 32(\$s3) # load word add \$t0, \$s2, \$t0 sw \$t0, 48(\$s3) # store word</pre>

3. Logical Operations

Instructions for bitwise manipulation

Useful for extracting and inserting groups of bits in a word

Operation	C	MIPS
Shift left	<<	sll
Shift right	>>	srl
Bitwise AND	&	and, andi
Bitwise OR		or, ori
Bitwise NOT	~	nor

4. Conditional Operations

1. Branch to a labeled instruction if a condition is true. Otherwise, continue sequentially.

beq rs, rt, L1

if (rs == rt) branch to instruction labeled L1;

bne rs, rt, L1

if (rs != rt) branch to instruction labeled L1;

j L1

2. Unconditional jump to instruction labeled L1. Set result to 1 if a condition is true. Otherwise, set to 0.

slt rd, rs, rt

if (rs < rt) rd = 1; else rd = 0;

slti rt, rs, constant

if (rs < constant) rt = 1; else rt = 0;

3. Use in combination with beq, bne

slt \$t0, \$s1, \$s2 # if (\$s1 < \$s2)

bne \$t0, \$zero, L # branch to L

Jump Instructions

4. Procedure call: jump and link.

jal ProcedureLabel

Address of following instruction put in \$ra.

Jumps to target address.

5. Procedure return: jump register

jr \$ra

Copies \$ra to program counter.

Can also be used for computed jumps.

e.g., for case/switch statements.

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	add \$s1,\$s2,\$s3	$\$s1 = \$s2 + \$s3$	Three register operands
	subtract	sub \$s1,\$s2,\$s3	$\$s1 = \$s2 - \$s3$	Three register operands
	add immediate	addi \$s1,\$s2,20	$\$s1 = \$s2 + 20$	Used to add constants
Data Transfer	load word	lw \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	Word from memory to register
	store word	sw \$s1,20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1$	Word from register to memory
	load half	lh \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	Halfword memory to register
	load half unsigned	lhu \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	Halfword memory to register
	store half	sh \$s1,20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1$	Halfword register to memory
	load byte	lb \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	Byte from memory to register
	load byte unsigned	lbu \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	Byte from memory to register
	store byte	sb \$s1,20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1$	Byte from register to memory
	load linked word	ll \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	Load word as 1st

				half of atomic swap
	store condition.	word sc \$s1,20(\$s2)	Memory[\$s2+20]=\$s1;\$s1 =0 or 1	Store word as 2nd half of atomic swap
	load upper immediate	lui \$s1,20	\$s1 = 20 * 216	Loads constant in upper 16 bits
Logical	and	and \$s1,\$s2,\$s3	\$s1 = \$s2 & \$s3	Three reg. operands; bit-by-bit AND
	or	or \$s1,\$s2,\$s3	\$s1 = \$s2 \$s3	Three reg. operands; bit-by-bit OR
	nor	nor \$s1,\$s2,\$s3	\$s1 = ~ (\$s2 \$s3)	Three reg. operands; bit-by-bit NOR
	and immediate	andi \$s1,\$s2,20	\$s1 = \$s2 & 20	Bit-by-bit AND reg with constant
	or immediate	ori \$s1,\$s2,20	\$s1 = \$s2 20	Bit-by-bit OR reg with constant
	shift left logical	sll \$s1,\$s2,10	\$s1 = \$s2 << 10	Shift left by constant
	shift right logical	srl \$s1,\$s2,10	\$s1 = \$s2 >> 10	Shift right by constant
Conditional branch	branch on equal	beq \$s1,\$s2,25	if (\$s1 == \$s2) go to PC + 4 + 100	Equal test; PC- relative branch
	branch on not equal	bne \$s1,\$s2,25	if (\$s1 != \$s2) go to PC + 4 + 100	Not equal test; PC- relative
	set on less than	slt \$s1,\$s2,\$s3,	if (\$s2 < \$s3) \$s1 = 1; else \$s1 = 0	Compare less than; for beq bne
	set on less than unsigned	sltu \$s1,\$s2,\$ s3	if (\$s2 < \$s3) \$s1 = 1; else \$s1 = 0	Compare less than unsigned
	set less than immediate	slti \$s1,\$s2,20	if (\$s2 < 20) \$s1 = 1; else \$s1 = 0	Compare less than constant
	set less than immediate unsigned	sltiu \$s1,\$s2,20	if (\$s2 < 20) \$s1 = 1; else \$s1 = 0	Compare less than constant unsigned
Unconditiona l jump	jump	j 2500	go to 10000	Jump to target address
	jump register	jr \$ra	go to \$ra	For switch, procedure return
	jump and link	jal 2500	\$ra = PC + 4; go to 10000	For procedure call

REPRESENT INSTRUCTIONS IN A COMPUTER SYSTEM

Instructions are encoded in binary Called machine code.

MIPS instructions:

Encoded as 32-bit instruction words.

Small number of formats encoding operation code (opcode), register numbers.

- Register numbers
\$t0 – \$t7 are reg's 8 – 15
\$t8 – \$t9 are reg's 24 – 25
\$s0 – \$s7 are reg's 16 – 23

1. R-Format:

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Instruction Fields:

op: operation code (opcode)
rs: first source register number
rt: second source register number
rd: destination register number

shamt: shift amount (00000 for now)
 funct: function code (extends opcode)

Example:

add \$t0, \$s1, \$s2

special	\$s1	\$s2	\$t0	0	add
0	17	18	8	0	32
000000	10001	10010	01000	00000	100000

00000010001100100100000000100000₂ = 02324020₁₆

2. I-Format:

op	rs	rt	constant or address
6 bits	5 bits	5 bits	16 bits

Immediate arithmetic and load/store instructions.

Instruction Fields:

rt: destination or source register number

Constant: -215 to +215 - 1

Address: offset added to base address in rs

Example:

lw \$t0,32(\$s3) # Temporary reg \$t0 gets A[8]

Here, 19 (for \$s3) is placed in the rs field, 8 (for \$t0) is placed in the rt field, and 32 is placed in the address field. Note that the meaning of the rt field has changed for this instruction: in a load word instruction, the rt field specifies the destination register, which receives the result of the load.

Instruction	Format	op	rs	rt	rd	shamt	funct	address
add	R	0	reg	reg	reg	0	32 _{fun}	n.a.
sub (subtract)	R	0	reg	reg	reg	0	34 _{fun}	n.a.
add immediate	I	8 _{fun}	reg	reg	n.a.	n.a.	n.a.	constant
lw (load word)	I	35 _{fun}	reg	reg	n.a.	n.a.	n.a.	address
sw (store word)	I	43 _{fun}	reg	reg	n.a.	n.a.	n.a.	address

In the table above, “reg” means a register number between 0 and 31, “address” means a 16-bit address, and “n.a.”(not applicable) means this field does not appear in this format. Note that add and sub instructions have the same value in the op field; the hardware uses the function field to decide the variant of the operation: add (32) or subtract (34).

3. J-Format:

j target- J-type is short for "jump type". The format of an J-type instruction looks like:

opcode	target
B31-26	B25-0

The semantics of the j instruction (j means jump) are:

PC ← PC31-28 IR25-0 00

where PC is the program counter, which stores the current address of the instruction being executed. You update the PC by using the upper 4 bits of the program counter, followed by the 26 bits of the target (which is the lower 26 bits of the instruction register), followed by two 0's, which creates a 32 bit address.

LOGICAL OPERATIONS

1. Logical Operations:

Logical operations	C operators	Java operators	MIPS instructions
Shift left	<<	<<	sll
Shift right	>>	>>>	srl
Bit-by-bit AND	&	&	and, andi
Bit-by-bit OR			or, ori
Bit-by-bit NOT	~	~	nor

2. SHIFT:

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

shamt: how many positions to shift
 Shift left logical
 Shift left and fill with 0 bits
 sll by i bits multiplies by 2^i
 Shift right logical
 Shift right and fill with 0 bits
 srl by i bits divides by 2^i (unsigned only)

3. AND:

Useful to mask bits in a word
 Select some bits, clear others to 0
 Eg: and \$t0, \$t1, \$t2

\$t2	0000 0000 0000 0000 0000 1101 1100 0000
\$t1	0000 0000 0000 0000 0011 1100 0000 0000
\$t0	0000 0000 0000 0000 0000 1100 0000 0000

4. OR:

Used to include bit in a word
 Set some bits to 1, leave others unchanged
 Eg: or \$t0, \$t1, \$t2

\$t2	0000 0000 0000 0000 0000 1101 1100 0000
\$t1	0000 0000 0000 0000 0011 1100 0000 0000
\$t0	0000 0000 0000 0000 0011 1101 1100 0000

5. NOT:

Useful to invert bits in a word
 Change 0 to 1, and 1 to 0

\$t1	0000 0000 0000 0000 0011 1100 0000 0000
\$t0	1111 1111 1111 1111 1100 0011 1111 1111

DECISION MAKING

Based on the input data and the values created during computation, different instructions execute. Decision making is commonly represented in programming languages using if statement, sometimes combined with go to statements and labels. MIPS assembly language includes two decision-making instructions, similar to an if statement with a go to. The first instruction is

beq register1, register2, L1

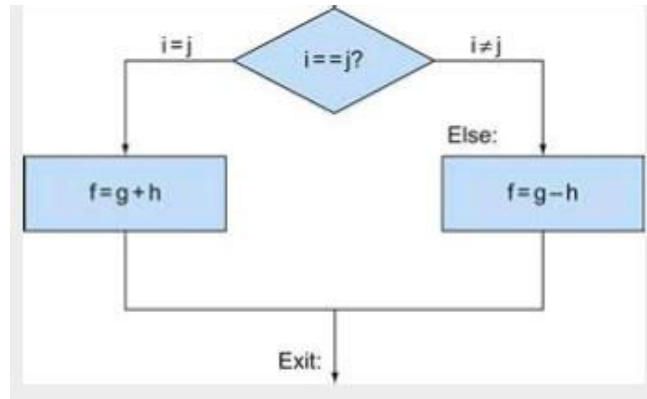
This instruction means go to the statement labeled L1 if the value in register1 equals the value in register2. The mnemonic beq stands for branch if equal. The second instruction is

bne register1, register2, L1

It means go to the statement labeled L1 if the value in register1 does not equal the value in register2. The mnemonic bne stands for branch if not equal. These two instructions are traditionally called conditional branches.

Example:

Compiling IF statements:



C code:

```
if (i==j) f = g+h;
else f = g-h;
```

• f, g, ... in \$s0, \$s1, ...

Compiled MIPS code:

```
    bne $s3, $s4, Else
    add $s0, $s1, $s2
    j    Exit
Else: sub $s0, $s1, $s2
Exit: ...
```

Assembler calculates addresses

Compiling Loop Statements:

C code:

```
while (save[i] == k) i += 1;
```

• i in \$s3, k in \$s5, address of save in \$s6

Compiled MIPS code:

```
Loop: sll $t1, $s3, 2
      add $t1, $t1, $s6
      lw  $t0, 0($t1)
      bne $t0, $s5, Exit
      addi $s3, $s3, 1
      j    Loop
Exit: ...
```

More Conditional operations:

Set result to 1 if a condition is true

Otherwise, set to 0

slt rd, rs, rt

if (rs < rt) rd = 1; else rd = 0;

slti rt, rs, constant

if (rs < constant) rt = 1; else rt = 0;

Use in combination with beq, bne

slt \$t0, \$s1, \$s2 # if (\$s1 < \$s2)

bne \$t0, \$zero, L # branch to L

Signed comparison: slt, slti

Unsigned comparison: sltu, sltui

Example

\$s0 = 1111 1111 1111 1111 1111 1111 1111 1111

\$s1 = 0000 0000 0000 0000 0000 0000 0000 0001

slt \$t0, \$s0, \$s1 # signed

-1 < +1 ⇒ \$t0 = 1

sltu \$t0, \$s0, \$s1 # unsigned

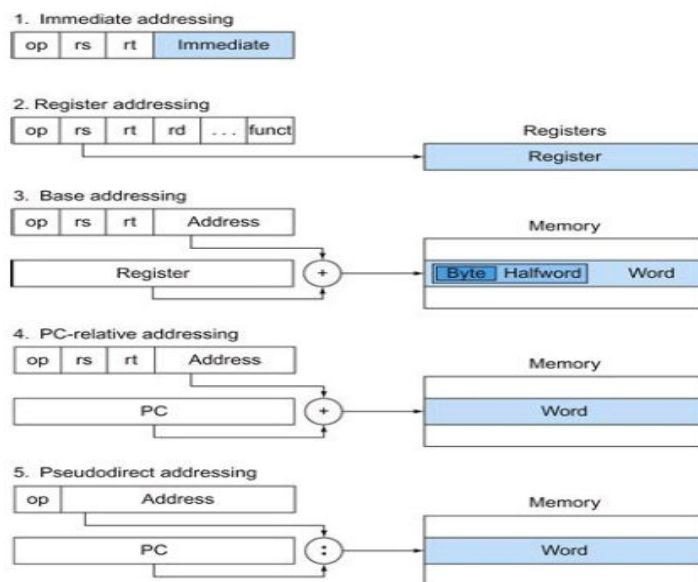
+4,294,967,295 > +1 ⇒ \$t0 = 0

MIPS ADDRESSING

Multiple forms of addressing are generically called as addressing modes. The method used to identify the location of an operand. The following are the various types of MIPS addressing modes.

1. **Immediate addressing**, where the operand is a constant within the instruction itself.
2. **Register addressing**, where the operand is a register.
3. **Base or displacement addressing**, where the operand is at the memory location whose address is the sum of a register and a constant in the instruction.
4. **PC-relative addressing**, where the branch address is the sum of the PC and a constant in the instruction.
5. **Pseudo direct addressing**, where the jump address is the 26 bits of the instruction concatenated with the upper bits of the PC.

The operands are shaded in color. The operand of mode 3 is in memory, whereas the operand for mode 2 is a register. Note that versions of load and store access bytes, half words, or words. For mode 1, the operand is 16 bits of the instruction itself. Modes 4 and 5 address instructions in memory, with mode 4 adding a 16-bit address shifted left 2 bits to the PC and mode 5 concatenating a 26-bit address shifted left 2 bits with the 4 upper bits of the PC.



UNIT II DATA REPRESENTATION AND ARITHMETIC FOR COMPUTERS

Signed Number Representation, Addition and Subtraction – Multiplication – Division – Fixed and Floating Point Representation – Floating Point Operations.

Signed Number Representation :

A signed integer is an integer with a positive '+' or negative sign '-' associated with it. Since the computer only understands binary, it is necessary to represent these signed integers in binary form.

In binary, signed Integer can be represented in three ways:

1. Signed bit.
2. 1's Complement.
3. 2's Complement.

Let us see why 2's complement is considered to be the best method.

Signed bit Representation

In the signed integer representation method the following rules are followed:

1. The MSB (Most Significant Bit) represents the sign of the Integer.
2. Magnitude is represented by other bits other than MSB i.e. (n-1) bits where n is the no. of bits.
3. If the number is positive, MSB is 0 else 1.
4. The range of signed integer representation of an n-bit number is given as $-(2^{n-1}-1)$ to $(2^{n-1}-1)$.

Example:

Let $n = 4$

UNIT II ARITHMETIC FOR COMPUTERS

ALU - Addition and subtraction – Multiplication – Division – Floating Point Representations - Floating Point operations – Subword parallelism

INTRODUCTION

ALU is responsible for performing arithmetic operations such as add, subtract, division and multiplication and logical operation such as AND, OR, Inverting etc. Arithmetic operation to be performed is based on data type. Two basic data types:

1. Fixed point numbers
2. Floating point numbers

Fixed point number

It allows the representation of number positive or negative integer numbers. The length of 1, 2, 4 or more bytes.

Floating point numbers

It allows the representation of number having both integer part and fractional part. The length of single precision (4 bytes) or double precision (8 bytes).

Big-Endian and Little-Endian Assignments

Two ways that byte addresses can be assigned across words

1. Big-Endian Assignments:

When lower byte addresses are used for the more significant bytes (leftmost bytes) of the word.

2. Little- Endian Assignments:

When lower byte addresses are used for the less significant bytes (rightmost bytes) of the word.

Fixed point number represent in two forms:

1. unsigned integer: (It represent positive number)
2. signed integer: (It represent negative number)

Signed magnitude representation:

The leftmost bit is represents sign of the number. The MSB(most significant bit) represents sign of the number. If MSB is 1, number is negative. If MSB is 0, number is positive.

Example:

$$10 = 0\ 000\ 1010$$

$$-6 = 1\ 000\ 0110$$

Techniques to represent signed numbers are:

1's complement

2's complement

1's complement- Invert all bits

Example: Consider 10011110, the one's complement is 01100001

2's complement- Two's complement is one more than the one's complement

Example: find the two's complement of 1 0 0 1 1 1 0 → 01100010

BINARY ADDITION

In binary addition, digits are added bit by bit from right to left carries passed to the next digit to the left.

Problem 1

Add $7_{10} + 6_{10}$ in binary

$$\begin{array}{r}
 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0111_{\text{two}} = 7_{\text{ten}} \\
 + \quad 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0110_{\text{two}} = 6_{\text{ten}} \\
 \hline
 = \quad 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1101_{\text{two}} = 13_{\text{ten}}
 \end{array}$$

BINARY SUBTRACTION

Subtraction of binary numbers can be done by adding the (2's complement + 1) of the second number to the first number. Binary subtraction is just binary addition of a negative number.

Problem 2

Subtract $7_{10} - 6_{10}$ in binary

$$\begin{array}{r}
 +7: \quad 0000\ 0000 \dots 0000\ 0111 \\
 -6: \quad 1111\ 1111 \dots 1111\ 1010 \\
 \hline
 +1: \quad 0000\ 0000 \dots 0000\ 0001
 \end{array}$$

BIT PAIR RECORDING ALGORITHM

Bit-pair recoding is the product of the multiplier results in using at most one summand for each pair of bits in the multiplier. It is derived directly from the Booth algorithm. Grouping the Booth-recoded multiplier bits in pairs will decrease the multiplication only by $n/2$ summands.

Multiplier bit pair		Multiplier bit on the right	Multiplicand selected at position i
i+1	i	i-1	
0	0	0	0
0	0	1	+1
0	1	0	+1
0	1	1	+2
1	0	0	-2
1	0	1	-1
1	1	0	-1
1	1	1	0

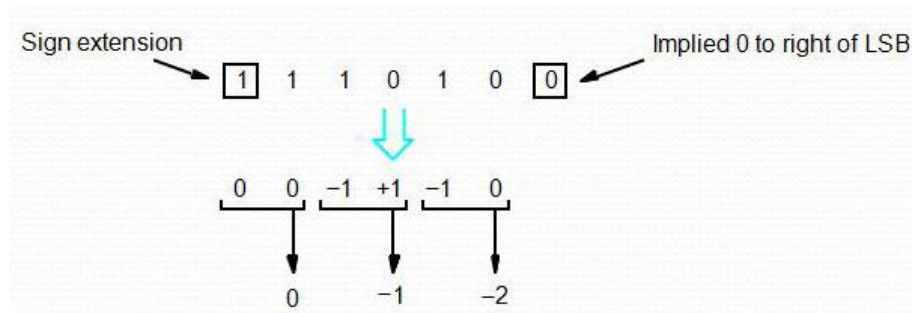
$$0 \times M = 0$$

$$1 \times M = M$$

$$2 \times M = 10 \times M$$

$$-1 \times M = 2' \text{ complement of } M$$

$$-2 \times M = 2' \text{ complement of } M \times 10$$



Problem 3

Multiply 13x-6 using bit pair recording algorithm

13→1101

6→0110

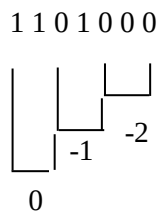
Add the sign bit

13→01101 (Multiplicand)

6→00110

-6→11010 (Multiplier)

Recode the multiplier



						0	1	1	0	1
						0		-1		-2
1	1	1	1	1	0	0	1	1	0	
1	1	1	1	0	0	1	1			
0	0	0	0	0	0					
1	1	1	0	1	1	0	0	1	0	

Verification

As the result is negative, take 2's complement of the product.

0001001110→78

Hence, the answer is correct.

Problem 4

Multiply 13x6 using bit pair recording algorithm

13→1101

6→0110

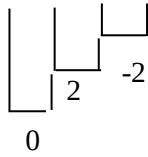
Add the sign bit

13→01101 (Multiplicand)

6→00110 (Multiplier)

Recode the multiplier

0 0 0 1 1 0 0



					0	1	1	0	1
					0		2		-2
1	1	1	1	1	0	0	1	1	0
0	0	0	1	1	0	1	0		
0	0	0	0	0	0				
0	0	0	1	0	0	1	1	1	0

Verification

0001001110 → 78

Hence, the answer is correct.

Problem 5

Multiply -13x-20 using bit pair recording algorithm

13 → 01101

20 → 10100

Add the sign bit

13 → 001101

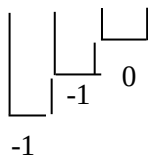
20 → 010100

-13 → 110011 (Multiplicand)

-20 → 101100 (Multiplier)

Recode the multiplier

1 0 1 1 0 0 0



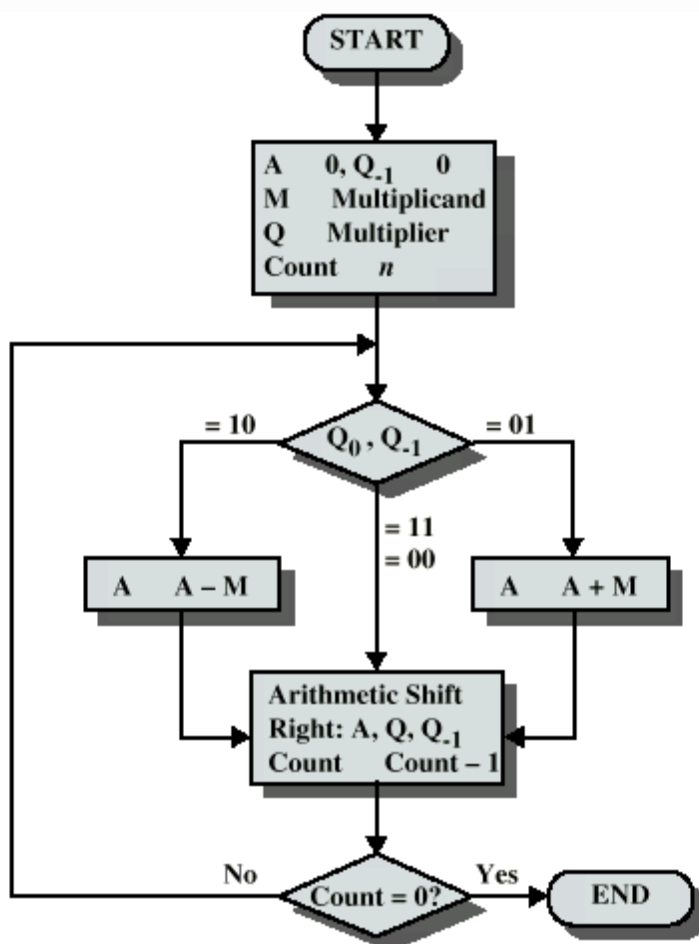
BOOTH'S MULTIPLICATION ALGORITHM

Step 1: First the registers are initialized with corresponding values (Q = Multiplier, M = Multiplicand, $A = 0$, $\text{Count} = \text{number of bits in multiplier } (n)$, $Q_{-1}=0$)

Step 2: Check the Q_0 and Q_{-1} bits. When two bits Q_0 and Q_{-1} are 00 or 11, perform arithmetic shift right operation to A , Q and Q_{-1} by 1 bit position. Decrement the count by 1.

- If the bits of Q_0 and Q_{-1} is 01, the multiplicand bits (M) will be added to the Accumulator (A) and the results are stored in Accumulator. After that perform the right shift operation to the A , Q and Q_{-1} by 1 bit position. Decrement the count by 1.
- If the bits of Q_0 and Q_{-1} is 10, the multiplicand bits (M) will be subtracted from the Accumulator (A) and the results are stored in Accumulator. After that perform the right shift operation to the A , Q and Q_{-1} by 1 bit position. Decrement the count by 1.

Step 3: Repeat step 2, until count is equal to 0. Otherwise, stop the process and the result of the multiplication will be stored in the A and Q registers.



Problem 7:**Multiply 7x3 using Booths Algorithm**

7 → 111

3 → 011

Add Sign bits

7 → 0111 (M)

3 → 0011 (Q)

Iteration	A	Q	Q ₋₁	Operation
0	0000	0011	0	Initial
1	1001	0011	0	A = A - M
	1100	1001	1	Arithmetic Shift right
2	1110	0100	1	Arithmetic Shift right
3	0101	0100	1	A = A + M
	0010	1010	0	Arithmetic Shift right
4	0001	0101	0	Arithmetic Shift right

Product = 0001 0101**Problem 8:****Multiply -7x3 using Booths Algorithm**

7 → 111

3 → 011

Add Sign bits

7 → 0111

3 → 0011 (Q)

-7 → 1001

Iteration	A	Q	Q ₋₁	Operation
0	0000	0011	0	Initial
1	0111	0011	0	A = A - M
	0011	1001	1	Arithmetic Shift right
2	0001	1100	1	Arithmetic Shift right
3	1010	1100	1	A = A + M
	1101	0110	0	Arithmetic Shift right
4	1110	1011	0	Arithmetic Shift right

Product = 1110 1011

Problem 9:**Multiply -13x-20 using Booths Algorithm**

13→01101

20→10100

Add Sign bits

13→001101

20→010100

-13→110011 (M)

-20→101011 (Q)

Iteration	A	Q	Q ₋₁	Operation
0	000000	101100	0	Initial
1	000000	010110	0	Arithmetic Shift right
2	000000	001011	0	Arithmetic Shift right
3	001101	001011	0	A=A-M
	000110	100101	1	Arithmetic Shift right
4	000011	010010	1	Arithmetic Shift right
5	110110	010010	1	A=A+M
	111011	001001	0	Arithmetic Shift right
6	001000	001001	0	A=A-M
	000100	000100	1	Arithmetic Shift right

Product = 000100 000100**Problem 10:****Multiply 4x-10 using Booths Algorithm**

4→0100

10→1010

Add Sign bits

4→00100 (M)

10→01010

-10→10110 (Q)

Iteration	A	Q	Q ₋₁	Operation
0	00000	10110	0	Initial
1	00000	01011	0	Arithmetic Shift right
2	11100	01011	0	A=A-M
	11110	00101	1	Arithmetic Shift right
3	11111	00010	1	Arithmetic Shift right
4	00011	00010	1	A=A+M
	00001	10001	0	Arithmetic Shift right
5	11101	10001	0	A=A-M
	11110	11000	1	Arithmetic Shift right

Product = 11110 11000

Sequential Circuit Multiplication algorithm

A [sequential circuit](#) is a combinational circuit with memory. The output of the sequential circuit depends on present inputs and present state [past outputs]. The information stored in the sequential circuit represents the present state. The present state and present input will define the output of the next state.

Binary Multiplier

A binary multiplier is used to multiply two binary numbers. It is a basic electronic circuit in [digital electronics](#), such as a computer. The binary multiplier is also called an **add-shift adder**.

A digital multiplier can be implemented using a variety of computer arithmetic techniques. The majority of techniques involve computing a set of partial products, which are then summed using binary adders.

Adder: An [adder](#), also known as summer, is a digital circuit that performs number addition. Adders are used in the arithmetic logic units of many computers and other types of processors (ALUs).

Types of Multipliers

1. **Bit Multiplier 2x2:** This multiplier can multiply two numbers with bit size = 2, which means that both the multiplier and the multiplicand can be of 2 bits.
2. **Bit Multiplier 3x3:** This multiplier has a maximum bit size of 3 bits and can multiply two numbers. The product's bit size will be 6.
3. **Bit Multiplier 4x4:** This multiplier can multiply a 4-bit binary number and produce an 8-bit product because the bit size of the product equals the sum of the bit sizes of the multiplier and multiplicand.

Before going to sequential binary multiplication first let's see how normal binary multiplication works:

Binary Multiplication

Consider two binary numbers num1 and num2

num1=12 → which equivalent to binary value as 1100 → multiplier

num2=13 → which equivalent to binary value as 1101 → multiplicand

Example 1:

```
  1 1 0 0
x 1 1 0 1
-----
```

```

      1 1 0 0
    0 0 0 0
  1 1 0 0
1 1 0 0
-----
1 0 0 1 1 1 0 0

```

The above multiplication is performed by a sequential binary multiplier as below:

Operation:

Consider two binary numbers num1 and num2

Step 1:

num1=12 → which equivalent to a binary value as 1100

num2=13 → which equivalent to binary value as 1101

Assume multiplier as M and multiplicand as Q

Step 2:

Here we also need the other two parameters accumulator and carry and initially the values of both accumulator and carry will be zero.

Let,

Accumulator a = 0000

Carry c = 0

Step 3:

Q = 1 1 0 1

q3 q2 q1 q0

if q0=0 then perform an only right shift operation

if q0=1 then perform add (A+M) and right shift operation

Here we have to perform up to 4 steps because the number of bits in the multiplier is 4

Steps	M (Multiplier)	C (Carry)	A(Accumulator)	Q (Multiplicand)	Operation
Step 1	1100	0	0000	1 1 0 1 q3q2q1q0	initialization q0=1 then perform the next step as add and shift right operation
	1100	0	0000+ 1100 ----- 1100 0110	1101 0 1 1 0 q3q2q1q0	A+M right shift operation of carry, accumulator, and quotient and discard last value i.e. q0 after shifting q0=0 perform next step only right shift operation
	1100	0	0110 0011	0110 0011	right shift operation q0=1 perform next step add (A+M) and shift right operation
Step 3	1100	0	0011	0011	A+M

Steps	M (Multiplier)	C (Carry)	A(Accumulator)	Q (Multiplicand)	Operation
			1100 ----- 1111 0111	1001	right shift operation of carry, accumulator, and quotient and discard last value i.e. q0 after shifting q0=1 perform add (A+M) and shift right operation
Step 4	1100	1 ----	0111 1100 ----- 0011 ----- 1001	1001 ----- 1100	A+M right shift operation of carry, accumulator, and quotient and discard last value i.e. q0 after shifting Here is the final result of multiplication because we have to perform only 4 steps as number bits is 4 in multiplier

Step 4:

Result = Combination of accumulator value(A) and Q =>10011100

the equivalent value is 156 obtained from the formula

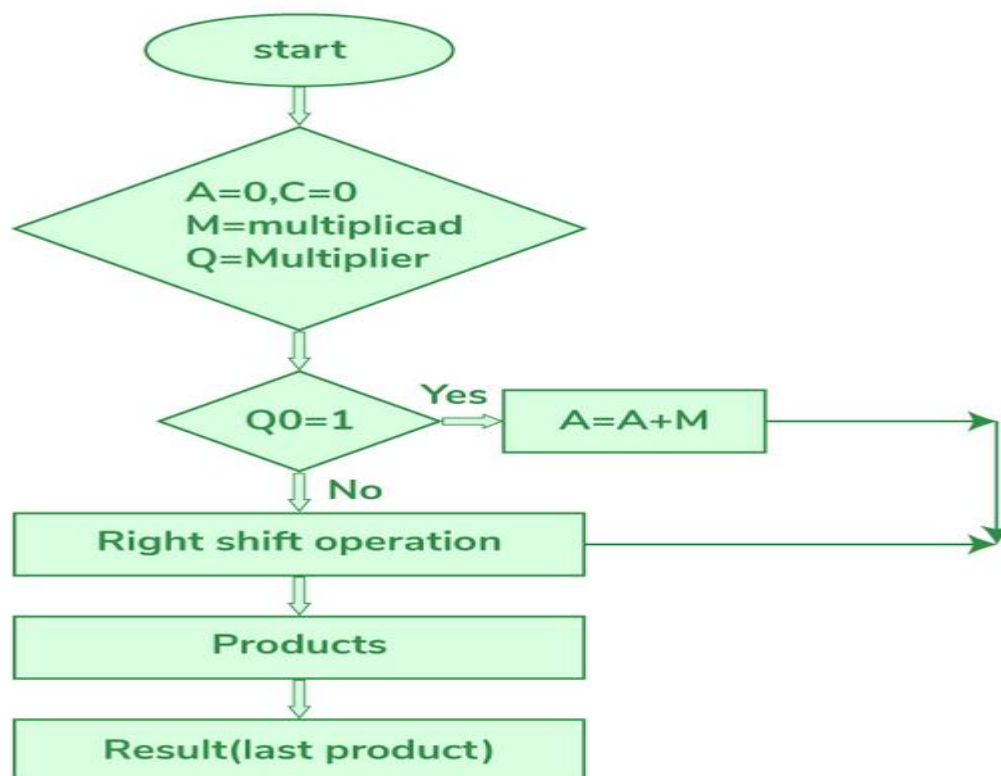
$$\dots\dots\dots+2^3+2^2+2^1+2^0$$

1 0 0 1 1 1 0 0

$$27+0+0+24+23+22+0+0 =156$$

Flow chart

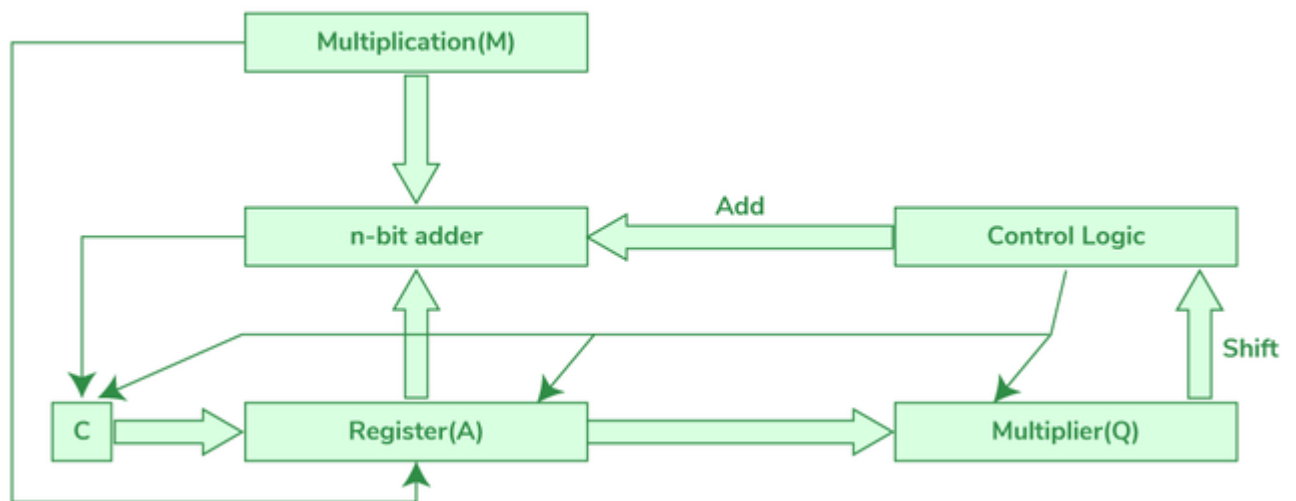
The flow chart explains the whole operation of the sequential binary multiplier in a simple manner. First, assign 0 to the accumulator and carry values. then check the LSB of Q i.e., Q_0 , if q_0 is 0 then perform only the right shift operation and if q_0 is 1 then perform the addition of the accumulator and multiplicand, store the result in the accumulator then perform the right shift operation. We have to continue this process based on the number of bits in the multiplier.



Flow chart of sequential binary multiplier

Hardware Circuit Diagram

The below diagram describes the hardware circuit of the sequential binary multiplier.



Hardware circuit diagram

Example 2: Here we have to perform up to 4 steps because the number of bits in the multiplier is 4.

Steps	M (Multiplier)	C (Carry)	A(Accumulator)	Q (Multiplicand)	Operation
Step1	0110	0	0000	1 1 1 0 q3q2q1q0	initialization q0=0 perform next step only right shift operation
		0	0000	0111	q0=1 then perform the next step as add and shift right operation
Step2	0110	0 0	0000+ 0110 -----	0111 0011	A+M right shift operation of, accumulator

Steps	M (Multiplier)	C (Carry)	A(Accumulator)	Q (Multiplicand)	Operation
			0110 0011		and quotient and discard last value i.e. q0 after shifting q0=1 then perform the next step as add and shift right operation
Step3	0110	0 0	0011 0110 ----- 1001 0100	0011 1001	A+M right shift operation of carry, accumulator, and quotient and discard last value i.e. q0 after shifting q0=1 then perform the next step as add and shift right operation

Steps	M (Multiplier)	C (Carry)	A(Accumulator)	Q (Multiplicand)	Operation
Step4	0110	0 0	0100	1001 0100	A+M
			0110		right shift operation of carry, accumulator, and quotient and discard last value i.e. q0 after shifting

			1010		Here is the final result of multiplication because we have to perform
			0101		only 4 steps as number bits are 4 in multiplier

Result:

Combination of accumulator value(A) and Q =>01010100

the equivalent value is 84 obtained from the formula

.....+ $2^3+2^2+2^1+2^0$

0 1 0 1 0 1 0 0

0+64+0+16+0+8+0+0 → 84

Right Shift Operation

Another procedure involved in binary multiplication when using a sequential binary multiplier is the right shift procedure; it is useful in the correct alignment

of the binary numbers to allow subsequent operations. Here's how the right shift operation is carried out and what it means in this context:

Concept of Right Shift

- **Logical Shift:** This entails moving all the bits to the right most side of the bar and placing a value of '0' on the extreme left side of the bar. This is normally employed where an unsigned number is being worked on.
- **Arithmetic Shift:** This involves a shifting of all bits to the right while the sign of the bit which is usually the left most in a signed number remains intact. For unsigned numbers this is essentially the same as a logical shift. Since, we are dealing with unsigned binary numbers in the case of sequential binary multipliers, we generally perform a logical shift.

How it works in Sequential Multiplication

Initial Setup: In multiplication you have two major components involved.

- **Accumulator (A):** Contains the intermediate sum outputs or the afterthoughts.
- **Multiplier (Q):** This one is actually holding the shifting format of the multiplicand in a binary format.

Right Shift Operation

Shift Right All Bits

- Every bit of the Accumulator (A) and the Multiplier (Q) is shifted one position in the right.
- The last bit of the Accumulator and the Multiplier is removed; the first bit of it is replaced by '0'.

Update Carry

- The term carry out that was shifted from the Accumulator or Multiplier often forms the new carry. It is utilized in the subsequent addition function.

Effect on Binary Numbers

- **For the Multiplier (Q):** In such a way, changing the place of the multiplier to the right contributes to preparation for the further work of the next LSB. When the LSB of the Q, that is, Q_0 is 1, then the current value of the Accumulator is added to the partial product accumulation. Finally after increment the whole binary number is shifted right.
- **For the Accumulator (A):** The Accumulator is shifted right with reference to its new position of the multiplier bits. This makes it ready for the next addition operation to take place.

Example

To explain it methodically, let's go through the step by step example, of a 4-bit sequence binary multiplier.

Initial Values:

- Multiplier (Q): 1101
- Multiplier (M): 1100
- Accumulator (A): 0000
- Carry (C): 0

Step-by-Step Right Shift Operation

1. Before Shift:

- A: 0000
- Q: 1101

Exercise the operation on the LSB of Q (Q₀). If Q₀ is 1, add (A + M), or (A + 0M depending on the proper treatment of the final carry during addition) to the accumulator and then shift.

Perform Addition

- Add (A + M): To sum 0000 and 1100 one gets 1100.
- Update Accumulator (A): 1100

Shift Right

- Right Shift Accumulator (A): 1100 becomes 0110
- Right Shift Multiplier (Q): 1101 is equal to 0110
- Carry (C): 0 (remains the same unless there was an overflow in addition)

Result After Shift

- A: 0110
- Q: 0110
- Carry (C): 0

This is done for the number of bits in the multiplier, thus, the number of bits in the product will be determined by the number of times this operation is done. In which for each bit in the multiplier, if the current is equal to one, performs the addition and store it in the accumulator along with shifting right.

Advantages of Sequential Binary Multiplier

- **Simplicity:** Sequential binary multipliers are not so complex and thus can easily be designed and implemented. They utilize elementary operations such as addition and shifting operation which are easily understandable and can also easily implemented in either in hardware or software.

- **Reduced Hardware Complexity:** There are a number of types of multipliers, but out of all of them sequential binary multipliers are notable for the fact that they use considerably fewer components as compared, for example, to parallel multipliers. They use fewer adders and logic gates because the operations are done one by one instead of all the operations at the same time.
- **Flexible Precision:** The sequential binary multipliers are designed in such a way that they can accommodate a change in the bit size. By changing the amount of steps, they are able to perform multiplication of numbers which have different length of bits, thus being able to handle a vast number of computations.
- **Ease of Integration:** In these multipliers, one can easily interface them to other bigger digital systems since they are comparatively simple. It integrates nicely with systems that already uses sequential logic for other processes.

Disadvantages of Sequential Binary Multiplier

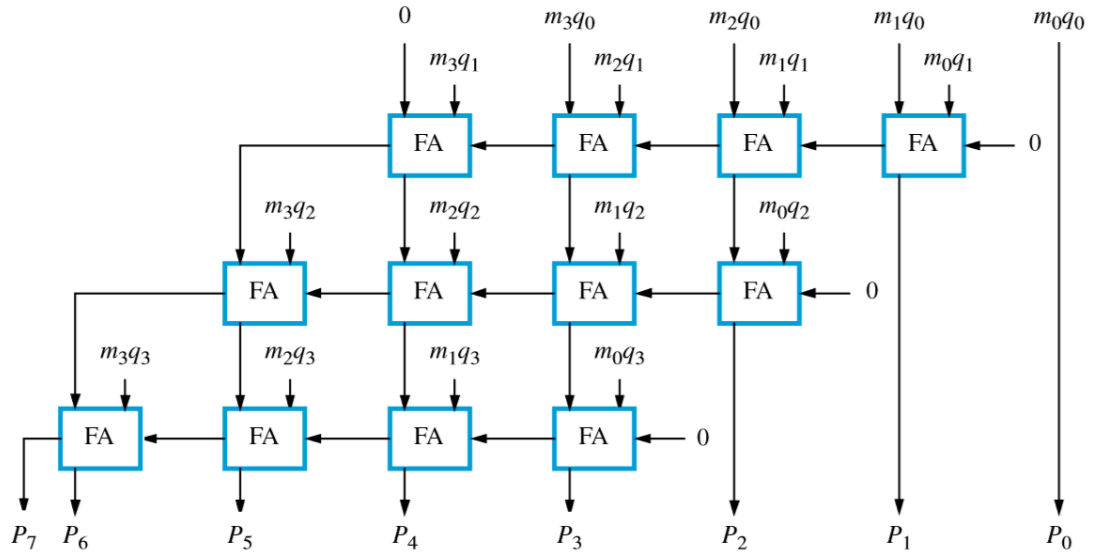
- **Speed:** Sequential binary multipliers are considered to be slower than parallel multipliers mainly because the former operates on the bits serial-wise. So it takes many clock cycles to compute each bit of the result and the entire multiplication may turn to be slow.
- **Sequential Nature:** Sequential operations mean that one has to wait for another to be carried out which is not efficient in high speed, applications that prefer parallel processing.
- **Complexity with Larger Bit Sizes:** It means that the more digits there are in the numbers which are to be multiplied, the more steps will be needed for the completion of the calculation. This can result in increased computation time as well a complicated control logic in the monitoring process.
- **Intermediate Storage Requirements:** An additional complication is introduced by use of an accumulator and carry registers which increases storage needs. There is unique an operational demand for additional storage space In case of large bit sizes, such needs may turn out to be substantial .
- **Educational Tools:** Sequential binary multipliers are simple and they are often employed in teaching on basic principles of Digital arithmetic and the design of Sequential Logic.

Applications of Sequential Binary Multiplier

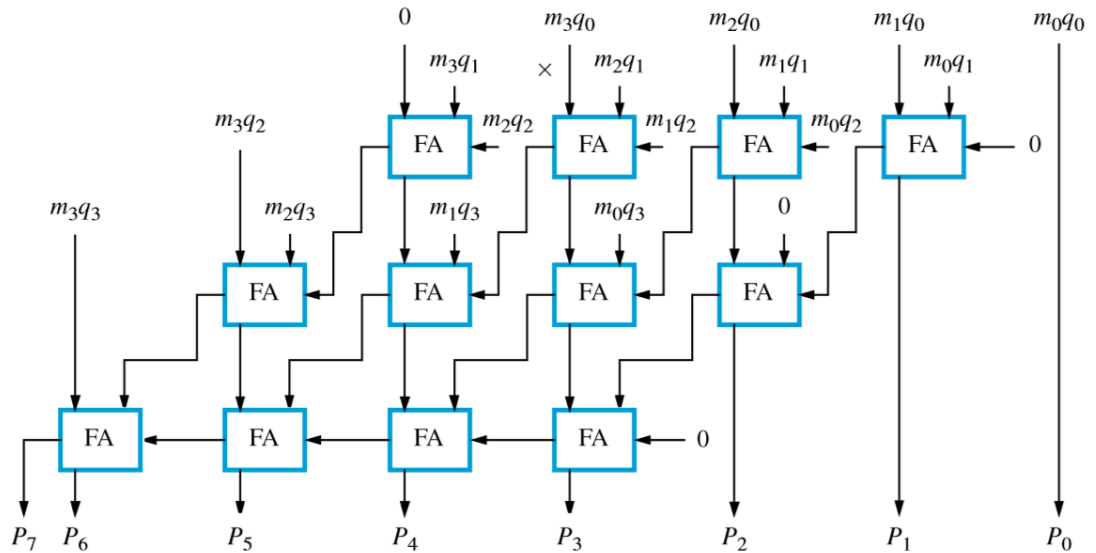
- **Embedded Systems:** Sequential binary multipliers are typically used in [embedded systems](#) resulting in both these benefits of less utilization of resources and simplicity. It is more versatile in the usage that is in [microcontrollers](#) and in digital signal processors where it can be used to multiply the small to medium-sized binary numbers.
- **Digital Signal Processing (DSP):** In [DSP](#) applications such as in the processing of signals, the operations that are usually done include multiplications and these are performed using sequential binary multipliers in algorithms like filtering and [Fourier transforms](#).
- **Arithmetic Logic Units (ALUs):** Sequential binary multipliers can be used for multiplication purposes in the ALU's of the processors. They are particularly effective for processors in which multiplication occurs fewer times than addition.
- **Educational Tools:** This is due to their simple structure and they can be used often in learning environment to teach basic arithmetic logic and sequential logic circuits.

CARRY-SAVE ADDITION Algorithm

1. Multiplication requires the addition of several summands. A technique called carry-save addition (CSA) can be used to speed up the process.



(a) Ripple-carry array



(b) Carry-save array

Figure 9.16 Ripple-carry and carry-save arrays for a 4×4 multiplier.

2. Consider the 4×4 multiplication array shown in Figure 9.16a. This structure is in the form of the array in which the first row consists of just the AND gates that produce the four inputs :: **m_3q_0 , m_2q_0 , m_1q_0 , and m_0q_0** .
3. Instead of letting the carries ripple along the rows, they can be “saved” and introduced into the next row, (at the correct weighted positions), as shown in Figure 9.16b.
4. This frees up an input to each of three full adders in the first row. These inputs can be used to introduce the third summand bits m_2q_2 , m_1q_2 , and m_0q_2 .
5. Now, two inputs (OUT OF 3 INPUTS OF FA) of each of three full adders in the second row are fed by the sum and carry outputs from the first row. The third input is used to introduce the bits :: **m_2q_3 , m_1q_3 , and m_0q_3** of the fourth summand.
6. The high-order bits : **m_3q_2 and m_3q_3** of the third and fourth summands are introduced into the remaining free full-adder inputs at the left end in the second and third rows.
7. The saved carry bits and the sum bits from the second row are now added in the third row, which is a ripple-carry adder, to produce the final product bits.
8. The delay through the carry-save array is somewhat less than the delay through the ripple-carry array. This is because the S and C vector outputs from each row are produced in parallel in one full-adder delay.

EXAMPLE

$$45 \times 63 = 2835$$

Division	Quotient	Remainder
$45 \div 2$	22	1
$22 \div 2$	11	0
$11 \div 2$	5	1
$5 \div 2$	2	1
$2 \div 2$	1	0
$1 \div 2$	0	1

$$45_{10} = 101101_2.$$

Division	Quotient	Remainder
$63 \div 2$	31	1
$31 \div 2$	15	1
$15 \div 2$	7	1
$7 \div 2$	3	1
$3 \div 2$	1	1
$1 \div 2$	0	1

$$63_{10} = 111111_2$$

So, the binary equivalent of $45 \times 63 = 2835$ is:

$$101100010011_2$$

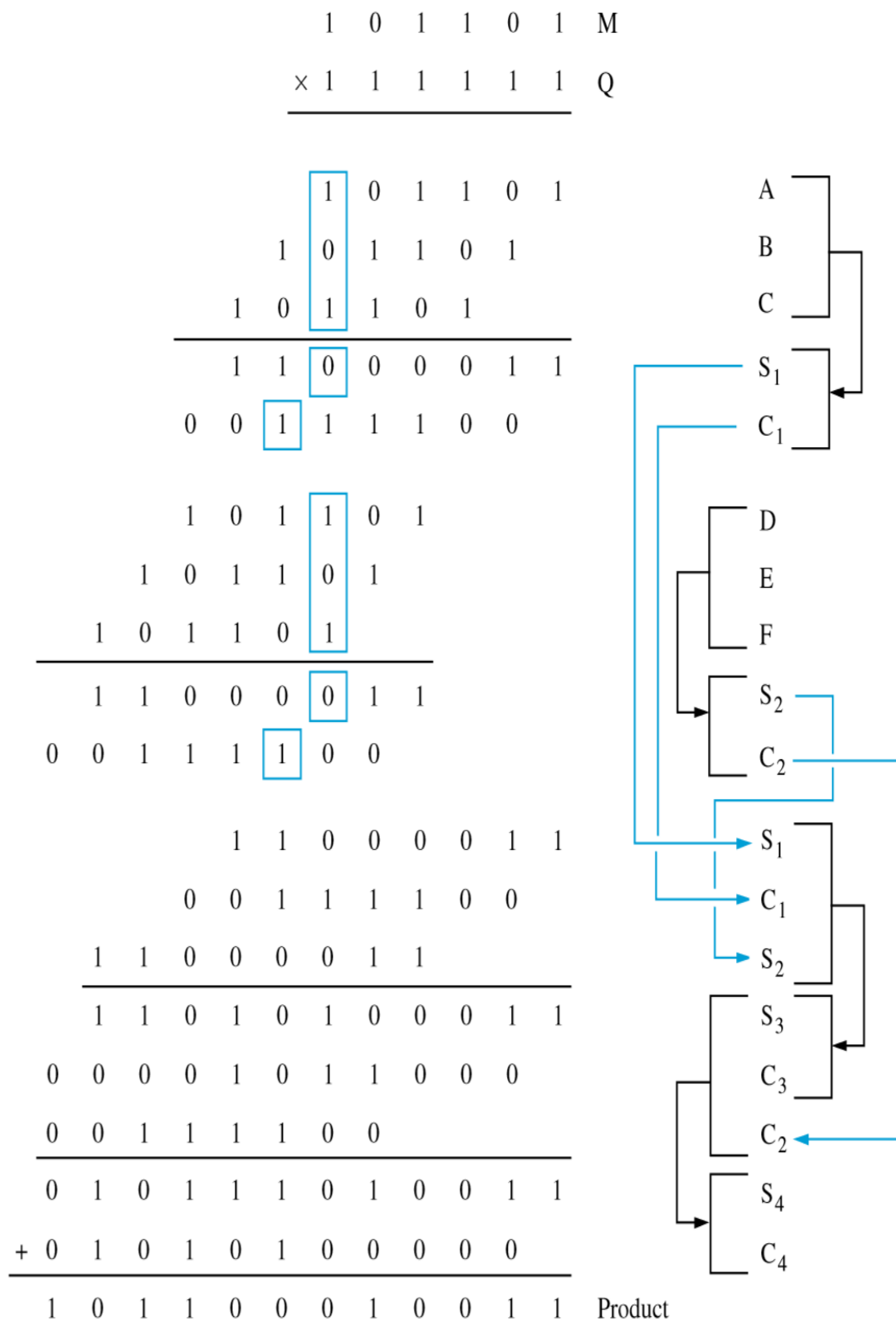


Figure 9.18 The multiplication example from Figure 9.17 performed using carry-save addition.

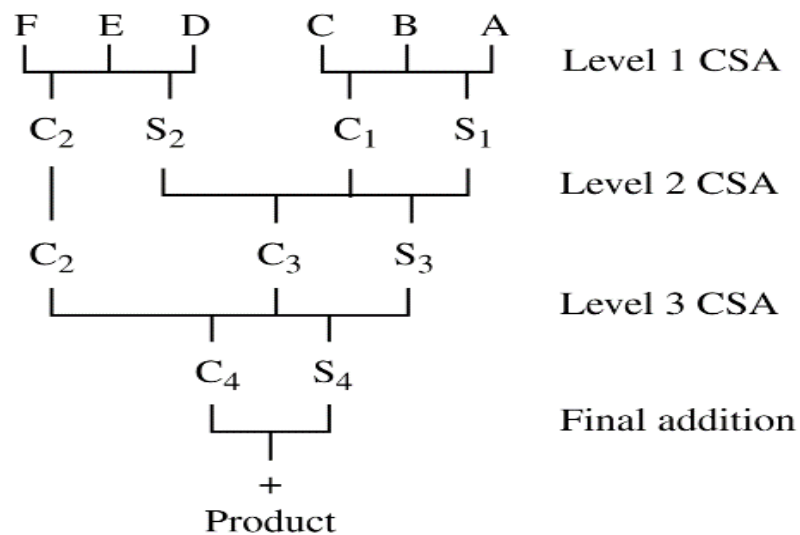


Figure 9.19 Schematic representation of the carry-save addition operations in Figure 9.18.

$$45 \times 63$$

$$45 \times 63 = 2835$$

binary equivalent

$$2835_{10} = 101100010011_2$$

So, the binary equivalent of $45 \times 63 = 2835$ is:

$$101100010011_2$$

Here we are going to multiply two 6-bit binary numbers						X	1	0	1	1	0	1	M
							1	1	1	1	1	1	Q
							1	0	1	1	0	1	A

						1	0	1	1	0	1		B
					1	0	1	1	0	1			C
					1	1	0	0	0	0	1	1	S1
				0	0	1	1	1	1	0	0		C1
				1	0	1	1	0	1				E
			1	0	1	1	0	1					F
		1	0	1	1	0	1						G
		1	1	0	0	0	0	1	1				S2
	0	0	1	1	1	1	0	0					C2
+					1	1	0	0	0	0	1	1	S1
				0	0	1	1	1	1	0	0		C1
		1	1	0	0	0	0	1	1				S2
+		1	1	0	1	0	1	0	0	0	1	1	S3
	0	0	0	0	1	0	1	1	0	0	0		C3
	0	0	1	1	1	1	0	0					C2
+	0	1	0	1	1	1	0	1	0	0	1	1	S4
	0	1	0	1	0	1	0	0	0	0	0		C4
	1	0	1	1	0	0	0	1	0	0	1	1	PRODUCT

101100010011₂

- 1) Group the summands in threes and perform carry-save addition on each of these groups in parallel to generate a set of S and C vectors in one full-adder delay.

- 2) Group all the S and C vectors into threes, and perform carry-save addition on them, generating a further set of S and C vectors in one more adder delay.
- 3) Continue with this process until there are only two vectors remaining.
- 4) The adder at each bit position of the three summands is called a 3-2 reducer, and the logic circuit structure that reduces a number of summands to two is called a CSA tree. The final two S and C vectors can be added in a carry-lookahead adder to produce the desired product.

When the number of summands is large, the time saved is proportionally much greater.

DIVISION ALGORITHM

RESTORING DIVISION ALGORITHM

It is usually performed on the fixed point fractional numbers. In this section we are going to perform restoring division with the help of unsigned numbers.

Step 1: First the registers are initialized with corresponding values (Q = Dividend, M = Divisor, $A = 0$, n = number of bits in dividend)

Step 2: The content of register A and Q is shifted right as if they are a single unit

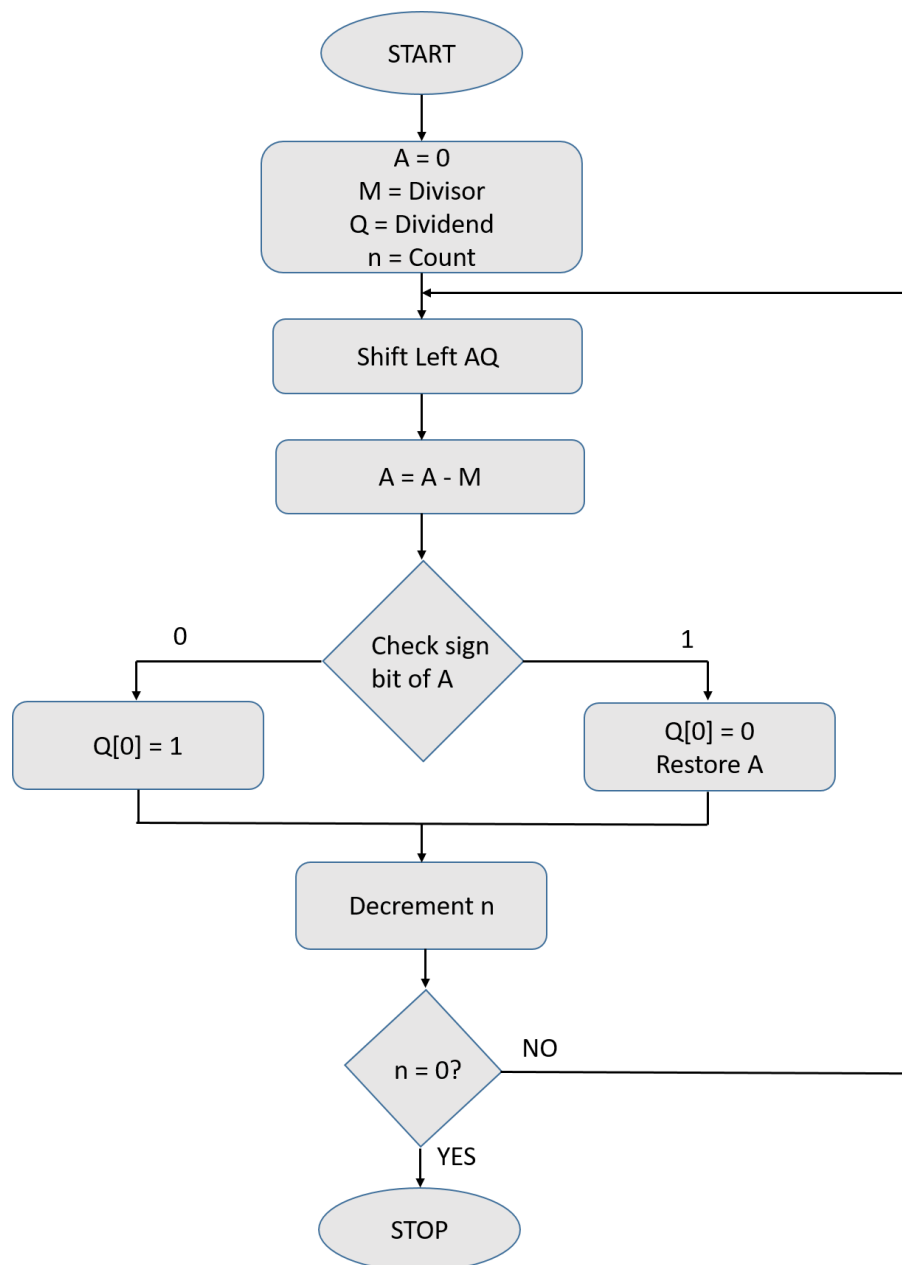
Step 3: The content of register M is subtracted from A and result is stored in A

Step 4: The most significant bit of the A is checked. If it is 0 the least significant bit of Q is set to 1. Otherwise if it is 1 the least significant bit of Q is set to 0 and value of register A is

Step 5: The value of counter n is decremented

Step 6: If the value of n becomes zero we get of the loop otherwise we repeat from step 2

Step 7: Finally, the register Q contain the quotient and A contain remainder



Problem 11

Divide $11 \div 3$ using restoring division algorithm

$11 \rightarrow 1011$ (Q)

$3 \rightarrow 0011$

N=4, therefore number of bits in A=5 bits and M=5 bits

$3 \rightarrow 00011$ (M)

Count (N)	A	Q	Operation
-	00000	1011	Initial
4	00001	011_	Shift Left AQ
	11110	011_	A=A-M
	00001	0110	$Q_0=0$, Restore A
3	00010	110_	Shift Left AQ
	11111	110_	A=A-M
	00010	1100	$Q_0=0$, Restore A
2	00101	100_	Shift Left AQ
	00010	100_	A=A-M
	00010	1001	$Q_0=1$
1	00101	001_	Shift Left AQ
	00010	001_	A=A-M
	00010	0011	$Q_0=1$

$A \rightarrow 00010$ (Remainder)

$Q \rightarrow 0011$ (Quotient)

Problem 12

Divide $25 \div 12$ using restoring division algorithm

$25 \rightarrow 11001$ (Q)

$12 \rightarrow 1100$

N=5, therefore number of bits in A=6 bits and M=6 bits

$12 \rightarrow 001100$ (M)

Count(N)	A	Q	Operation
-	000000	11001	Initial
5	000001	1001_	Shift Left AQ
	110101	1001_	A=A-M
	000001	10010	Q ₀ =0, Restore A
4	000011	0010_	Shift Left AQ
	110111	0010_	A=A-M
	000011	00100	Q ₀ =0, Restore A
3	000110	0100_	Shift Left AQ
	111010	0100_	A=A-M
	000110	01000	Q ₀ =0, Restore A
2	001100	1000_	Shift Left AQ
	000000	1000_	A=A-M
	000000	10001	Q ₀ =1
1	000001	0001_	Shift Left AQ
	110101	0001_	A=A-M
	000001	00010	Q ₀ =0, Restore A

A → 000001 (Remainder)

Q → 00010 (Quotient)

NON-RESTORING DIVISION ALGORITHM

Step 1: In this step, the corresponding value will be initialized to the registers, i.e., register A will contain value 0, register M will contain Divisor, register Q will contain Dividend, and N is used to specify the number of bits in dividend.

Step 2: In this step, we will check the sign bit of A.

Step 3: If this bit of register A is 1, then shift the value of AQ through left, and perform $A = A + M$. If this bit is 0, then shift the value of AQ into left and perform $A = A - M$. That means in case of 0, the 2's complement of M is added into register A, and the result is stored into A.

Step 4: Now, we will check the sign bit of A again.

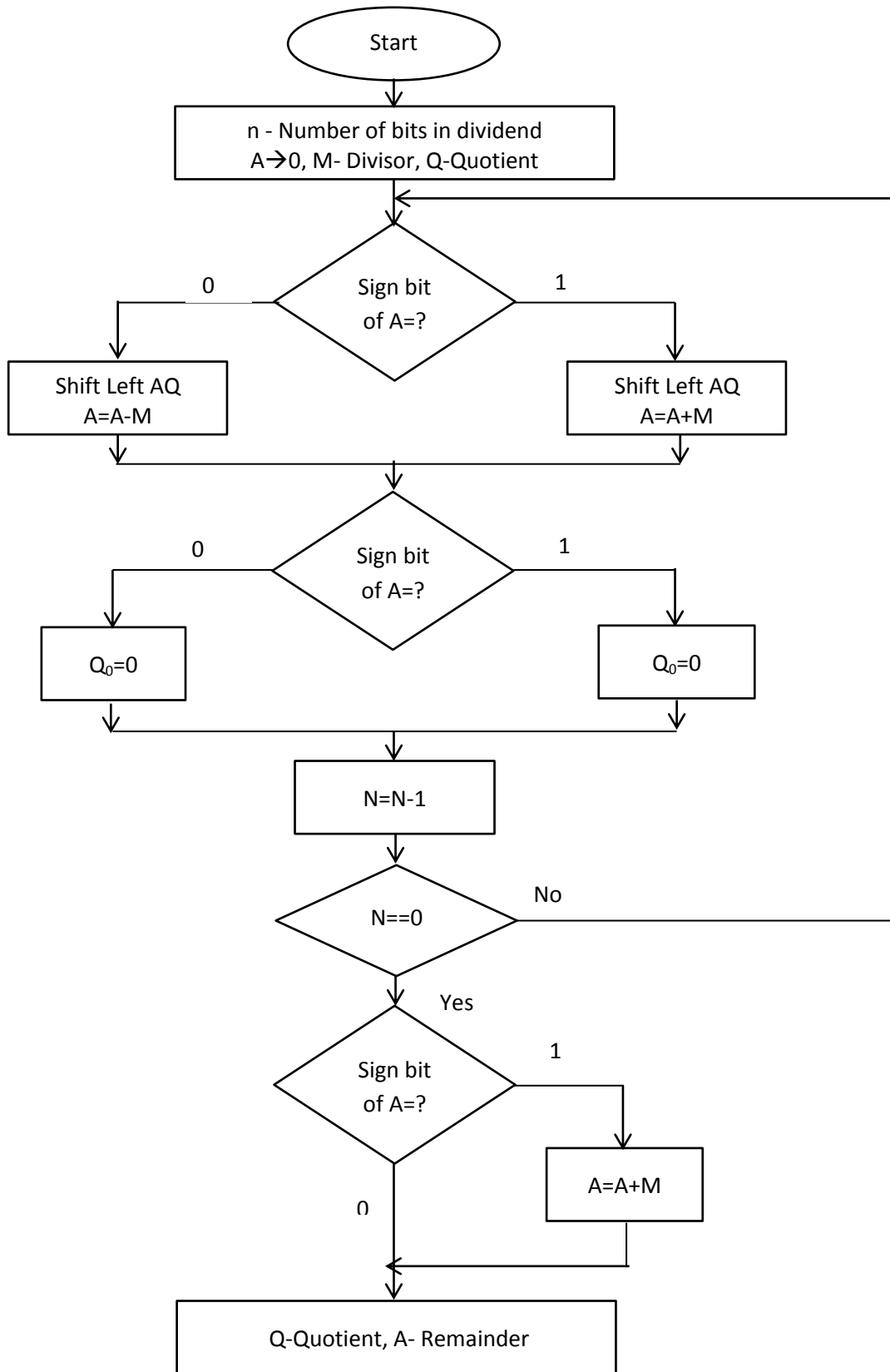
Step 5: If this bit of register A is 1, then Q₀ will become 0. If this bit is 0, then Q₀ will become 1. Here Q₀ indicates the least significant bit of Q.

Step 6: After that, the value of N will be decremented. Here N is used as a counter.

Step 7: If the value of N = 0, then we will go to the next step. Otherwise, we have to again go to step 2.

Step 8: We will perform $A = A + M$ if the sign bit of register A is 1.

Step 9: This is the last step. In this step, register A contains the remainder, and register Q contains the quotient.



Problem 13

Divide $11 \div 3$ using non-restoring division algorithm

$11 \rightarrow 1011$ (Q)

$3 \rightarrow 0011$

$N=4$, therefore number of bits in $A=5$ bits and $M=5$ bits

$3 \rightarrow 00011$ (M)

Count (N)	A	Q	Operation
-	00000	1011	Initial
4	00001	011_	Shift Left AQ
	11110	011_	$A=A-M$
	11110	0110	$Q_0=0$
3	11100	110_	Shift Left AQ
	11111	110_	$A=A+M$
	11111	1100	$Q_0=0$
2	11111	100_	Shift Left AQ
	00010	100_	$A=A+M$
	00010	1001	$Q_0=1$
1	00101	001_	Shift Left AQ
	00010	001_	$A=A-M$
	00010	0011	$Q_0=1$

$A \rightarrow 00010$ (Remainder)

$Q \rightarrow 0011$ (Quotient)

Problem 14**Divide $32 \div 7$ using non-restoring division algorithm** **$32 \rightarrow 100000$ (Q)** **$7 \rightarrow 000111$** **N=6, therefore number of bits in A=7 bits and M=7 bits** **$7 \rightarrow 0000111$ (M)**

Count(N)	A	Q	Operation
-	0000000	100000	Initial
6	0000001 1111010 1111010	00000_ 00000_ 000000	Shift Left AQ $A=A-M$ $Q_0=0$
5	1110100 1111011 1111011	00000_ 00000_ 000000	Shift Left AQ $A=A+M$ $Q_0=0$
4	1110110 1111101 1111101	00000_ 00000_ 000000	Shift Left AQ $A=A+M$ $Q_0=0$
3	1111010 0000001 0000001	00000_ 00000_ 000001	Shift Left AQ $A=A+M$ $Q_0=1$
2	0000010 1111011 1111011	00001_ 00001_ 000010	Shift Left AQ $A=A-M$ $Q_0=0$
1	1110110 1111101 1111101	00010_ 00010_ 000100	Shift Left AQ $A=A+M$ $Q_0=0$

Since the sign bit of A is 1, Perform $A=A+M$ **Hence, $A=0000100$** **$A \rightarrow 0000100$ (Remainder)** **$Q \rightarrow 000100$ (Quotient)**

FLOATING POINT REPRESENTATION

Computer arithmetic that supports such numbers is called floating point because it represents numbers in which the binary point is not fixed, as it is for integers. A number in scientific notation that has no leading 0s is called a normalized number, which is the usual way to write it.

For example, 1.0×10^9 is in normalized scientific notation

Normalized: 1.0×2^{-1}

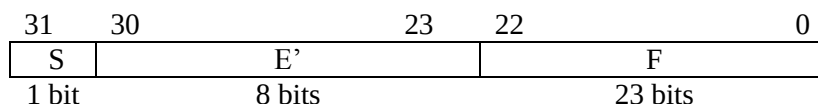
Not Normalized: 0.1×10^{-8} , 10.0×10^{-10}

The representation of a MIPS floating-point number is shown below, where s is the sign of the floating-point number (1 meaning negative), $exponent$ is the value of the 8-bit exponent field (including the sign of the exponent), and $fraction$ is the 23-bit number.

IEEE Standard format for representing floating point numbers

IEEE standard format for single precision floating point number

It is a 32-bit standard representation which occupies 32 bit word.

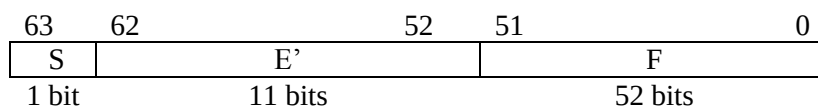


Normalized Value = $\pm 1.F * 2^{E' - 127}$

where $E' = E + 127$

IEEE standard format for double precision floating point number

It is a 64-bit standard representation which occupies 64 bit word.



Normalized Value = $\pm 1.F * 2^{E' - 1023}$

Where, $E' = E + 1023$

Problem 15

Represent 1259.125_{10} in single precision and double precision format

Step 1: Convert decimal number in to binary format

2	1259	
2	629	-1
2	314	-1
2	157	-0
2	78	-1
2	39	-0
2	19	-1
2	9	-1
2	4	-1
2	2	-0
1	1	-0

$$0.125 \times 2 = 0.25$$

$$0.25 \times 2 = 0.5$$

$$0.5 \times 2 = 1.02$$

$$1259 \rightarrow 10011101011$$

$$0.125 \rightarrow 0.001$$

$$1259.125 \rightarrow 10011101011.001$$

Step 2: Normalize the number

$$1.0011101011001 \times 2^{10}$$

Step 3: Single precision representation

S=0 (Since the number is a positive number, its sign bit is zero)

$$E=10$$

$$E' = E + 127 = 10 + 127 = 137$$

$$137 \rightarrow 10001001$$

0	10001001	001110101100100000000000
1 bit	8 bits	23 bits

Step 4: Double precision representation

$$S=0$$

$$E=10$$

$$E' = E + 1023 = 10 + 1023 = 1033$$

$$1033 \rightarrow 10000001001$$

0	10000001001	00111010110010000000000....000
1 bit	11 bits	52 bits

Problem 16

Represent -307.1875_{10} in single precision and double precision format

Step 1: Convert decimal number in to binary format

$$-307.1875 \rightarrow 100110011.0011$$

Step 2: Normalize the number

$$-1.001100110011 \times 2^8$$

Step 3: Single precision representation

S=1 (Since the number is a negative number, its sign bit is one)

$$E=8$$

$$E' = E + 127 = 8 + 127 = 135$$

135 → 10000111

1	10000111	001100110011000000000000
1 bit	8 bits	23 bits

Step 4: Double precision representation

S=0

E=8

$E' = E + 1023 = 8 + 1023 = 1031$

1031 → 10000000111

1	10000000111	00110011001100000...000
1 bit	11 bits	52 bits

Problem 17

Represent -0.75_{10} in single precision format.

Step 1: Convert decimal number in to binary format

$-0.75 \rightarrow -0.11_2$

Step 2: Normalize the number

-1.1×2^{-1}

Step 3: Single precision representation

S=1 (Since the number is a negative number, its sign bit is one)

E=-1

$E' = E + 127 = -1 + 127 = 126$

126 → 01111110

1	01111110	100000000000000000000000
1 bit	8 bits	23 bits

Exceptions

According to IEEE standards, the processor sets exception flags if underflow, overflow, divide by zero occurs.

Underflow: In a single precision, if the number requires an exponent less than -126 or in a double precision, if the number require an exponent less than -1022 to represent its normalized form the underflow occurs.

Overflow: In a single precision, if the number requires an exponent greater than +127 or in a double precision, if the number require an exponent greater than +1023 to represent its normalized form the overflow occurs.

FLOATING POINT ADDITION

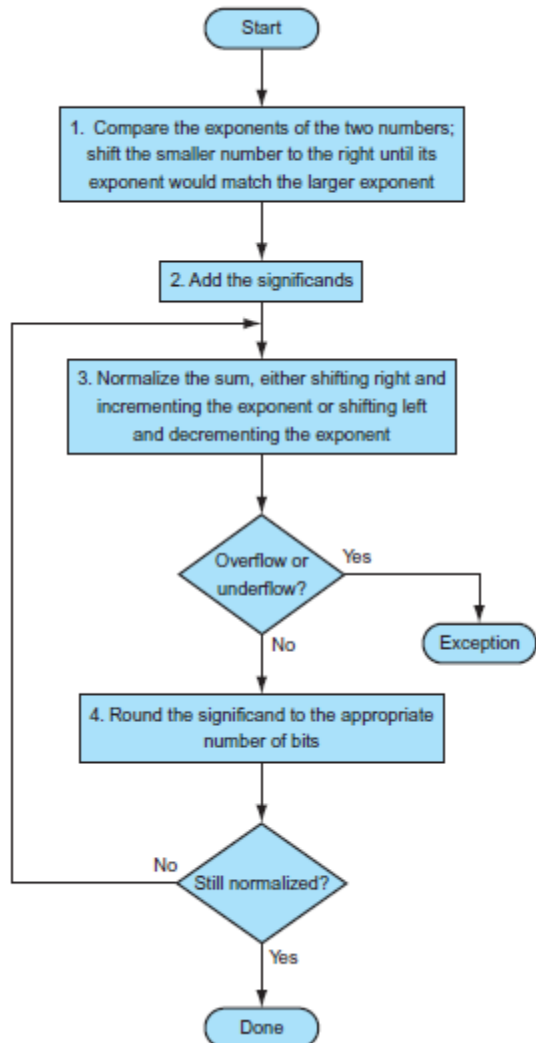
Rules for floating point addition and subtraction

Step 1: Select the number with a smaller exponent and shift its mantissa right, a number of steps equal to the difference in exponents.

Step 2: Set the exponent of the result equal to the larger exponent

Step 3: Perform addition and subtraction on the mantissa and determine the sign of the result

Step 4: Normalize the result, if necessary



Problem 18

Add the number 0.5_{10} and -0.4375_{10} using floating point addition.

Step 1: Convert the number into binary

$$0.5 \rightarrow 0.1$$

$$-0.4375 \rightarrow -0.0111$$

Step 2: Normalize the number

$$0.1 \rightarrow 1.0 \times 2^{-1}$$

$$-0.0111 \rightarrow -1.11 \times 2^{-2}$$

Step 3: Take the significand (mantissa) of the lesser exponent shift it right until its exponent matches the larger number

$$-1.110 \times 2^{-2} \rightarrow -0.111 \times 2^{-1}$$

Step 4: Add the significand

$$\begin{aligned} &= 1.0 \times 2^{-1} + (-0.111 \times 2^{-1}) \\ &= 0.001_2 \times 2^{-1} \end{aligned}$$

Step 5: Normalize the sum, Checking for overflow and Underflow

$$1.0_2 \times 2^{-4}$$

Since, $127 \geq 4 \geq -126$. No overflow or Underflow

Step 6: Round the sum to 4 bits

$$\begin{aligned} \text{Sum is then } 1.000_2 \times 2^{-4} &= 0.0001000_2 = 0.0001_2 \\ &= 1/2^4 = 1/16 = 0.0625_{10} \end{aligned}$$

Problem 19

Add the number 0.5_{10} and -0.4375_{10} using floating point addition.

Step 1: Convert the number in to binary

$$0.5 \rightarrow 0.1$$

$$-0.4375 \rightarrow -0.0111$$

Step 2: Normalize the number

$$0.1 \rightarrow 1.0 \times 2^{-1}$$

$$-0.0111 \rightarrow -1.11 \times 2^{-2}$$

Step 3: Take the significand (mantissa) of the lesser exponent shift it right until its exponent matches the larger number

$$-1.110 \times 2^{-2} \rightarrow -0.111 \times 2^{-1}$$

Step 4: Add the significand

$$\begin{aligned} &= 1.0 \times 2^{-1} + (-0.111 \times 2^{-1}) \\ &= 0.001_2 \times 2^{-1} \end{aligned}$$

Step 5: Normalize the sum, Checking for overflow and Underflow

$$1.0_2 \times 2^{-4}$$

Since, $127 \geq 4 \geq -126$. No overflow or Underflow

Step 6: Round the sum to 4 bits

$$\begin{aligned} \text{Sum is then } 1.000_2 \times 2^{-4} &= 0.0001000_2 = 0.0001_2 \\ &= 1/2^4 = 1/16 = 0.0625_{10} \end{aligned}$$

Problem 20

Add the number 9.75_{10} and 18.5625_{10} using floating point addition.

Step 1: Convert the number in to binary

$$9.7 \rightarrow 01001.110$$

$$18.5625 \rightarrow 010010.1001$$

Step 2: Normalize the number

$$01001.110 \rightarrow 1.001110 \times 2^3$$

$$010010.1001 \rightarrow 1.00101001 \times 2^4$$

Step 3: Take the significand (mantissa) of the lesser exponent shift it right until its exponent matches the larger number

$$1.001110 \times 2^3 \rightarrow 0.1001110 \times 2^4$$

Step 4: Add the significand

$$\begin{aligned} &= 0.1001110 \times 2^4 + 1.00101001 \times 2^4 \\ &= 1.11000101 \times 2^4 \end{aligned}$$

Step 5: Normalize the sum, Checking for overflow and Underflow

$$1.11000101 \times 2^4$$

Since, $127 \geq 4 \geq -126$. No overflow or Underflow

Sum is then $1.11000101_2 \times 2^4 = 11100.0101_2 = 28.3125_{10}$

Problem 21

Add the number 0.75_{10} and 0.275_{10} using floating point addition.

Step 1: Convert the number in to binary

$$0.75 \rightarrow 0.11$$

$$-0.275 \rightarrow -0.01000110$$

Step 2: Normalize the number

$$0.11 \rightarrow 1.1 \times 2^{-1}$$

$$-0.01000110 \rightarrow -1.000110 \times 2^{-2}$$

Step 3: Take the significand (mantissa) of the lesser exponent shift it right until its exponent matches the larger number

$$-1.000110 \times 2^{-2} \rightarrow -0.1000110 \times 2^{-1}$$

Step 4: Add the significand

$$= 1.1 \times 2^{-1} + (-0.1000110 \times 2^{-1})$$

$$= 0.1111010 \times 2^{-1}$$

Step 5: Normalize the sum, checking for overflow and Underflow

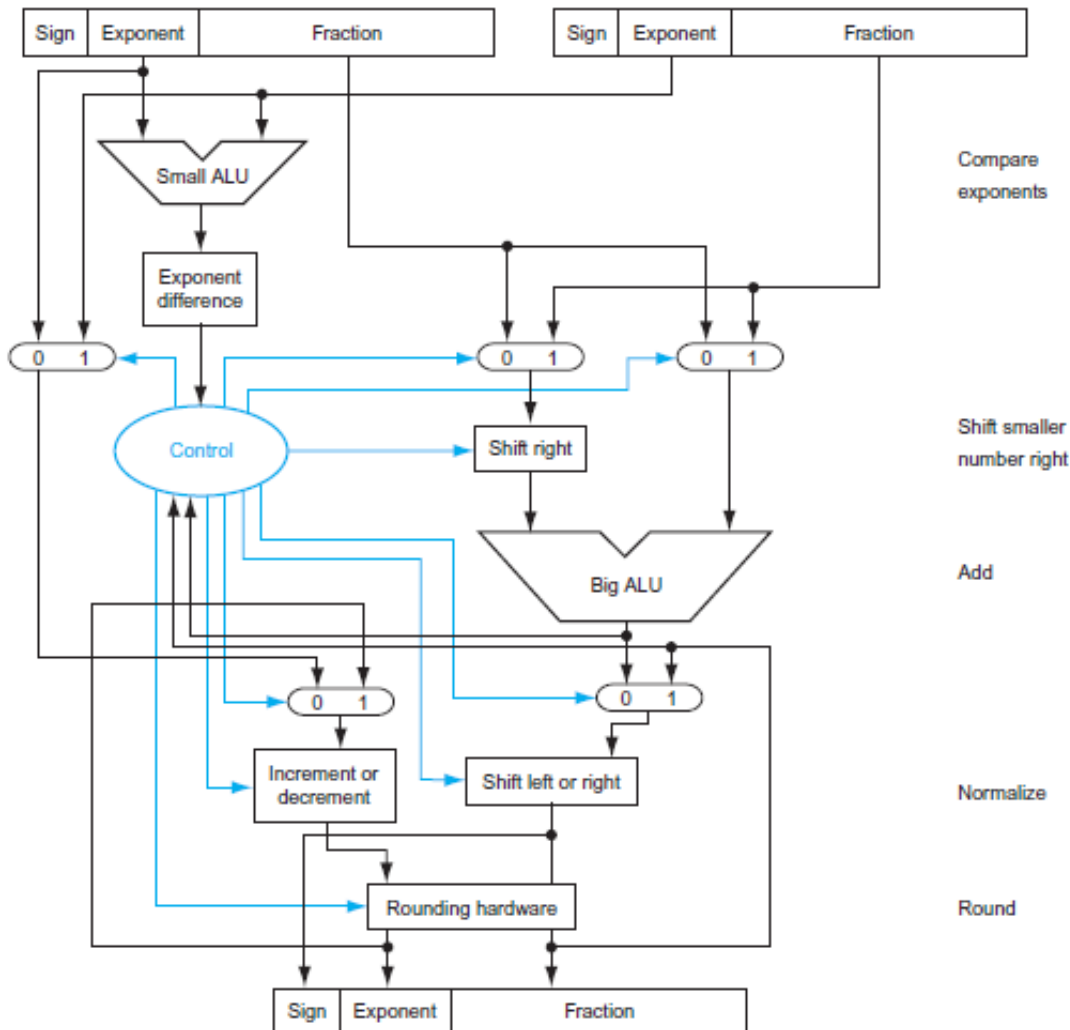
$$1.111010 \times 2^{-2}$$

Since, $127 \geq 2 \geq -126$. No overflow or Underflow

Sum is then $1.111010 \times 2^{-2} = 0.1111010 = 0.475_{10}$

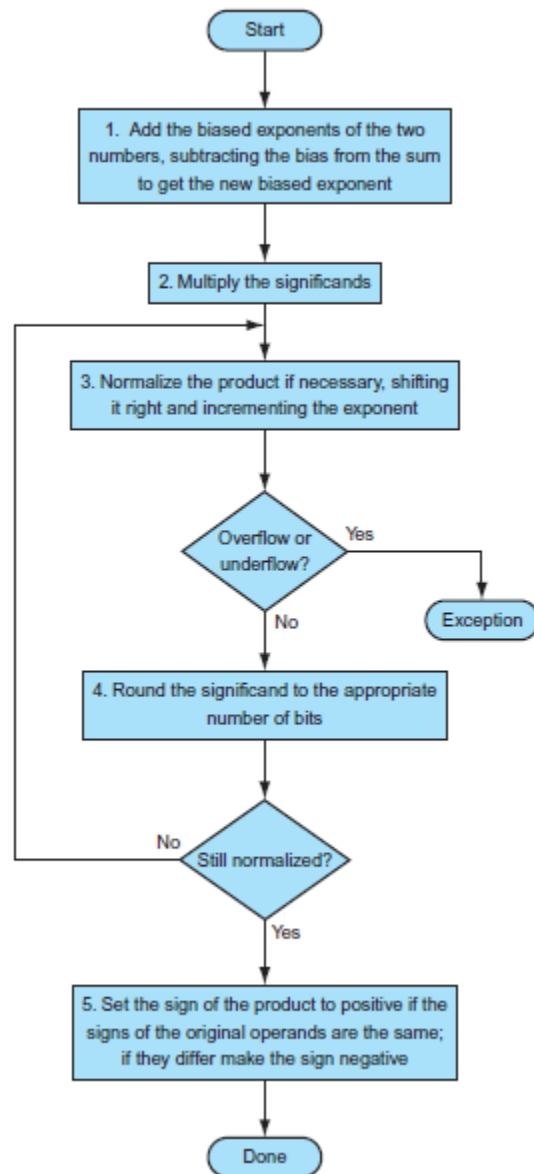
FLOATING POINT ADDER HARDWARE

- First, the exponent of one operand is subtracted from the other using the small ALU to determine which is larger and by how much.
- This difference controls the three multiplexors; from left to right, they select the larger exponent, the significand of the smaller number, and the significand of the larger number. The smaller significand is shifted right, and then the significands are added together using the big ALU.
- The normalization step then shifts the sum left or right and increments or decrements the exponent.
- Rounding then creates the final result, which may require normalizing again to produce the actual final result.



Block diagram of an arithmetic unit dedicated to floating-point addition

FLOATING POINT MULTIPLICATION



Problem 22

Multiply $1.110 \times 10^{10} \times 9.200 \times 10^{-5}$

Step 1: Add exponents

$$\text{New exponent} = 10 + -5 = 5$$

Step 2: Multiply significands

$$\begin{array}{r} 1.110_{\text{ten}} \\ \times 9.200_{\text{ten}} \\ \hline 0000 \\ 0000 \\ 2220 \\ 9990 \\ \hline 10212000_{\text{ten}} \end{array}$$

$$1.110 \times 9.200 = 10.212 \Rightarrow 10.212 \times 10^5$$

Step 3: Normalize result & check for over/underflow

$$1.0212 \times 10^6$$

Step 4: Round and renormalize if necessary

$$1.021 \times 10^6$$

Step 5: Determine sign of result from signs of operands

$$+1.021 \times 10^6$$

Problem 23

Multiply $0.5 \times (-0.4375)$

Step 1: Convert the number in to binary

$$.5 \rightarrow 0.1$$

$$-0.4375 \rightarrow -0.0111$$

Step 2: Normalize the number

$$0.1 \rightarrow 1.0 \times 2^{-1}$$

$$-0.0111 \rightarrow -1.11 \times 2^{-2}$$

Step 3: Add exponents

$$\text{New exponent} = -1 + -2 = -3$$

Step 4: Multiply significands

$$\begin{array}{r} 1.000_{\text{two}} \\ \times 1.110_{\text{two}} \\ \hline 0000 \\ 1000 \\ 1000 \\ 1000 \\ \hline 1110000_{\text{two}} \end{array}$$

$$1.000 \times 1.110 \times 2^{-3} = -1.11 \times 2^{-3}$$

SUBWORD PARALLELISM

Many graphics systems originally used 8 bits to represent each of the three primary colors plus 8 bits for a location of a pixel. The addition of speakers and microphones for teleconferencing and video games suggested support of sound as well. Audio samples need more than 8 bits of precision, but 16 bits are sufficient.

Every microprocessor has special support so that bytes and halfwords take up less space when stored in memory, but due to the infrequency of arithmetic operations on these data sizes in typical integer programs, there was little support beyond data transfers. Architects recognized that many graphics and audio applications would perform the same operation on vectors of this data. By partitioning the carry chains within a 128-bit adder, a processor could use **parallelism** to perform simultaneous operations on short vectors of sixteen 8-bit operands, eight 16-bit operands, four 32-bit operands, or two 64-bit operands. The cost of such partitioned adders was small.

A subword is a lower precision unit of data contained within a word. In subword parallelism, multiple subwords are packed into a word and then process whole words. With the appropriate subword boundaries this technique results in parallel processing of subwords. Since the same instruction is applied to all subwords within the word. This is a form of SIMD (Single Instruction Multiple Data) processing. It is possible to apply subword parallelism to noncontiguous subwords of different sizes within a word. In practical implementation is simple if subwords are same size and they are contiguous within a word. The data parallel programs that benefit from subword parallelism tend to process data that are of the same size.

For example,

- If word size is 64bits and subwords sizes are 8,16 and 32 bits. Hence an instruction operates on eight 8bit subwords, four 16bit subwords, two 32bit subwords or one 64bit subword in parallel.
- Subword parallelism is an efficient and flexible solution for media processing because algorithm exhibit a great deal of data parallelism on lower precision data.
- It is also useful for computations unrelated to multimedia that exhibit data parallelism on lower precision data.
- Graphics and audio applications can take advantage of performing simultaneous operations on short vectors

Example: 128-bit adder

- Sixteen 8-bit adder
- Eight 16-bit adder
- Four 32-bit adder

Also called data-level parallelism, vector parallelism, or Single Instruction, Multiple Data (SIMD)

UNIT III PROCESSOR AND CONTROL UNIT

Basic MIPS implementation – Building datapath – Control Implementation scheme – Pipelining – Pipelined datapath and control – Handling Data hazards & Control hazards – Exceptions.

I. BASIC MIPS IMPLEMENTATION:

MIPS implementation includes a subset of the core MIPS instruction set:

- The memory-reference instructions load word (lw) and store word (sw)
- The arithmetic-logical instructions add, sub, AND, OR, and slt
- The instructions branch equal (beq) and jump (j)

Overview of the Implementation

For every instruction, the first two steps are identical:

1. Send the program counter (PC) to the memory that contains the code and fetch the instruction from that memory.
2. Read one or two registers, using fields of the instruction to select the registers to read. For the load word instruction, need to read only one register, but most other instructions required to read two registers.

After these two steps, the actions required to complete the instruction depend on the instruction class.

Three instruction classes:

- Memory reference
- Arithmetic-logical
- Branch

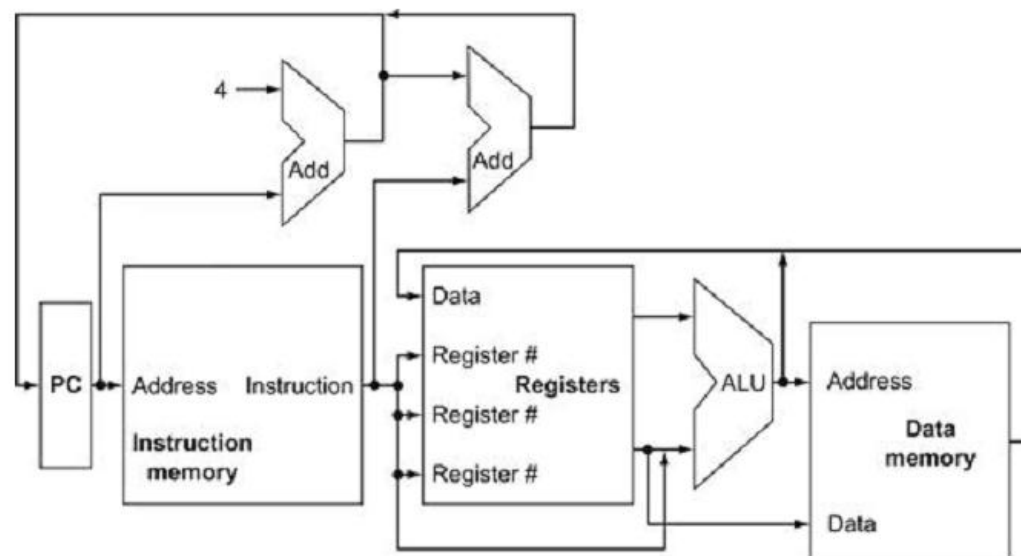
The simplicity and regularity of the MIPS instruction set simplifies the implementation by making the execution of many of the instruction classes similar. All instruction classes, except jump, use the arithmetic-logical unit (ALU) after reading the registers. For eg:

- The memory-reference instructions use the ALU for an address calculation.
- Arithmetic-logical instructions for the operation execution.
- Branches for comparison.

After using the ALU, the actions required to complete various instruction classes differ.

- A memory-reference instruction will need to access the memory either to read data for a load or write data for a store.
- An arithmetic-logical or load instruction must write the data from the ALU or memory back into a register.
- Branch instruction, may need to change the next instruction address based on the comparison; otherwise, the PC should be incremented by 4 to get the address of the next instruction.

Figure 1 shows the high-level or abstract view of a MIPS implementation, focusing on the various functional units and the major components and their interconnection.



- All instructions start by using the program counter to supply the instruction address to the instruction memory.
- After the instruction is fetched, the register operands used by an instruction as specified by fields of that instruction are fetched. Once the register operands have been fetched they can be operated on:
 - To compute a memory address (for a load or store),
 - To compute an arithmetic result (for an integer arithmetic-logical instruction)
 - Performs comparison (for a branch).
- If the instruction is an arithmetic-logical instruction, the result from the ALU must be written to a register.
- If the operation is a load or store, the ALU result is used as an address to either store a value from the registers or load a value from memory into the register. The result from the ALU or memory is written back into the register file.
- Branches require the use of the ALU output to determine the next instruction address, which comes either from the ALU (where the PC and branch offset are summed) or from an adder that increments the current PC by 4.

MIPS implementation shown in **Figure 1** depicts that data going to a particular unit as coming from two different sources.

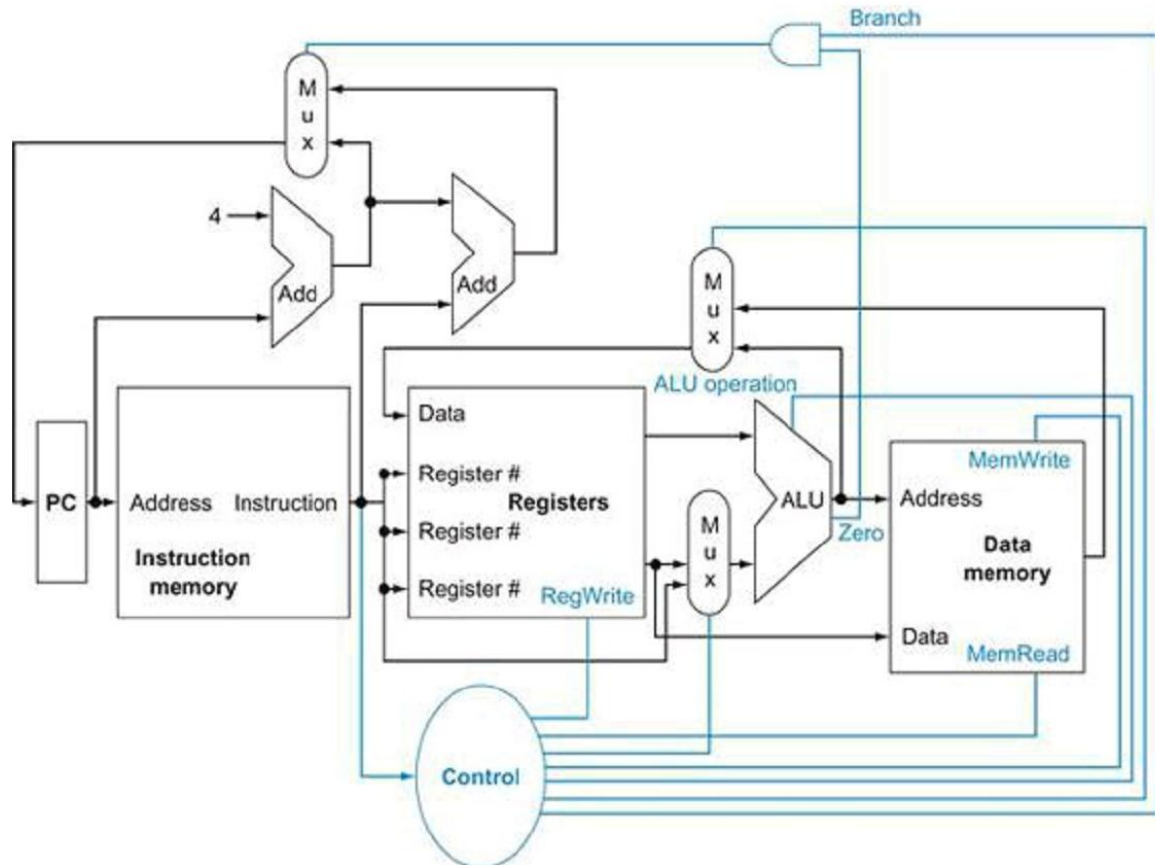
For example,

- The value written into the PC can come from one of two adders.
- The data written into the register file can come from either the ALU or the

data memory, and the second input to the ALU can come from a register or the immediate field of the instruction.

An logic element is added that selects one input among the multiple sources and steers that one to its destination. This logic element which performs this selection is commonly called a **multiplexor (MUX)** or data **selector**.

Figure 2 shows the basic implementation of the MIPS subset, including the necessary multiplexors and control lines



- The top multiplexor (“Mux”) controls what value replaces the PC (PC+4 or the branch destination address).
- The middle multiplexor, whose output returns to the register file, is used to steer the output of the ALU (in the case of an arithmetic-logical instruction) or the output of the data memory (in the case of a load) for writing into the register file.
- The bottommost multiplexor is used to determine whether the second ALU input is from the registers (for an arithmetic-logical instruction or a branch) or from the offset field of the instruction (for a load or store).
- The added control lines determine the operation performed at the ALU, whether the data memory should read or write, and whether the registers should perform a write operation.

The **datapath elements** in the MIPS implementation consist of two different types of **logic elements**:

- Elements that operate on data values - **Combinational**
- Elements that contain state - **State**

Combinational

element:

Elements that operate on data values are all **combinational**, which means that their outputs depend only on the current inputs. Eg: **AND gate or an ALU**.

State element:

A memory element, such as a **register or a memory**. An element contains state if it has some internal storage. They can be saved and restored the. Thus, these state elements completely characterize the computer.

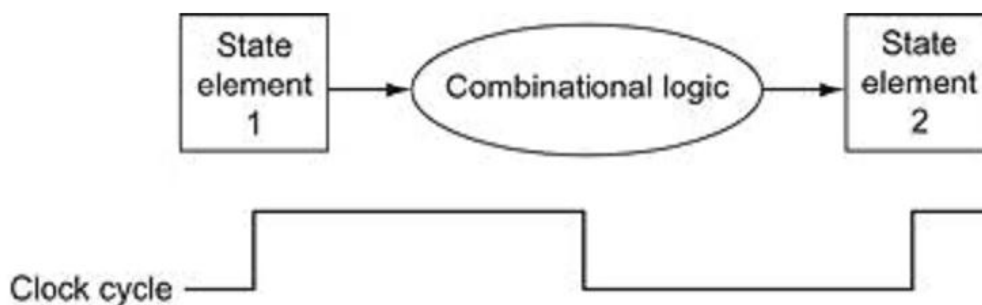


Clocking methodology

The approach used to determine when data is valid and stable relative to the clock.

Edge-triggered clocking

A clocking scheme in which all state changes occur on a clock edge.



- Combinational logic, state elements, and the clock are closely related.
- In a synchronous digital system, the clock determines when elements with state will write values into internal storage.
- All state elements including memory, are assumed to be positive edge-triggered; that is, they change on the rising clock edge.

Control signal

A signal used for multiplex or selection or for directing the operation of a functional unit; contrasts with a data signal, which contains information that is operated on by a functional unit.

Asserted

The signal is logically high or true.

Deasserted

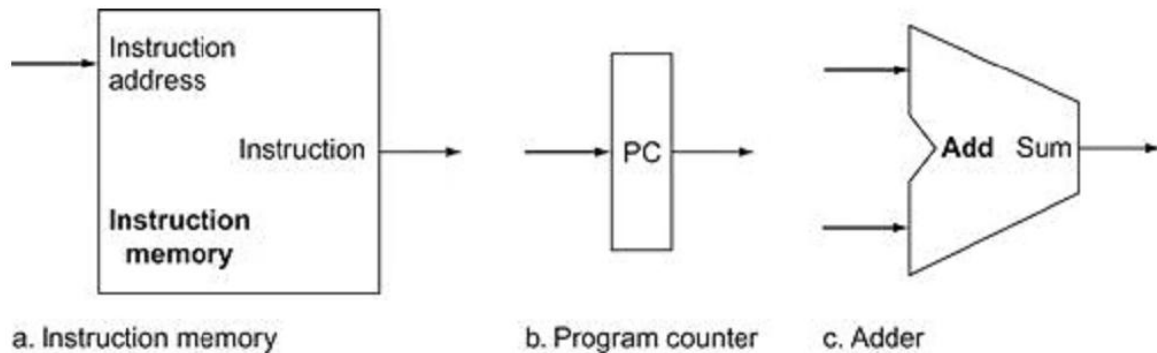
The signal is logically low or false.

II. Building datapath

Data path is a unit used to operate on or hold data within a processor. In the MIPS

implementation, the datapath elements include the instruction and data memories, the register file, the ALU, and adders.

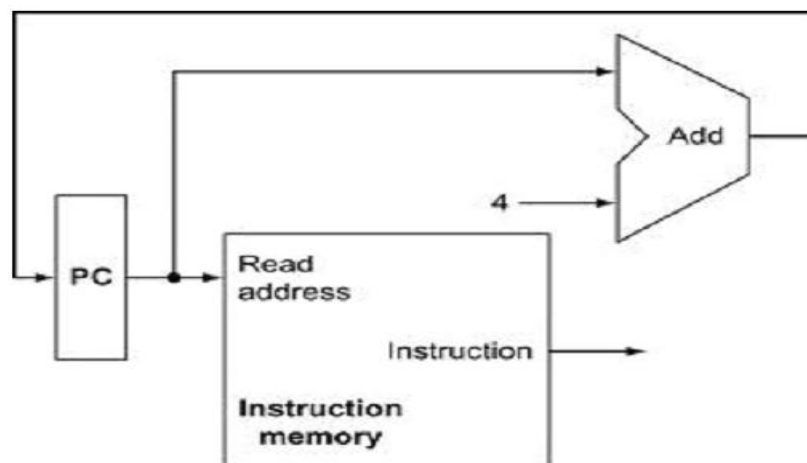
Program counter (PC)
The register containing the address of the instruction in the program being executed.



Datapath elements

i) Instruction fetch : Three datapath elements involved in building datapath of instruction fetch.

- Two state elements are needed to store and access instructions. They are
 - **Instruction memory**
 - **Program Counter(PC)**
- Third element needed to compute the next instruction address is:
 - **Adder**



Instruction memory:

- Only provide read access because the datapath does not write instructions.
- It is treated as a combinational logic: the output at any time reflects the contents of the location specified by the address input
- No read control signal is needed.

Program Counter:

- It is a 32-bit register that is written at the end of every clock cycle.
- It does not need a write control signal.

Adder

- It is an ALU wired to always add its two 32-bit inputs and place the sum on its output.

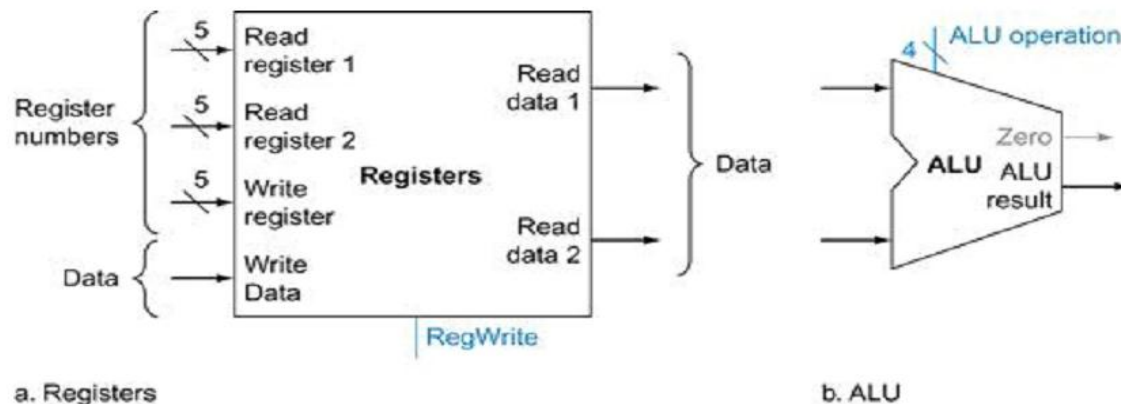
Instruction execution starts by fetching the instruction from memory location pointed by PC. After every fetch PC is incremented, so that it points at the next instruction, 4 bytes later. Figure below shows the datapath that fetches instructions and increments the PC to obtain the address of the next sequential instruction.

ii) R-format instructions: R-format instructions have three register operands, so it will need to

- Read two registers
 - Perform an operation on the contents of the registers,
 - And write the result to a register.
- These instructions are either R-type instructions or arithmetic logical instructions (since they perform arithmetic or logical operations). This instruction class includes add, sub, and, or, and slt.

Eg: add \$t1, \$t2, \$t3, which reads \$t2 and \$t3 and writes \$t1. Two datapath elements involved in building datapath of R-format instructions. They are:

- Register file**
- ALU**



Register file:

The processor's 32 general-purpose registers are stored in a structure called a register file.

A register file is a collection of registers in which any register can be read or written by specifying the number of the register in the file.

The register file contains the register state of the computer.

All the registers in the register file contains two read ports and one write port.

The register file always outputs the contents of the registers corresponding to the Read register inputs on the outputs; no other control inputs are needed.

Register write must be explicitly indicated by asserting the write control signal.

- Write control signals are edge triggered, so that all the write inputs (i.e., the value to be written, the register number, and the write control signal) must be valid at the clock edge.
- Since writes to the register file are edge-triggered, our design can legally read and write the same register within a clock cycle
 - The read will get the value written in an earlier clock cycle, while the value written will be available to a read in a subsequent clock cycle.
 - The inputs carrying the register number to the register file are all 5 bits wide.
 - The lines carrying data values are 32 bits wide.

ALU:

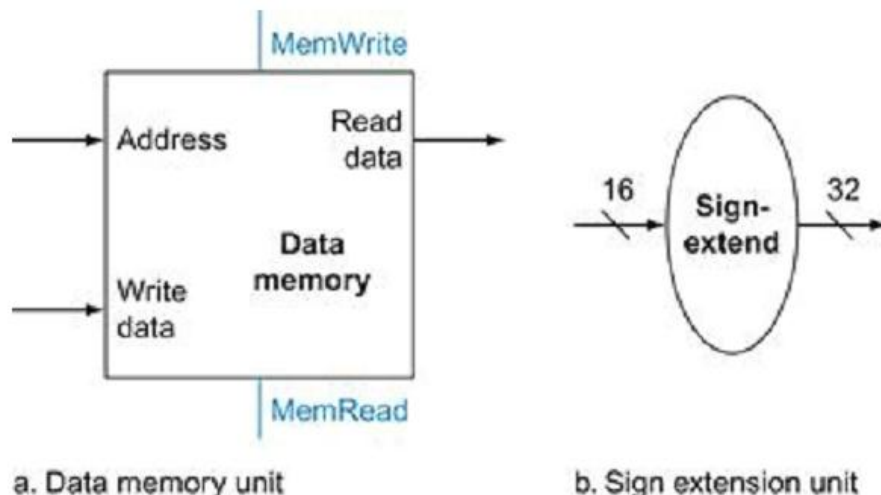
- In addition, there is an A L U to operate on the values read from the registers.
- The operation to be performed by the ALU is controlled with the ALU operation signal, which is 4 bits wide.
- A L U, which takes two 32-bit inputs and produces a 32-bit result, as well as a 1-bit signal if the result is 0

iii) MIPS load word and store word instructions: which have the general form

- lw \$t1, offset_value(\$t2)
- sw \$t1, offset_value (\$t2) .
- These instructions compute a memory address by adding the base register, which is \$t2, to the 16-bit signed offset field contained in the instruction.
- If the instruction is a store, the value to be stored must also be read from the register file where it resides in \$t1.
- If the instruction is a load, the value read from memory must be written into the register file in the specified register, which is \$t1.

The **datapath elements** (units) needed to implement loads and stores, in addition to the register file and ALU are :

- **Data memory unit**
- **Sign extension unit.**



- **Sign-extend**

The sign extension unit has a 16-bit input that is sign-extended into a 32-bit result appearing on the output.

- **Data Memory unit**

- The memory unit is a state element with inputs for the address and the write data, and a single output for the read result.
- There are separate read and write controls, although only one of these may be asserted on any given clock.
- The memory unit needs a read signal, since, unlike the register file, reading the value of an invalid address can cause problems
- Data memory is edge-triggered for writes.
- Standard memory chips actually have a write enable signal that is used for writes.

iv) **MIPS Branch Instruction: Which has the general form:**

beq \$t1, \$t2, offset

- The beq instruction has three operands, two registers that are compared for equality, and a 16-bit offset used to compute the branch target address relative to the branch instruction address.
- The address specified in a branch, which becomes the new program counter (PC) address if the branch is taken. In the MIPS architecture the branch target is given by the sum of the offset field of the instruction and the address of the instruction following the branch.

To implement this instruction

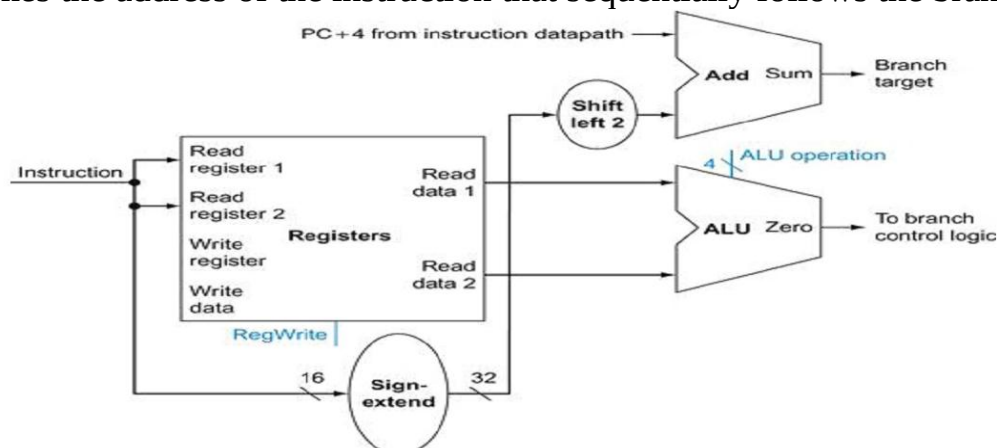
- Compute the branch target address by adding the sign-extended offset field of the instruction to the PC.

Branch taken:

A branch where the branch condition is satisfied and the program counter (PC) becomes the branch target. All unconditional branches are taken branches.

Branch not taken or (untaken branch):

A branch where the branch condition is false and the program counter (PC) becomes the address of the instruction that sequentially follows the branch.



The datapath elements for a branch instruction are:

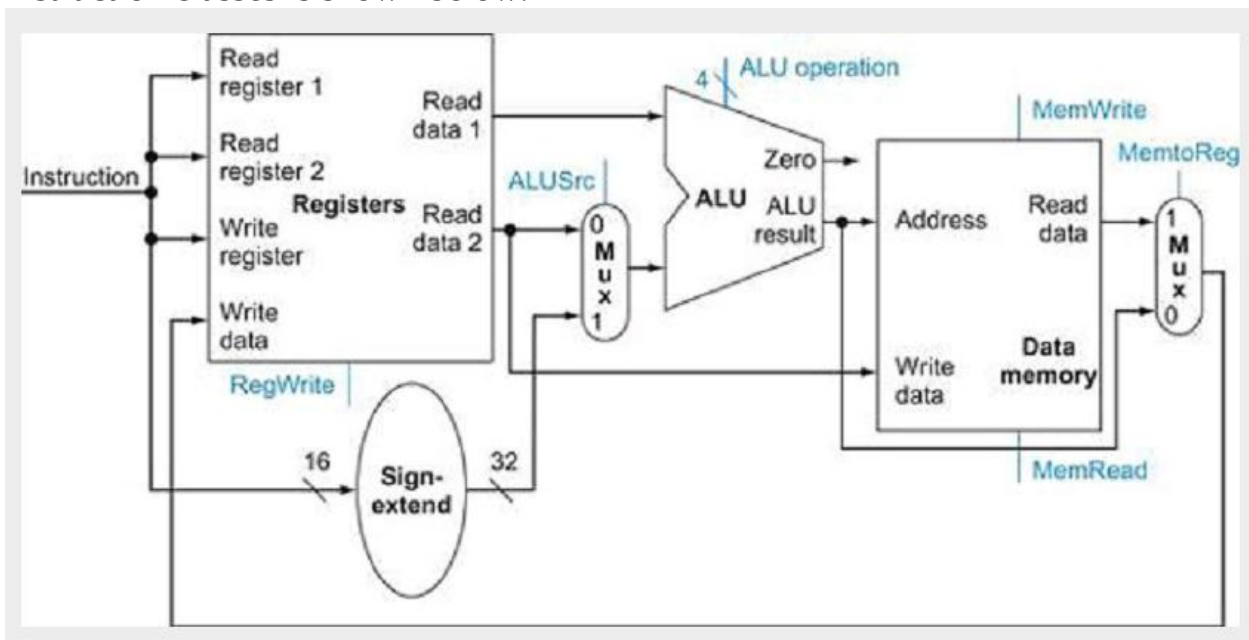
- **ALU** - To evaluate the branch condition
 - **Separate Adder** - To compute the branch target as the sum of the incremented PC and the sign-extended, lower 16 bits of the instruction (the branch displacement), shifted left 2 bits.
 - The unit labeled Shift left 2 is simply a routing of the signals between input and output that adds 00_2 to the low order end of the sign-extended offset field; no actual shift hardware is needed, since the amount of the “shift” is constant. Since we know that the offset was sign-extended from 16 bits, the shift will throw away only “sign bits.”
- Control logic is used to decide whether the incremented PC or branch target should replace the PC, based on the Zero output of the ALU.

Delayed branch

A type of branch where the instruction immediately following the branch is always executed, independent of whether the branch condition is true or false.

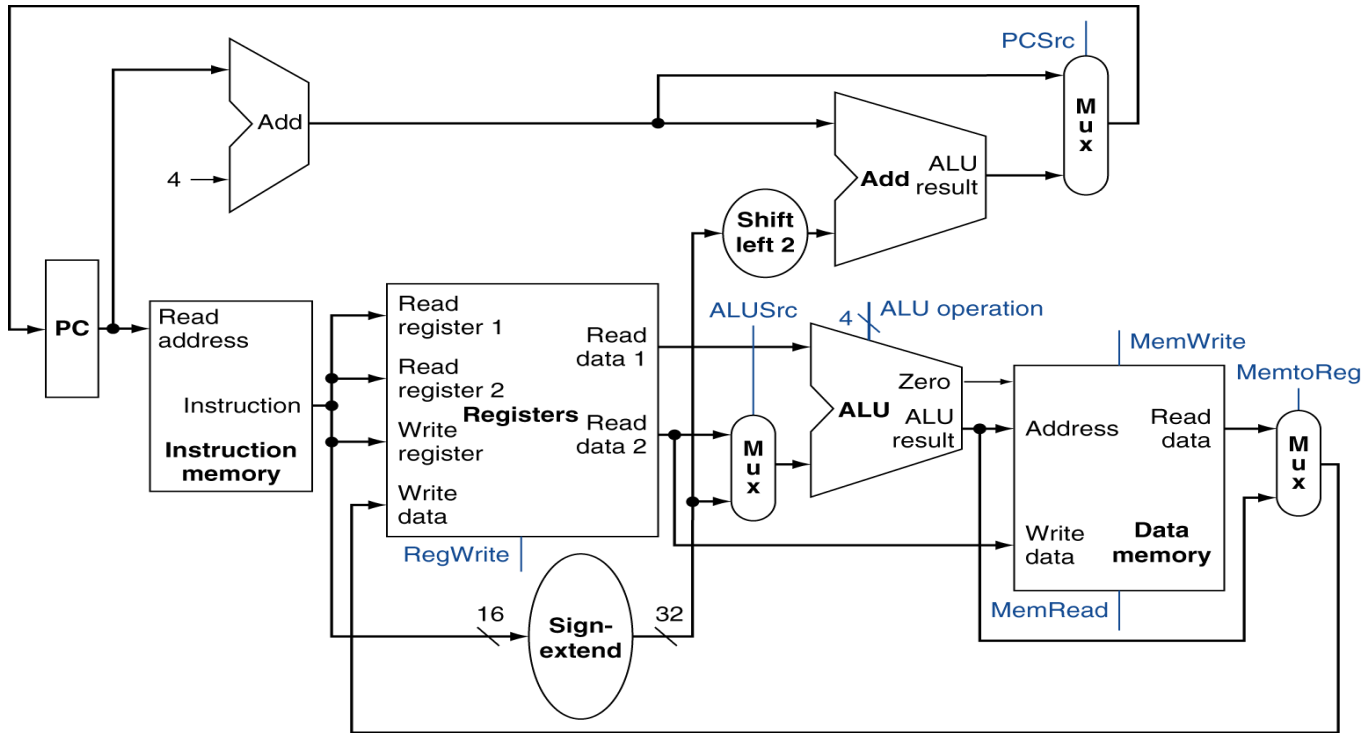
Creating a Single Datapath

The simple datapath for the MIPS architecture combines the elements required by different instruction classes is shown below:



- This datapath can execute the basic instructions (load/store word, ALU operations, and branches) in a single clock cycle. An additional multiplexor is needed to integrate branches.
- All the pieces to make a simple datapath for the core MIPS architecture are combined by adding the datapath for instruction fetch, the datapath from R-type and memory instructions and the datapath for branches.

Full Datapath



III. Control Implementation scheme

The ALU Control

■ **ALU used for**

- **Load/Store:** F = add
- **Branch:** F = subtract
- **R-type:** F depends on funct field

- The MIPS ALU defines the 6 following combinations of four control inputs:

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set on less than
1100	NOR

- Depending on the instruction class, the A LU will need to perform one of these first five functions).
- For load word and store word instructions, A LU is used to compute the

memory address by addition.

- For the R-type instructions, the ALU needs to perform one of the five actions (AND, OR, subtract, add, or set on less than), depending on the value of the 6-bit funct field in the low-order bits of the instruction.
- For branch equal, the ALU must perform a subtraction.
- 4-bit ALU control input can be generated using a small control unit that has the function field of the instruction as input and a 2-bit control field, which is called as ALUOp.
- ALUOp indicates whether the operation to be performed should be add (00) for loads and stores, subtract (01) for beq, or determined by the operation encoded in the funct field (10).
- The output of the ALU control unit is a 4-bit signal that directly controls the ALU by generating one of the 4-bit combinations shown previously. Figure , shows how to set the ALU control inputs based on the 2-bit ALUOp control and the 6-bit function code.

Instruction opcode	ALUOp	Instruction operation	Funct field	Desired ALU action	ALU control input
LW	00	load word	XXXXXX	add	0010
SW	00	store word	XXXXXX	add	0010
Branch equal	01	branch equal	XXXXXX	subtract	0110
R-type	10	add	100000	add	0010
R-type	10	subtract	100010	subtract	0110
R-type	10	AND	100100	AND	0000
R-type	10	OR	100101	OR	0001
R-type	10	set on less than	101010	set on less than	0111

- The opcode, listed in the first column of the table above, determines the setting of the ALUOp bits. All the encodings are shown in binary.
- When the ALUOp code is 00 or 01, the desired ALU action does not depend on the function code field; in this case, it is treated as “don’t care” about the value of the function code, and the funct field is shown as XXXXXX.
- When the ALUOp value is 10, then the function code is used to set the ALU control input.

→ There are several different ways to implement the mapping from the 2-bit ALUOp field and the 6-bit funct field to the four ALU operation control bits. But only a small number of the 64 possible values of the function field are of interest.

It is useful to create a truth table for the interesting combinations of the function code field and the ALUOp bits. This truth table shows how the 4-bit ALU control is set depending on these two input fields. Since the full truth table is very large (2⁸=256 entries) and it also many of these input combinations are **don’t**

cares, it is shown only the truth table entries for which the ALU control must have a specific value.

TRUTH TABLE

Representation of a logical operation by listing all the values of the inputs and then in each case showing what the resulting outputs should be.

ALUOp		Func field						Operation
ALUOp1	ALUOp0	F5	F4	F3	F2	F1	F0	
0	0	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	0110
1	X	X	X	0	0	0	0	0010
1	X	X	X	0	0	1	0	0110
1	X	X	X	0	1	0	0	0000
1	X	X	X	0	1	0	1	0001
1	X	X	X	1	0	1	0	0111

DON'T-CARE TERM

An element of a logical function in which the output does not depend on the values of all the inputs. Don't-care terms may be specified in different ways.

Designing the Main Control Unit

Formats of the three instruction classes: the R-type, branch, and load-store instructions.

Field	0	rs	rt	rd	shamt	funct
Bit positions	31:26	25:21	20:16	15:11	10:6	5:0

a. R-type instruction

Field	35 or 43	rs	rt	address
Bit positions	31:26	25:21	20:16	15:0

b. Load or store instruction

Field	4	rs	rt	address
Bit positions	31:26	25:21	20:16	15:0

c. Branch instruction

The three instruction classes (R-type, load and store, and branch) use two different instruction formats. The jump instructions use another format.

a) Instruction format for R-format instructions:

- Which all have an opcode of 0.
- These instructions have three register operands: rs, rt, and rd.
- Fields rs and rt are sources, and rd is the destination.
- The ALU function is in the funct field and is decoded by the ALU control design.
- Eg for R-type instructions that we implement are add, sub, AND, OR,

and slt.

- The shamt field is used only for shifts.

b) Instruction format for load (opcode=35ten) and store (opcode=43ten) instructions:

- The register rs is the base register that is added to the 16-bit address field to form the memory address.
- For loads, rt is the destination register for the loaded value.
- For stores, rt is the source register whose value should be stored into memory.

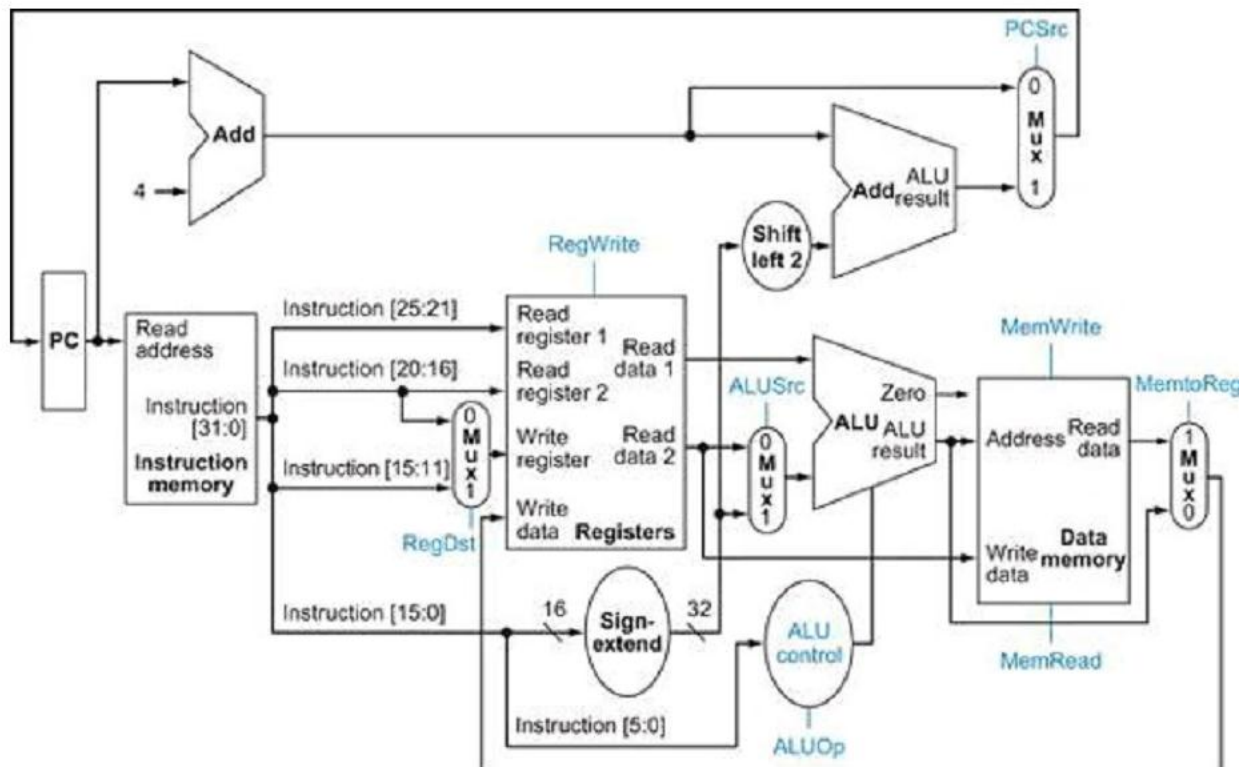
c) Instruction format for branch equal (opcode=4):

- The registers rs and rt are the source registers that are compared for equality.
- The 16-bit address field is sign extended, shifted, and added to the PC+4 to compute the branch target address.

OPCODE

The field that denotes the operation and format of an instruction is called Opcode.

Figure shows datapath with all necessary multiplexors and all control lines.



The ALU control block has also been added. The PC does not require a write control, since it is written once at the end of every clock cycle; the branch control logic determines whether it is written with the incremented PC or the branch target address.

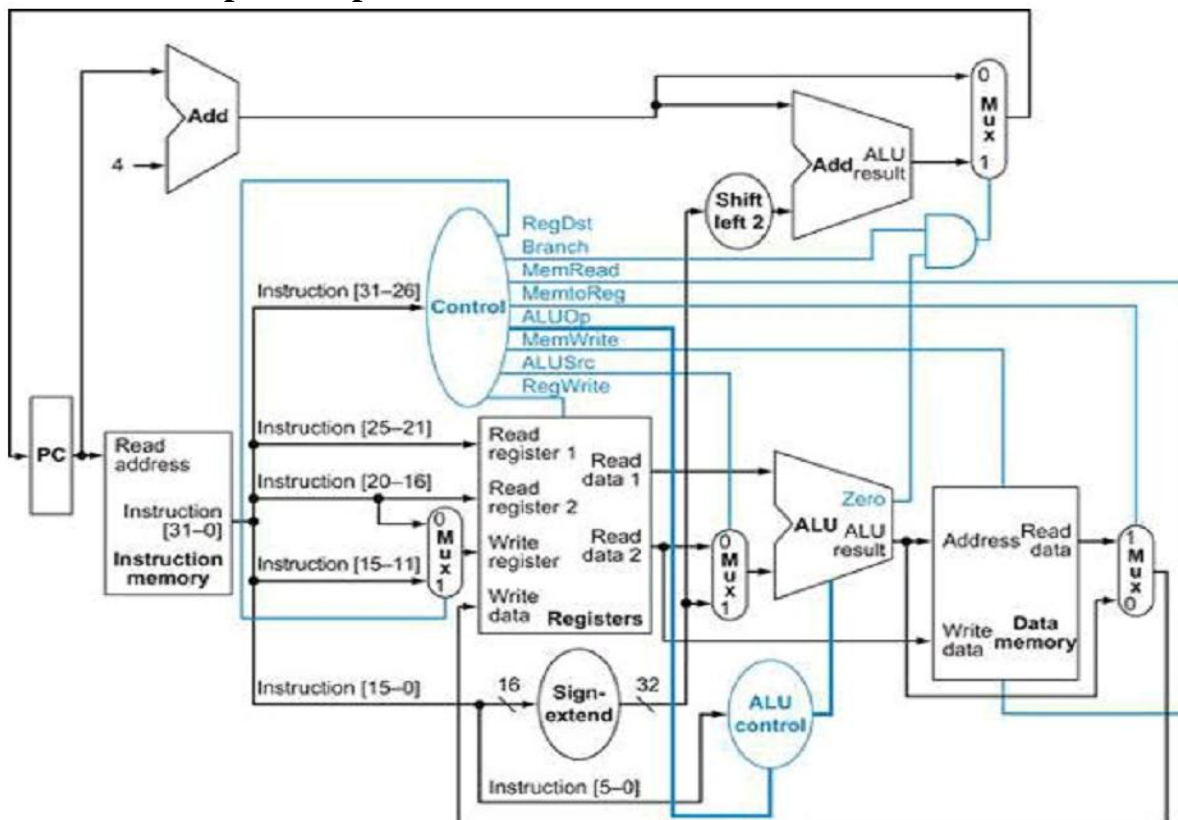
Function of seven control lines

Signal name	Effect when deasserted	Effect when asserted
RegDst	The register destination number for the Write register comes from the rt field (bits 20:16).	The register destination number for the Write register comes from the rd field (bits 15:11).
RegWrite	None.	The register on the Write register input is written with the value on the Write data input.
ALUSrc	The second ALU operand comes from the second register file output (Read data 2).	The second ALU operand is the sign-extended, lower 16 bits of the instruction.
PCSrc	The PC is replaced by the output of the adder that computes the value of $PC + 4$.	The PC is replaced by the output of the adder that computes the branch target.
MemRead	None.	Data memory contents designated by the address input are put on the Read data output.
MemWrite	None.	Data memory contents designated by the address input are replaced by the value on the Write data input.
MemtoReg	The value fed to the register Write data input comes from the ALU.	The value fed to the register Write data input comes from the data memory.

The effect of each of the seven control signals: When the 1-bit control to a two-way multiplexor is asserted, the multiplexor selects the input corresponding to 1. Otherwise, if the control is deasserted, the multiplexor selects the 0 input.

- All the state elements have the clock as an implicit input and that the clock is used in controlling writes.
- Gating the clock externally to a state element can create timing problems.

The simple datapath with the control unit



- ✓ The input to the control unit is the 6-bit opcode field from the instruction.
- ✓ The outputs of the control unit consist of three 1-bit signals that are used to control multiplexors (RegDst, ALUSrc, and MemtoReg).
- ✓ Three signals for controlling reads and writes in the register file and data memory (RegWrite, MemRead, and MemWrite).
- ✓ 1 –bit signal used in determining whether to possibly branch (Branch), and a 2-bit control signal for the ALU (ALUOp).
- ✓ An AND gate is used to combine the branch control signal and the Zero output from the ALU;
- ✓ The AND gate output controls the selection of the next PC.
- ✓ PCSrc is now a derived signal, rather than one coming directly from the control unit.

Figure below shows truth table for the control unit, defines how the control signals should be set for each opcode:

Instruction	RegDst	ALUSrc	Memto-Reg	Reg-Write	Mem-Read	Mem-Write	Branch	ALUOp1	ALUOp0
R-format	1	0	0	1	0	0	0	1	0
lw	0	1	1	1	1	0	0	0	0
sw	X	1	X	0	0	1	0	0	0
beq	X	0	X	0	0	0	1	0	1

Setting of the control lines depends only on the opcode, and defines whether each control signal should be 0, 1, or don't care (X) for each of the opcode values.

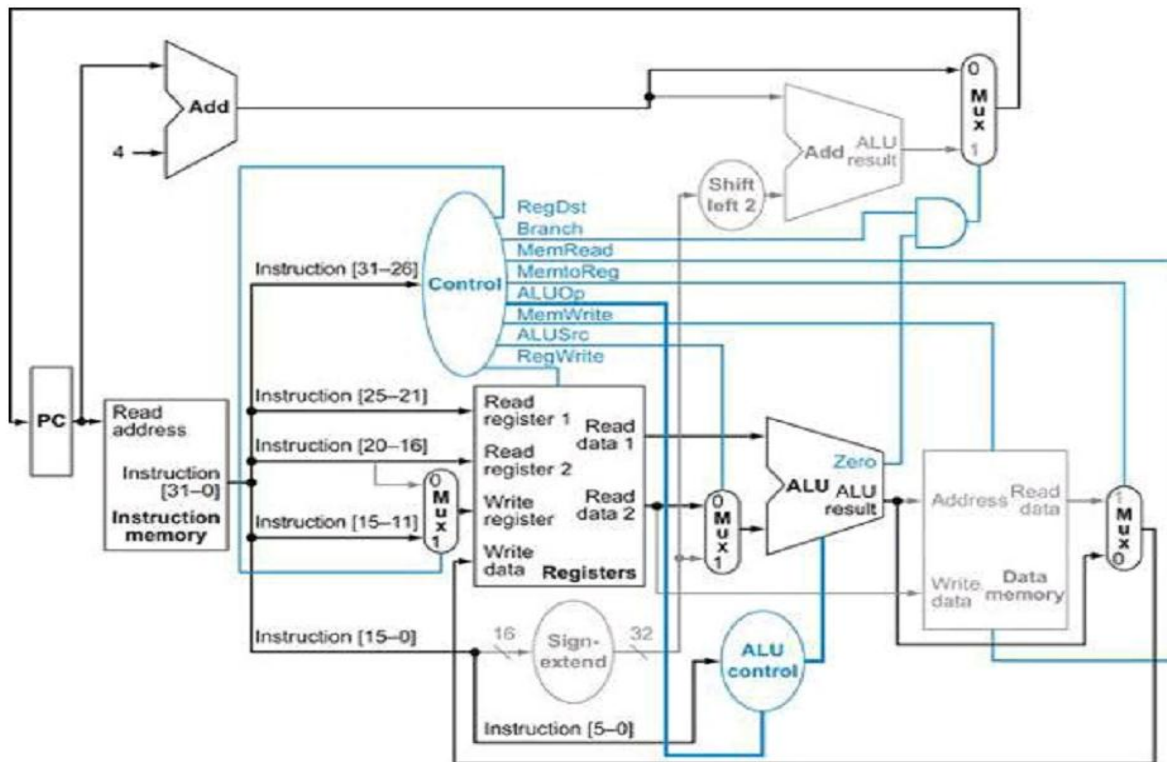
Operation of the Datapath

a) Operation of the datapath for an R-type instruction

Eg: Add \$t1, \$t2, \$t3

Steps to execute the instruction

1. The instruction is fetched, and the PC is incremented.
2. Two registers, \$t2 and \$t3, are read from the register file; also, the main control unit computes the setting of the control lines during this step.
3. The ALU operates on the data read from the register file, using the function code (bits 5:0, which is the funct field, of the instruction) to generate the ALU function.
4. The result from the ALU is written into the register file using bits 15:11 of the instruction to select the destination register (\$t1).

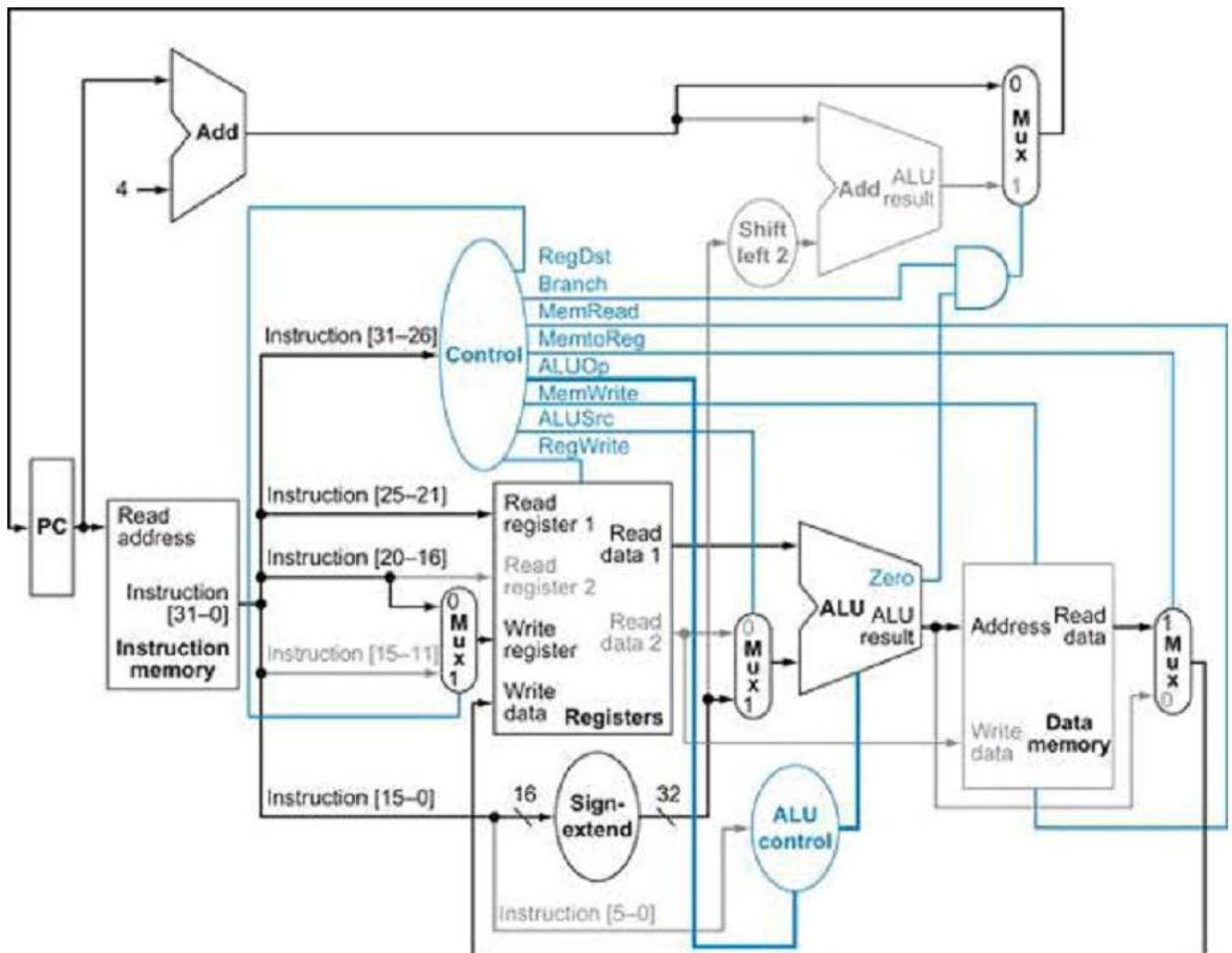


b. Operation of the datapath for load word instruction

Eg: lw \$t1, offset(\$t2)

Steps to execute the instruction

1. An instruction is fetched from the instruction memory, and the PC is incremented. A register (\$t2) value is read from the register file.
2. The ALU computes the sum of the value read from the register file and sign extended, lower 16 bits of the instruction (offset).
3. The sum from the ALU is used as the address for the data memory.
4. The data from the memory unit is written into the register file; the register destination is given by bits 20:16 of the instruction (\$t1).



c. Operation of the datapath for branch-on-equal instruction

Eg: beq \$t1, \$t2, offset

Steps to execute the instruction

1. An instruction is fetched from the instruction memory, and the PC is incremented.
2. Two registers, \$t1 and \$t2, are read from the register file.
3. The ALU performs a subtract on the data values read from the register file. The Value of PC+4 is added to the sign-extended, lower 16 bits of the instruction (offset) shifted left by two; the result is the branch target address.
4. The Zero result from the ALU is used to decide which adder result to store into the PC.

Eg: j offset

Instruction format for the jump instruction (opcode=2):

Steps to execute the instruction

1. Concatenating the upper 4 bits of the current PC+4 to the 26-bit address field in the jump instruction.
2. Add 00 as the 2 low-order bits



Although the single-cycle design will work correctly, it would not be used in modern designs because it is inefficient.

- ✓ Cock cycle must have the same length for every instruction in this single-

cycle design.

- ✓ The longest possible path in the processor determines the clock cycle.
- ✓ This path is almost certainly a load instruction, which uses five functional units in series:
 - Instruction memory
 - Register file
 - A LU
 - Data memory

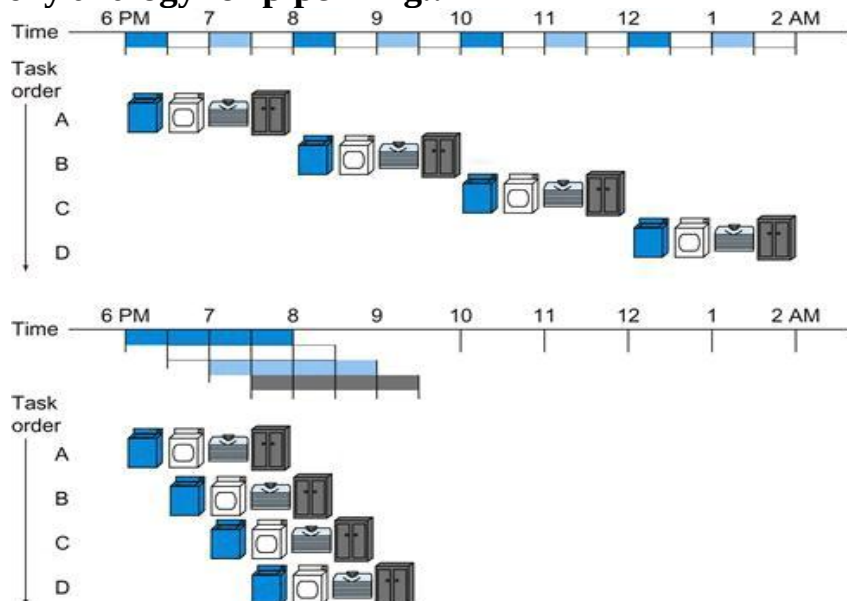
The overall performance of a single-cycle implementation is likely to be poor, since the clock cycle is too long. The penalty for using the single-cycle design with a fixed clock cycle is significant, but might be considered acceptable for small instruction set. Historically, early computers with very simple instruction sets use this implementation technique. However, if we tried to implement the floating-point unit or an instruction set with more complex instructions, this single-cycle design wouldn't work well at all.

IV. Pipelining :

- Pipelining is an implementation technique in which multiple instructions are overlapped in execution much like an assembly line.
- In other words, concurrent execution of instructions.



The laundry analogy for pipelining.:



Ann, Brian, Cathy, and Don each have dirty clothes to be washed, dried, folded, and put away. The washer, dryer, “folder,” and “storer” each take 30 minutes for their task. Sequential laundry takes 8 hours for 4 loads of wash, while pipelined laundry takes just 3.5 hours.

The same principles apply to processors where we pipeline instruction-execution.

MIPS instructions classically take five steps:

1. Fetch instruction from memory.
2. Read registers while decoding the instruction. The regular format of MIPS instructions allows reading and decoding to occur simultaneously.
3. Execute the operation or calculate an address.
4. Access an operand in data memory.
5. Write the result into a register.

Single-Cycle versus Pipelined Performance

Let us consider the foll. Eight instructions:

load word (lw), store word (sw), add (add), subtract (sub), AND (and), OR (or), set less than (slt), and branch on equal (beq).

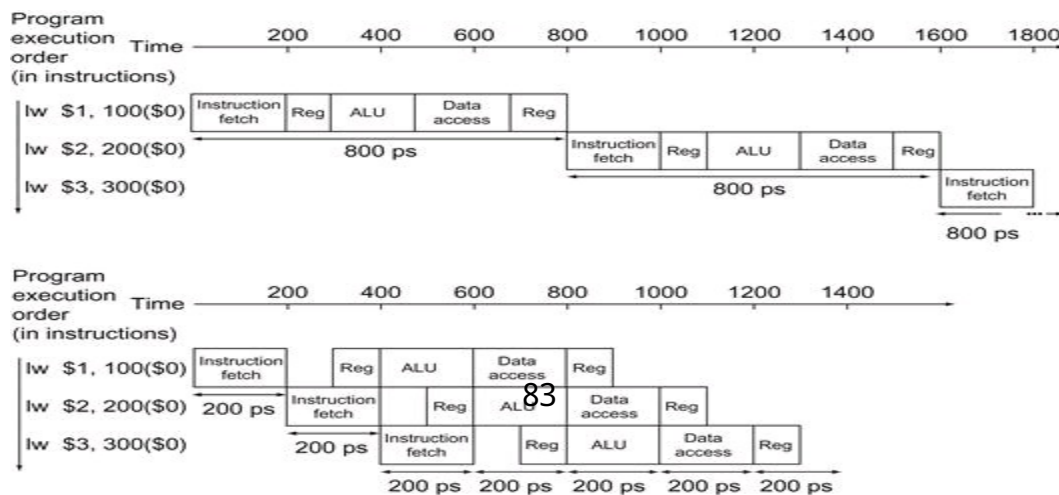
Let us compare the average time between instructions of a single-cycle implementation, in which all instructions take one clock cycle, to a pipelined implementation. The operation times for the major functional units in this example are :

- ⤴ 200 ps for memory access
- ⤴ 200 ps for ALU operation
- ⤴ 100 ps for register file read or write.

In the single-cycle model, every instruction takes exactly one clock cycle, so the clock cycle must be stretched to accommodate the slowest instruction.

Instruction class	Instruction fetch	Register read	ALU operation	Data access	Register write	Total time
Load word (lw)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store word (sw)	200 ps	100 ps	200 ps	200 ps		700 ps
R-format (add, sub, AND, OR, slt)	200 ps	100 ps	200 ps		100 ps	600 ps
Branch (beq)	200 ps	100 ps	200 ps			500 ps

Single-cycle, nonpipelined execution in top versus pipelined execution in bottom:



All the pipeline stages take a single clock cycle, so the clock cycle must be long enough to accommodate the slowest operation. Just as the single-cycle design must take the worst-case clock cycle of 800 ps, even though some instructions can be as fast as 500 ps, the pipelined execution clock

cycle must have the worst-case clock cycle of 200 ps, even though some stages take only 100 ps. Pipelining still offers a fourfold performance improvement: the time between the first and fourth instructions is 3×200 ps or 600 ps.

Pipelining speed-up formula :

If the stages are perfectly balanced, then the time between instructions on the pipelined processor

assuming ideal conditions is equal to

$$\text{Time between instructions}_{\text{pipelined}} = \frac{\text{Time between instructions}_{\text{nonpipelined}}}{\text{Number of pipe stages}}$$

Designing Instruction Sets for Pipelining :

1. First, all MIPS instructions are the same length. This restriction makes it much easier to fetch instructions in the first pipeline stage and to decode them in the second stage.
2. Second, MIPS has only a few instruction formats, with the source register fields being located in the same place in each instruction.
3. Third, memory operands only appear in loads or stores in MIPS. This restriction means we can use the execute stage to calculate the memory address and then access memory in the following stage.
4. Operands must be aligned in memory.

PIPELINED DATAPATH and CONTROL

The goal of pipelining is to allow multiple instructions execute at the same time. We may need to perform several operations in a clock cycle such as increment the PC and add registers at the same time, fetch one instruction while another one reads or writes data, etc.,

Pipelined Datapath

The division of an instruction into five stages means a five-stage pipeline, which in turn means that up to five instructions will be in execution during any single clock cycle. Thus, we must separate the datapath into five pieces, with each piece named corresponding to a stage of instruction execution.

1. IF: Instruction Fetch
2. ID: Instruction decode and register file read
3. EX: Execution or Address Calculation
4. MEM: Data Memory Access
5. WB: Write Back

In figure 3.31, these five components correspond roughly to the way the datapath is drawn; instructions and data move generally from left to right through the five stages as they complete execution.

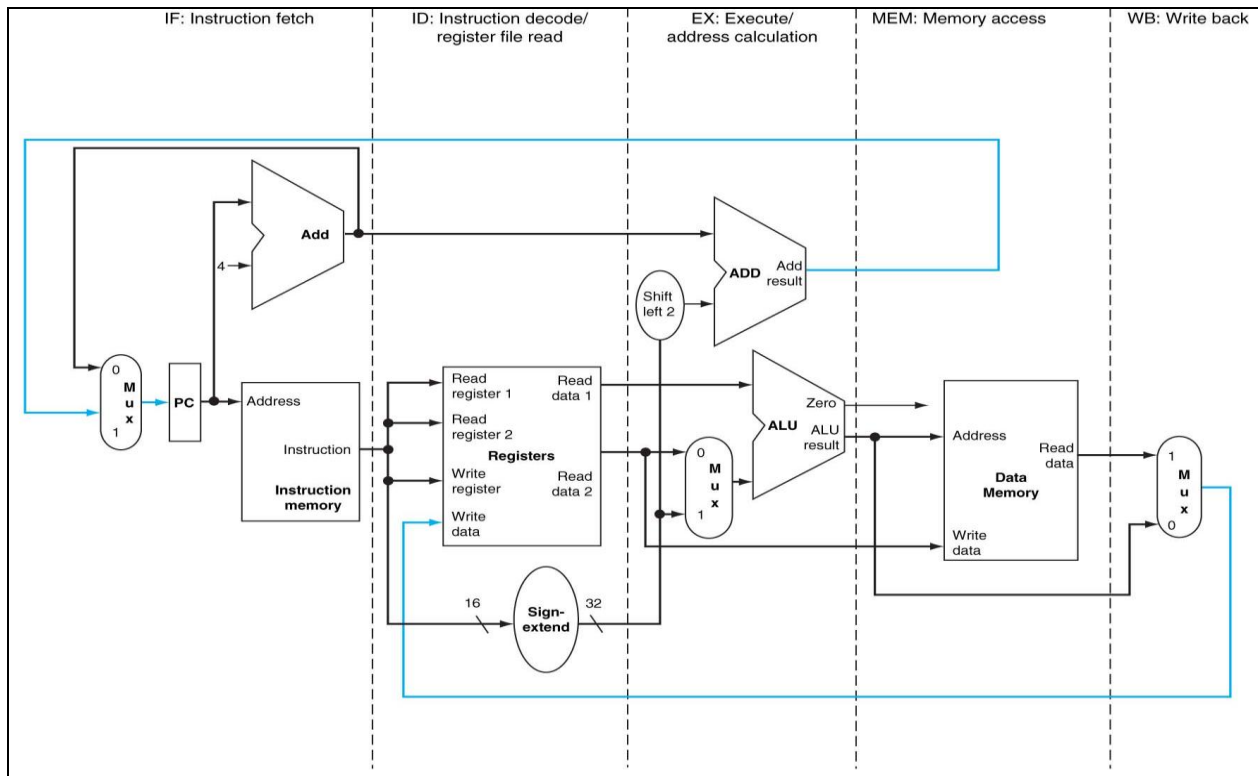


Figure 3.31 Pipeline Process for Single-Cycle Datapath

There are however, two exceptions to this left-to-right flow of instructions:

- The write-back stage, which places the result back into the register file in the middle of the datapath.
- The selection of the next value of the PC, choosing between the incremented PC and the branch address from the MEM stage.

Data flowing from left does not affect the current instruction; only later instructions in the pipeline are influenced by these reverse data movements. Note that the first right-to-left flow of data can lead to data hazards and the second leads to control hazards.

Figure 3.32 shows the execution of the instructions by displaying their private data paths on a common timeline.

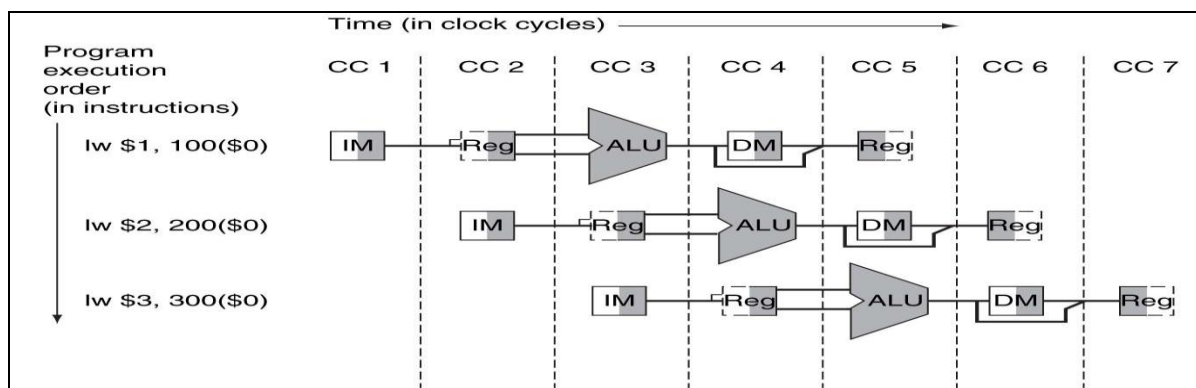


Figure 3.32 Instructions being executed using the single-cycle datapath in Figure 4.33, assuming pipelined execution

- This figure seems to suggest that three instructions need three datapaths.
- Instead, add registers to hold data so that portions of a single datapath can be shared during instruction execution.
- The instruction memory is used during only one of the five stages of an instruction, allowing it to be shared by following instructions during the other four stages.
- To retain the value of an individual instruction for its other four stages, the value read from instruction memory must be saved in a register.

Figure 3.33 shows the pipelined datapath with the pipeline registers highlighted.

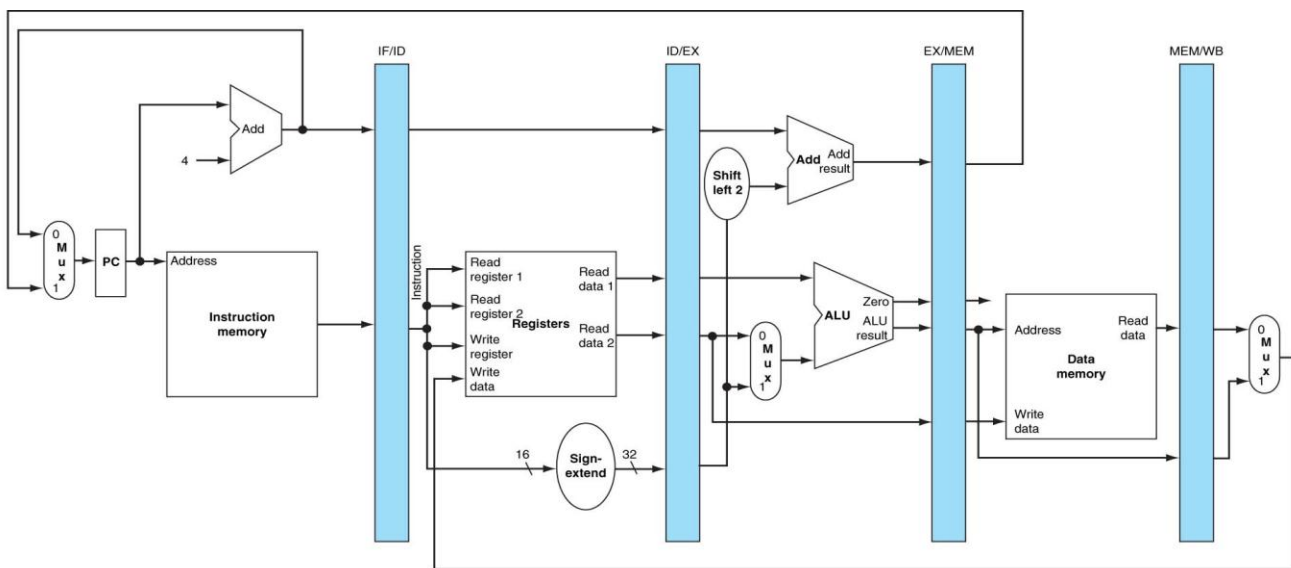


Figure 3.33 The pipelined version of the datapath in Figure 3.32

Examples:

To show how the pipelining works, throughout this chapter we show sequences of figures to demonstrate operation over time.

Pipelined Datapath for a Load Instruction:

The following sequence of diagrams shows the active portions of the datapath highlighted as a load instruction goes through the five stages of pipelined execution.

The five stages of the load instruction are the following:

1. Instruction Fetch: Figure 3.34 shows the instruction being read from memory using the address in the PC and then being placed in the IF/ID pipeline register.
2. The PC address is incremented by 4 and then written back into the PC to be ready for the next clock cycle. This incremented address is saved in the IF/ID pipeline register in case it is needed later for an instruction, such as beq.
3. The computer cannot know which type of instruction is being fetched, so it must prepare for any instruction, passing potentially needed information down the pipeline.

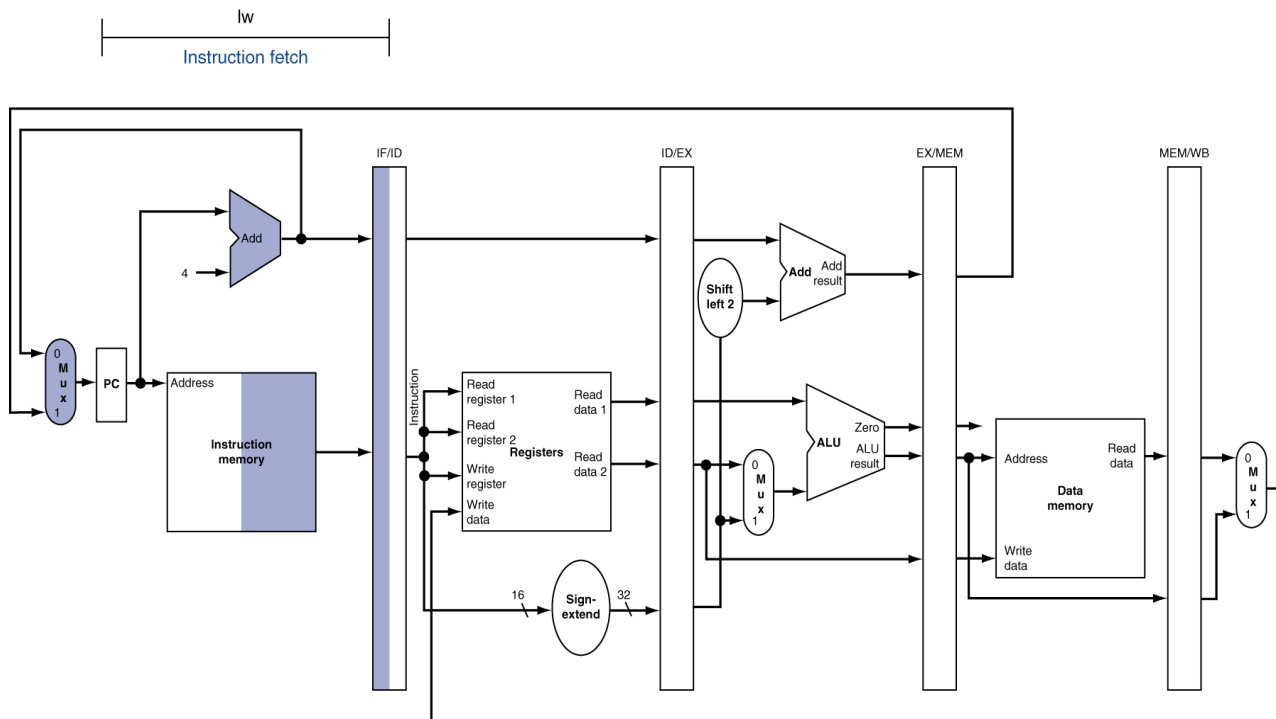


Figure 3.34 First stage (IF) of a Load instruction, with the active portions of the datapath in Figure 3.33 highlighted

4. Instruction Decode and Read Register File Read: Figure 3.35 shows the instruction decode portion of the IF/ID pipeline register supplying the 16-bit immediate field, which is sign-extended to 32-bits, and the register numbers to read the two registers. All three values are stored in the ID/EX pipeline register, along with the incremented PC address..

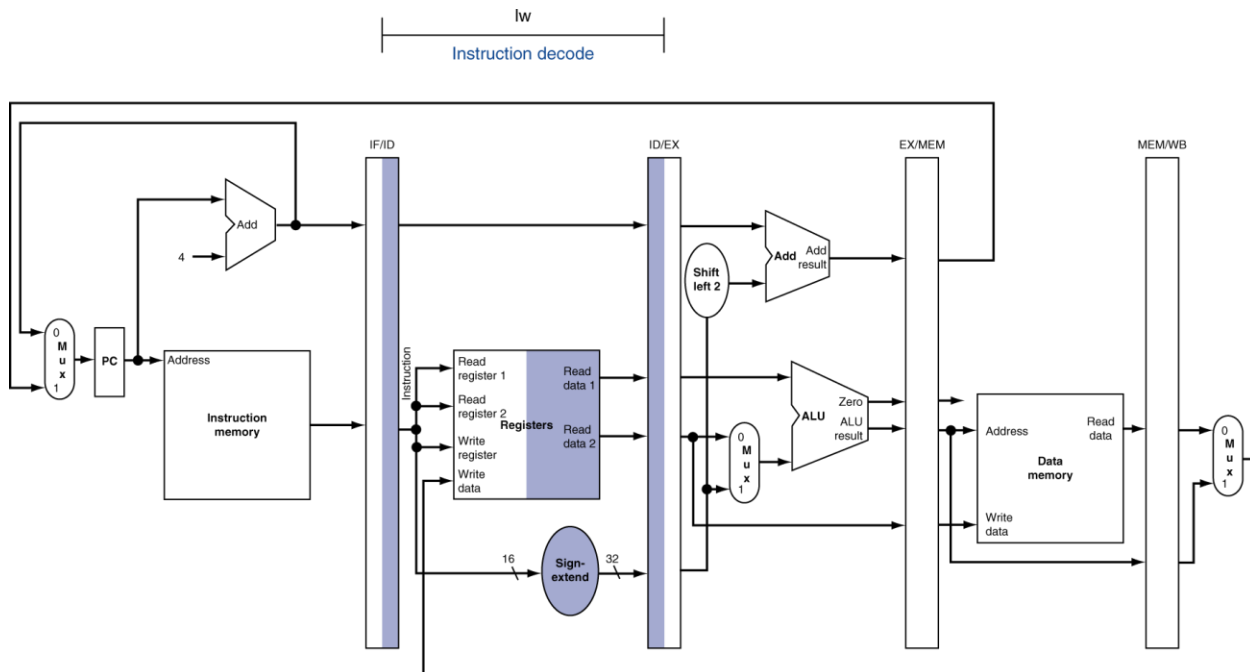


Figure 3.35 Second stage (ID) of a Load instruction, with the active portions of the datapath in Figure 3.33 highlighted

5. Execute or Address Calculation: Figure 3.36 shows that the load reads the contents of register 1 and the sign-extended immediate from the ID/EX pipeline register and adds them using the ALU. That sum is placed in the EX/MEM pipeline register.

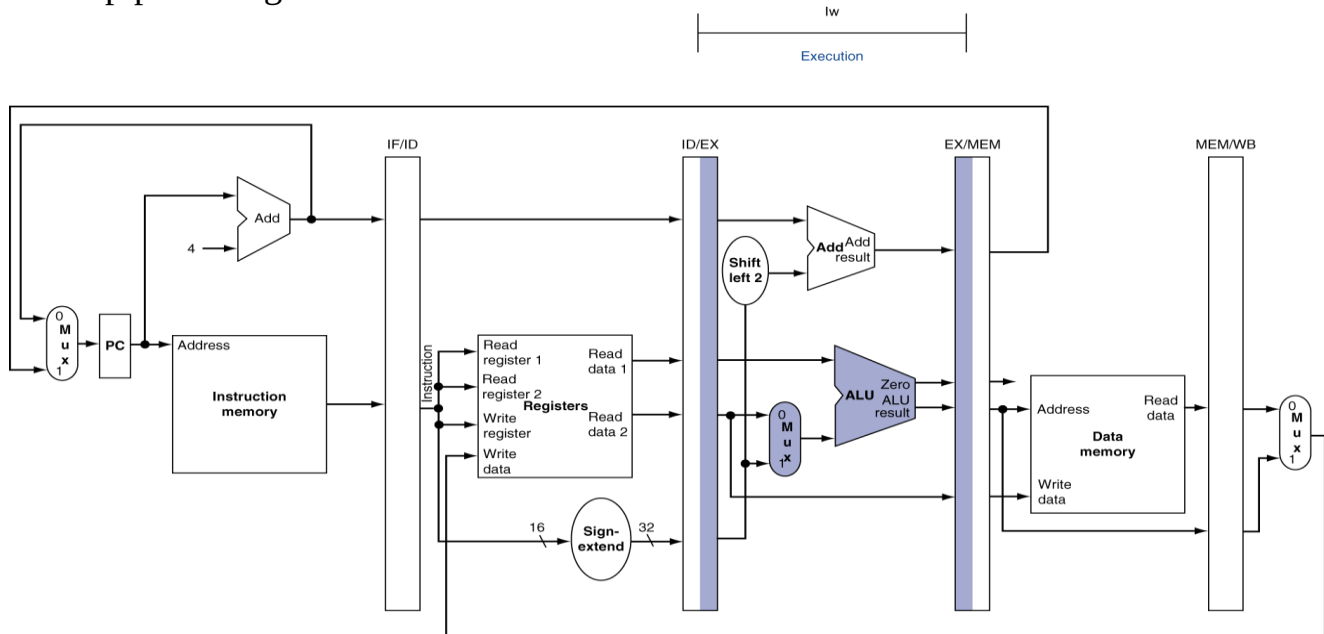


Figure 3.36 Third stage (EX) of a Load instruction, with the active portions of the datapath in Figure 3.33 highlighted

6. Memory Access: Figure 3.37 shows the load instruction reading the data memory using the address from the EX/MEM pipeline register and loading the data into the MEM/WB pipeline register.

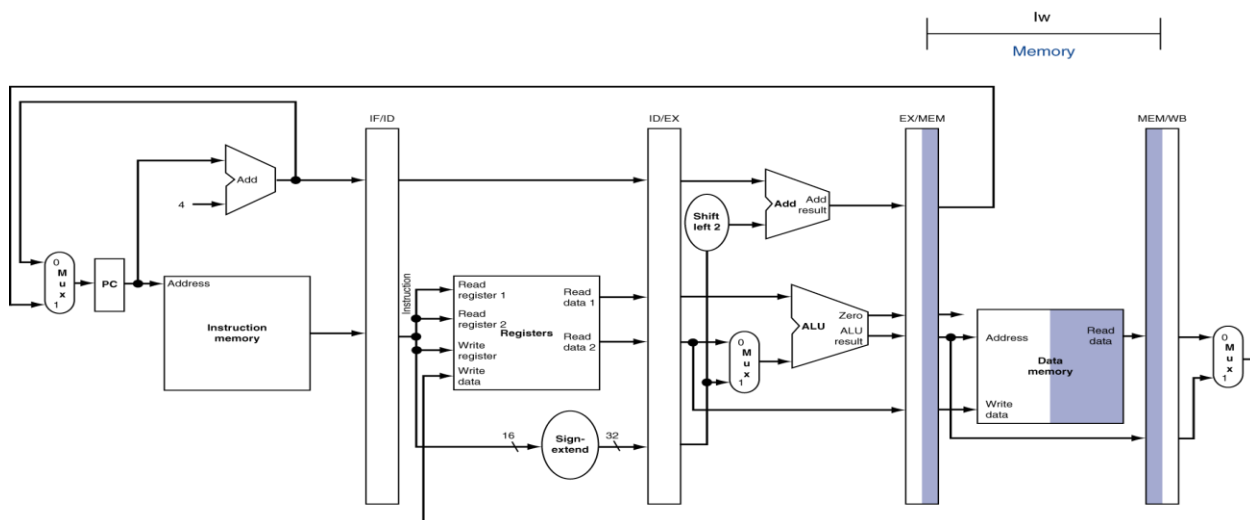


Figure 3.37 Fourth stage (MEM) of a Load instruction, with the active portions of the datapath in Figure 3.33 highlighted

7. Write-Back: Figure 3.38 shows the final step – reading the data from the MEM/WB pipeline register and writing it into the register file in the middle of the figure.

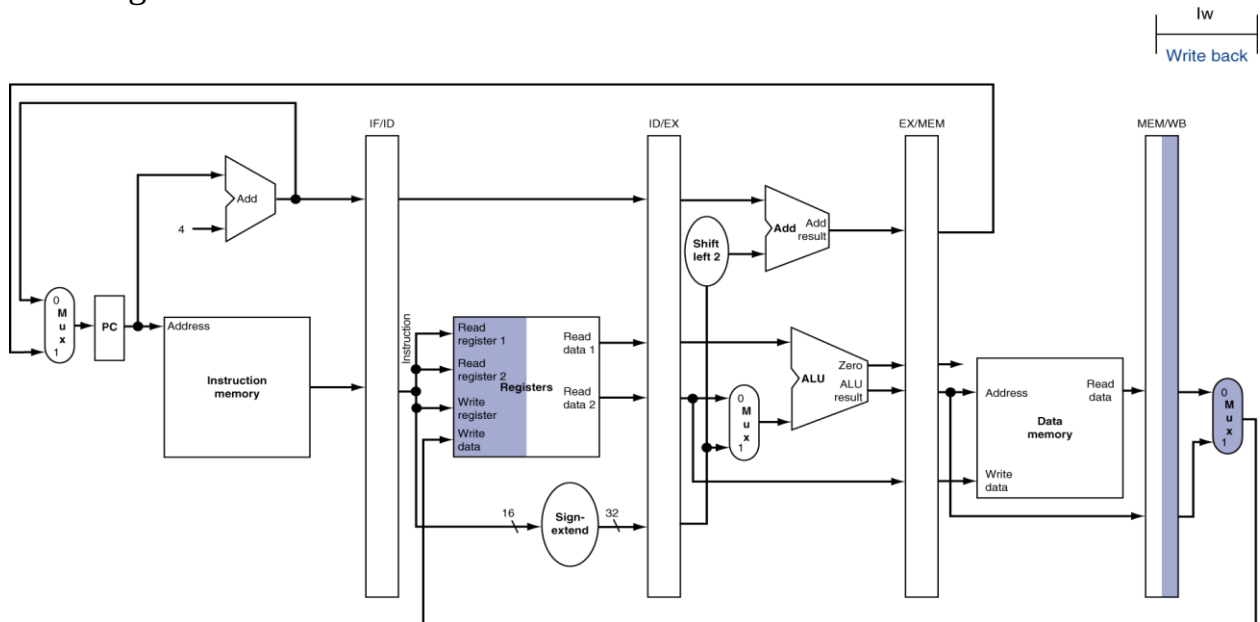


Figure 3.38 Fifth stage (WB) of a Load instruction, with the active portions of the datapath in Figure 3.33 highlighted

Pipelined Datapath for a Store Instruction:

Here are the five pipe stages of the store instruction:

1. Instruction Fetch: the instruction is read from memory using the address in the PC and then is placed in the IF/ID pipeline register. This stage occurs before the instruction is identified, so the figure 3.34 works for store as well.

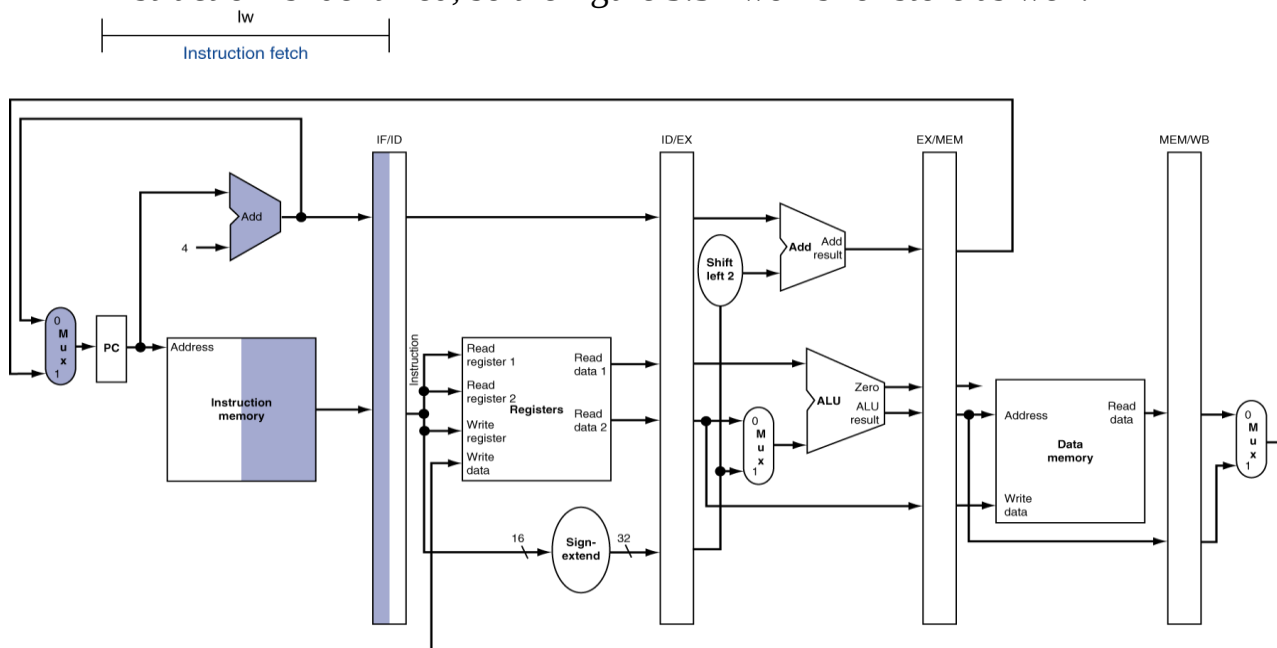


Figure 3.39 First stage (IF) of a Store instruction, with the active portions of the datapath

in Figure 3.33 highlighted – same as Figure 3.34

2. Instruction Decode and Read Register File: The instruction in the IF/ID pipeline register supplies the register numbers for reading two registers and extends the sign of the 16-bit immediate. These three 32-bit values are all stored in the ID/EX pipeline register. Figure 3.35 for load instruction also shows the operations of the second stage for stores.

These first two stages are executed by all instructions, since it is too early to know the type of the instruction.

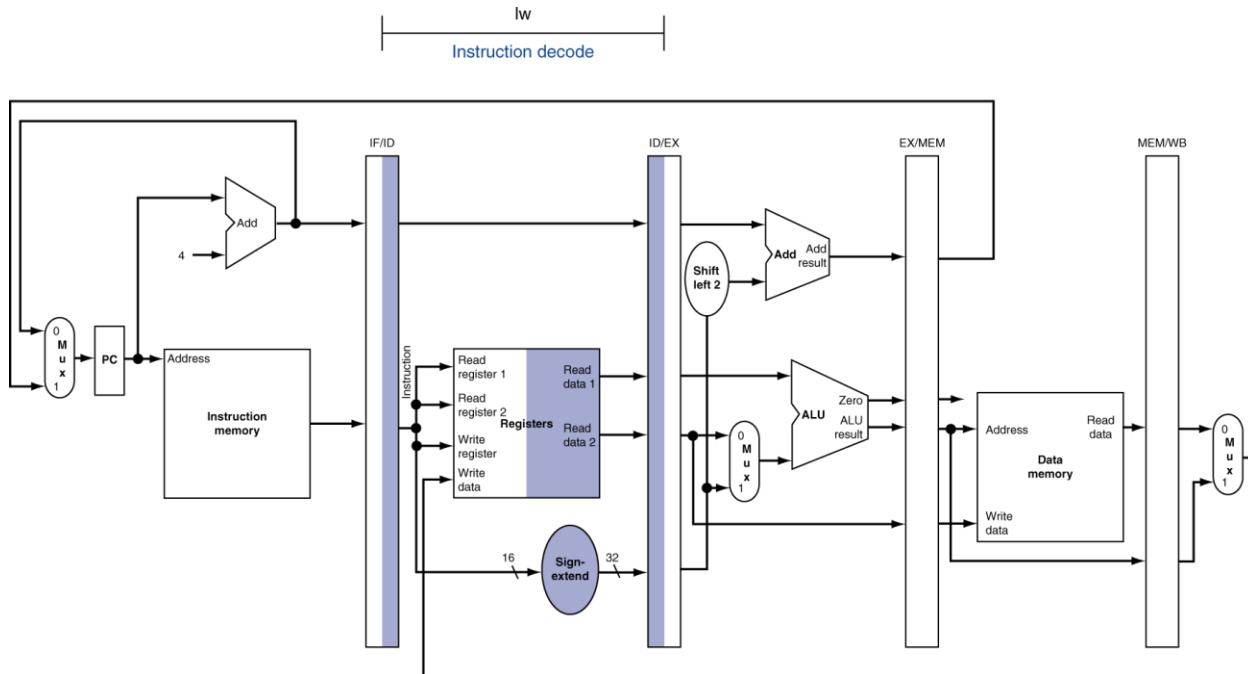


Figure 3.40 Second stage (ID) of a Store instruction, with the active portions of the datapath in Figure 3.33 highlighted – same as Figure 3.35

3. Execute and Address Calculation: Figure 3.41 shows the third step of a store instruction in which the effective address is placed in the EX/MEM pipeline register.

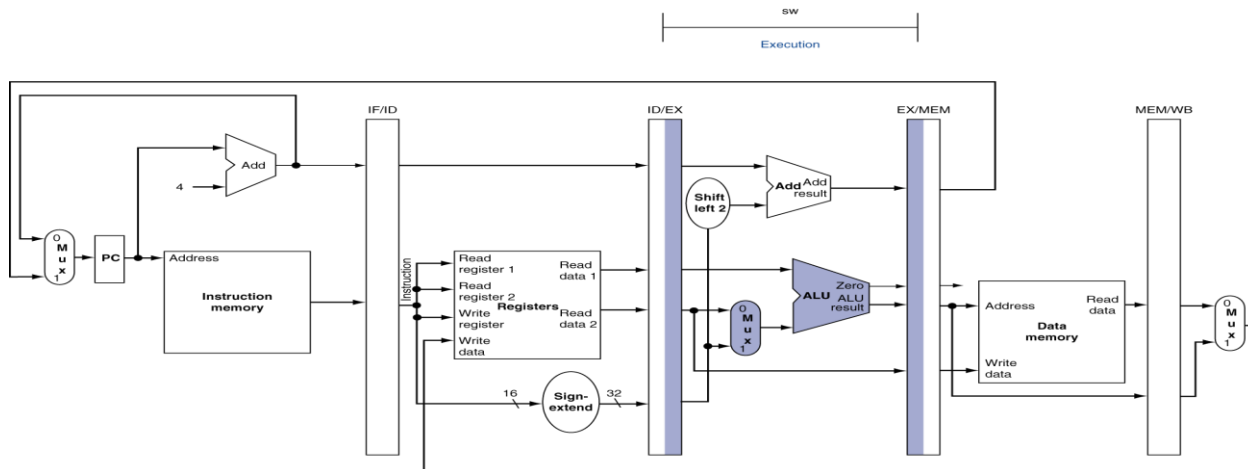


Figure 3.41 Third stage (EX) of a Store instruction, with the active portions of the datapath in Figure 3.33 highlighted

4. Memory Access: Figure 3.42 shows the data being written to memory. Note that the register containing the data to be stored was read in an earlier stage and stored in ID/EX. The only way to make the data available during the MEM stage is to place the data into the EX/MEM pipeline register in the EX stage, just as we stored the effective address into EX/MEM.

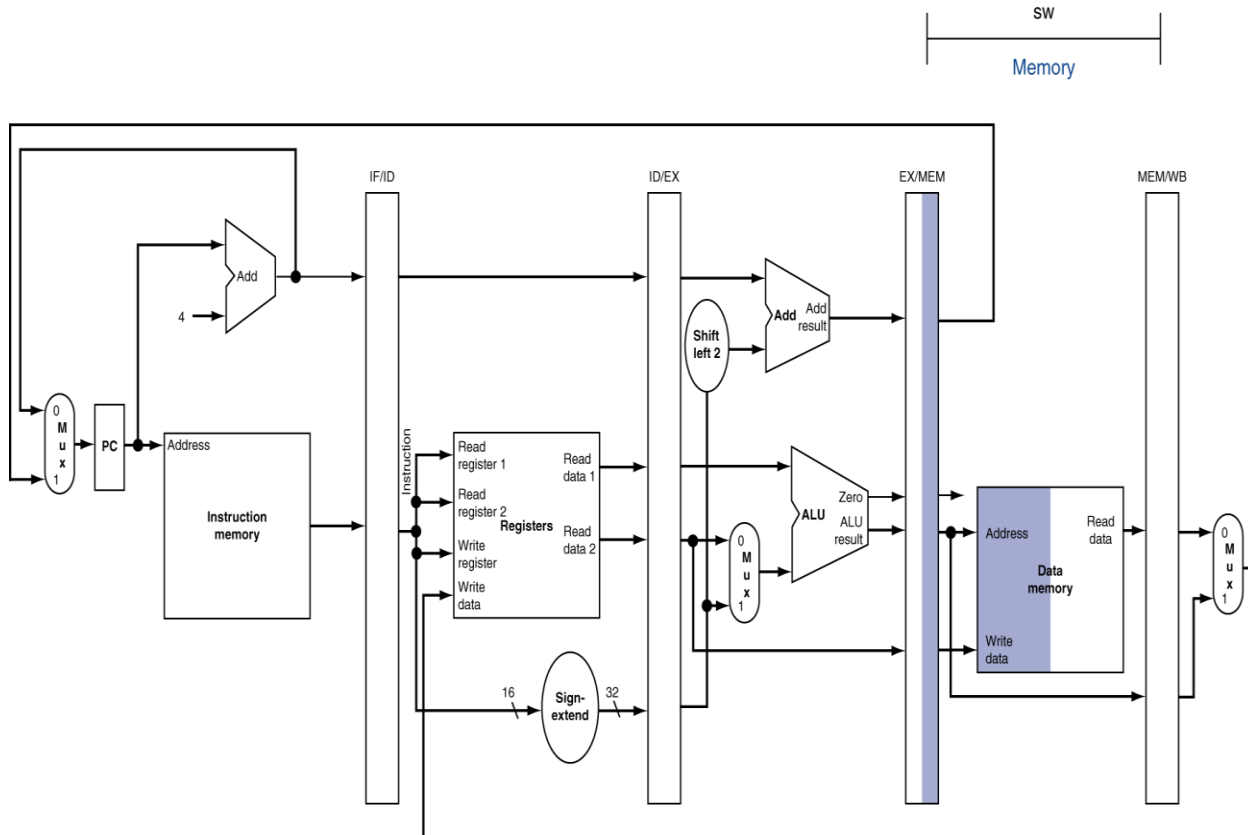


Figure 3.42 Fourth stage (MEM) of a Store instruction, with the active portions of the datapath in Figure 3.33 highlighted

Figure 3.33 highlighted

5. Write-Back: Figure 3.43 how the final step of the store. For this instruction, nothing happens in the write-back stage. Since every instruction behind the store is already in progress, we have no way to accelerate those instructions. Hence, an instruction passes through a stage even if there is nothing to do, because later instructions are already progressing at the maximum rate.

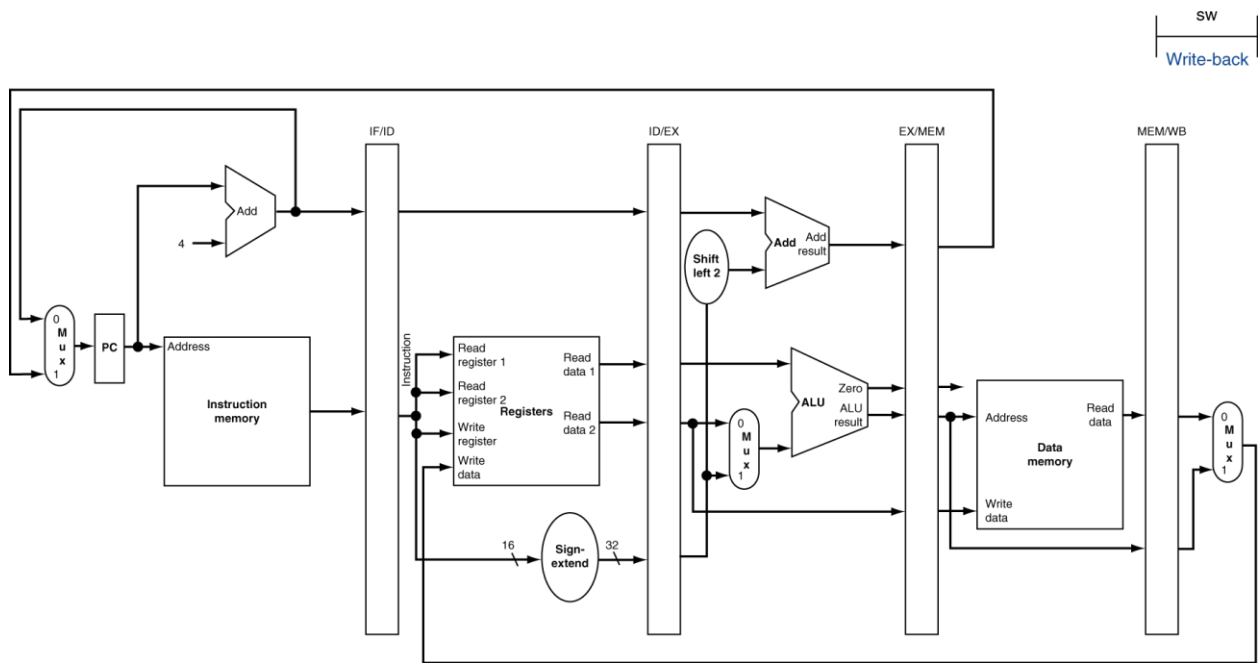


Figure 3.43 Fifth stage (WB) of a Store instruction, with the active portions of the datapath in

Figure 3.33 highlighted

Both load and store illustrate a second key point: each logical component of the datapath – such as instruction memory, register read ports, ALU, data memory and register write port – can be used only within a single pipeline stage. Otherwise, we would have a structural hazard. Hence these components, and their control, can be associated with a single pipeline stage.

A bug in the design of the load instruction.

Figure 3.44 shows the correct version of the datapath, passing the write register number first to the ID/EX register, then to the EX/MEM register, and finally to the MEM/WB register. The register number is used during the WB stage to specify the register to be written.

Figure 3.45 shows a single drawing of the corrected datapath, highlighting the hardware used in all five stages of the load word instruction.

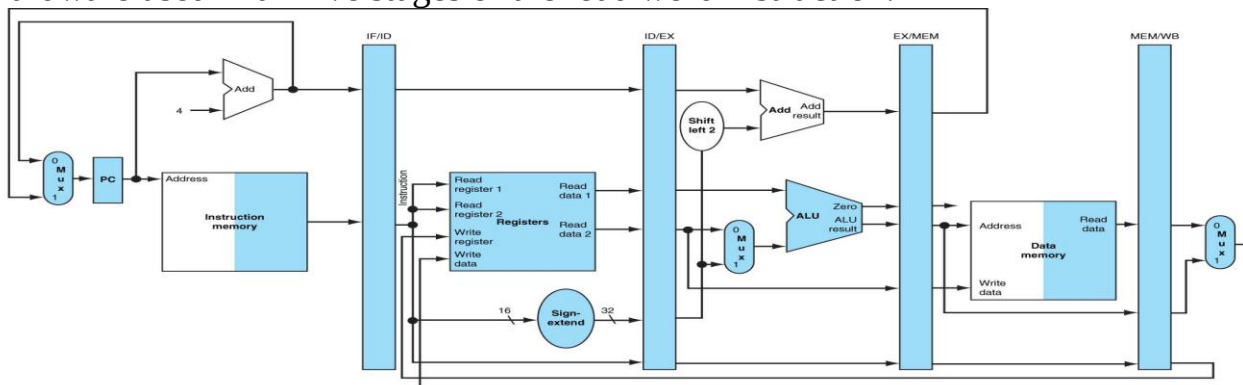


Figure 3.45 The portion of the datapath

Graphically Representing Pipelines

Pipelining can be difficult to understand, since many instructions are simultaneously executing in a single datapath in every clock cycle. To aid understanding, there are two basic styles of representing pipeline figures:

1. Multiple clock-cycle pipeline diagrams
2. Single clock-cycle pipeline diagrams

For example consider the following five instruction sequence:

```
lw    $t0, 20($t1)
sub    $t1, $t2, $t3
add    $t2, $t3, $t4
lw     $t3, 24($t1)
add    $t4, $t5, $t6
```

Multiple Clock-Cycle Pipeline Diagrams:

The multiple clock-cycle diagrams are simpler but do not contain all the details. Figure 3.46 shows the multiple clock-cycle pipeline diagram for these instructions. Time advances from left to right across the page in these diagrams, and instructions advance from the top to bottom of the page.

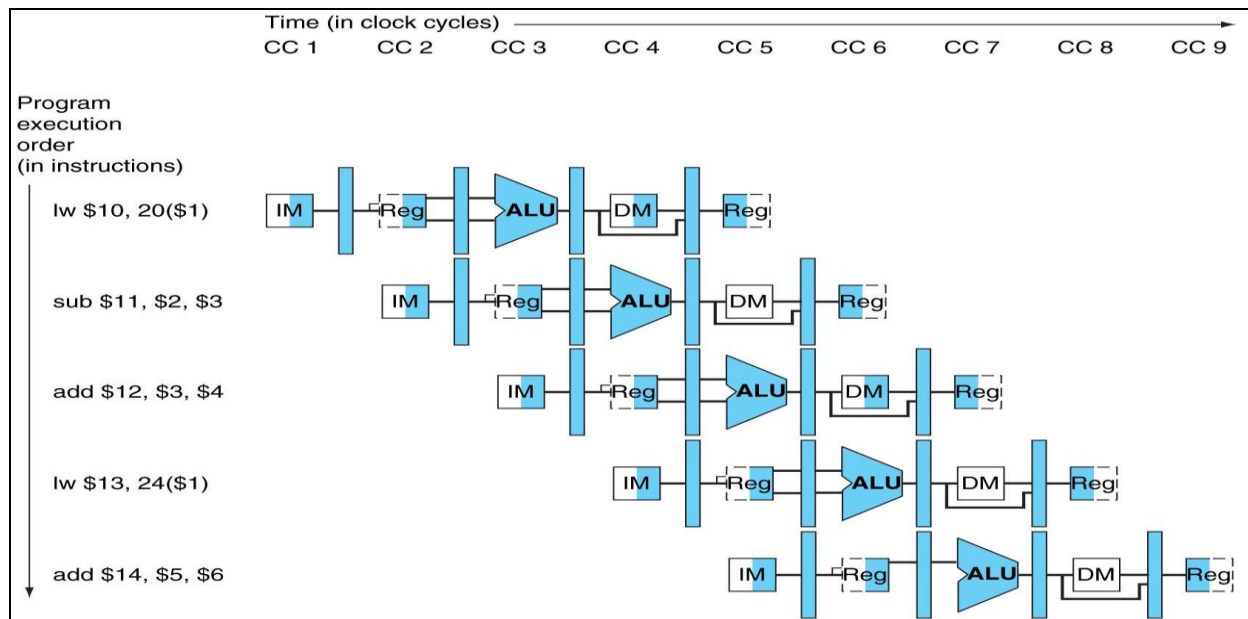


Figure 3.46 Multiple-clock-cycle pipeline diagram of five instructions

A representation of the pipeline stages is placed in each portion along the instruction axis, occupying the proper clock cycles. These stylized datapaths represent the five stages of our pipeline graphically, but a rectangle naming each pipe stage works just as well. Figure 3.47 shows the more traditional version of the multiple clock-cycle pipeline diagram. Figure 3.46 shows the physical resources used at each stage, while Figure 3.47 uses the name of each stage.

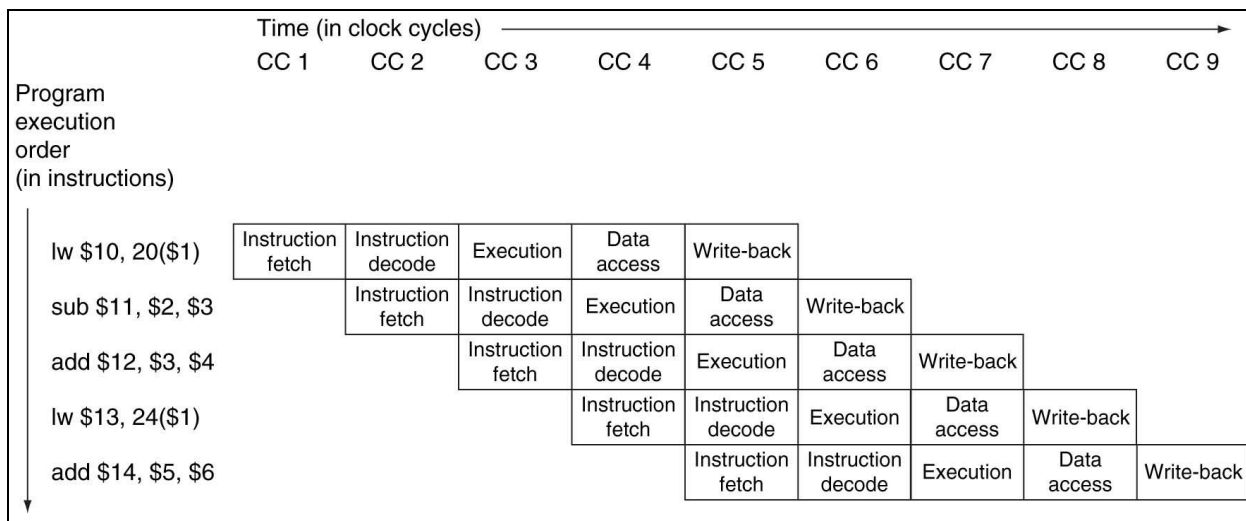


Figure 3.47 Traditional multiple-clock-cycle pipeline diagram of five instructions in Figure 3.46

Single Clock-Cycle Pipeline Diagrams:

Single clock-cycle pipeline diagrams show the state of the entire datapath during a single clock cycle, and usually all five instructions in the pipeline are identified by labels above their respective pipeline stages.

A single clock-cycle diagram represents a vertical slice through a set of multiple clock-cycle diagrams, showing the usage of the datapath by each of the instructions in the pipeline at the designated clock cycle. The single clock-cycle diagrams have more detail and take significantly more space to show the same number of clock cycles.

PIPELINE CONTROL

Just as we added control to the single-cycle datapath, we now add control to the pipelined datapath. To provide control in a pipelined datapath, the following task has to be followed:

- The first step is to label the control lines on the existing datapath.
- Borrow/ Use the control lines from the simple datapath.
- Use the same ALU control logic, branch logic, destination-register-number multiplexor, and control lines.

To specify control for the pipeline, we need only to set the control values during each pipeline stage. Because each control line is associated with a component active in only a single pipeline stage, we can divide the control lines into five groups according to the pipeline stage.

1. Instruction Fetch: The control signals to read instruction memory and to write the PC are always asserted, so there is nothing special to control in this pipeline stage.
2. Instruction Decode / Register File Read: As in the previous stage, the same thing happens at every clock cycle, so there are no optional control lines to set.

3. Execution / Address Calculation: The signals to be set are RegDst, ALUOp, and ALUSrc. These signals select the result register, the ALU operation, and either read data 2 or a sign-extended immediate for the ALU.
4. Memory Access: The control lines set in this stage are Branch, MemRead, and MemWrite. These signals are set by the branch equal, load, and store instructions, respectively. The PCSrc selects the next sequential address unless control asserts Branch and the ALU result was 0.
5. Write Back: The two control lines are MemtoReg, which decides between sending the ALU result or the memory value to the register file, and RegWrite, which writes the chosen value.

Figure 3.50 shows that these control signals are then used in the appropriate pipeline stage as the instruction moves down the pipeline, just as the destination register number for loads moves down the pipeline.

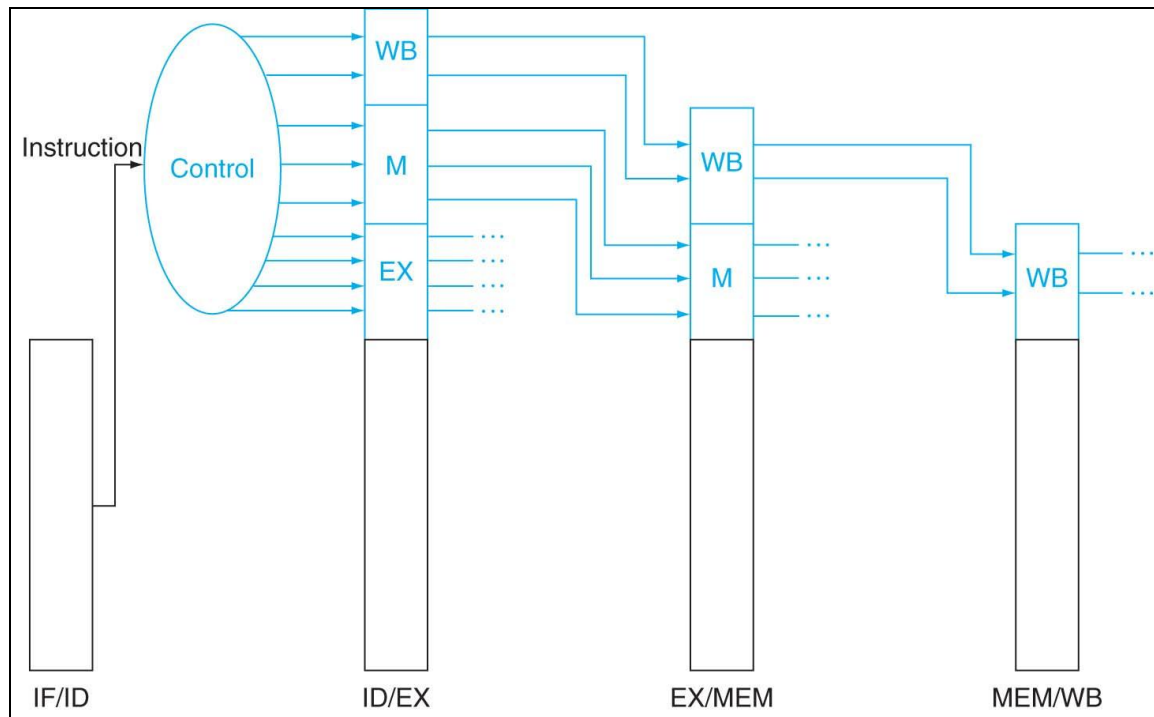


Figure 3.50 The control lines for the final three stages

Implementing control means setting the nine control lines to these values in each stage for each instruction. The simplest way to do this is to extend the pipeline registers to include control information. Figure 3.51 shows the full datapath with the extended pipeline registers and with the control lines connected to the proper stage.

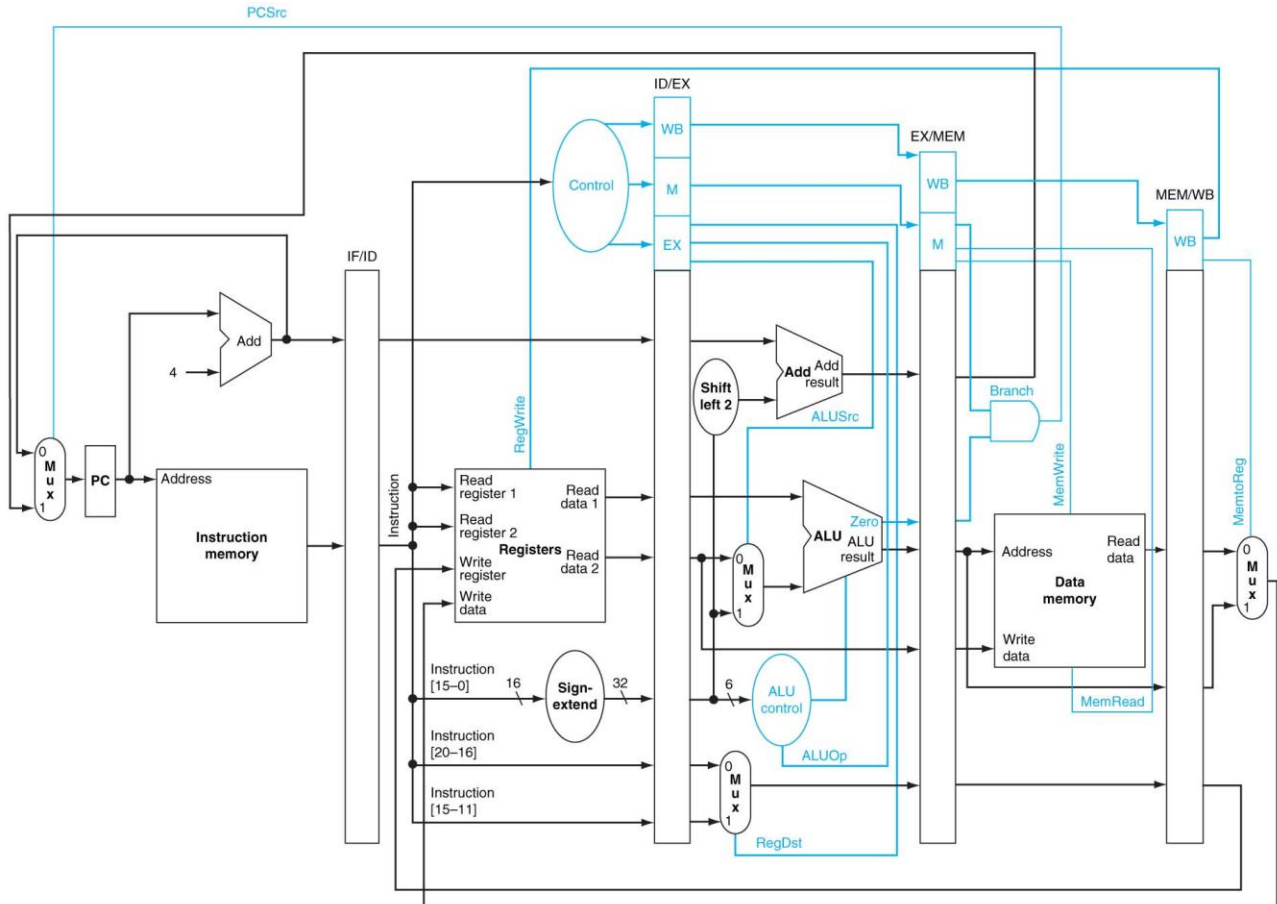


Figure 3.51 The pipelined datapath of Figure 3.49, with the control signals connected to the control portions of the pipe line registers

PIPELINE HAZARDS OR STALL

There are situations in pipelining when the next instruction cannot be executed in the following clock cycle. These events are called hazards, and there are three different types.

1. Structural hazard.
2. Data hazards
3. Control hazard

1. Structural hazard: When a planned instruction cannot execute in the proper clock cycle because the hardware does not support the combination of instructions that are set to execute. In other words, when more than one instruction try to access the same hardware, there is a delay in pipeline execution called Structural **hazard**.

In general MIPS instruction set was designed to be pipelined, making it fairly easy for designers to avoid structural hazards when designing a pipeline. Suppose, however, that there is a single memory instead of two memories.

Example Structural Hazard

	1	2	3	4	5	6	7	8	9
ld r2, 0(r1)	F	D	X	M	W				
add r1, r3, r4		F	D	X	M	W			
sub r1, r3, r5			F	D	X	M	W		
st r6, 0(r1)				F	D	X	M	W	

- **Structural hazard:** resource needed twice in one cycle
 - Example: unified instruction & data cache
 - Solutions?
 - Separate instruction/data caches
 - Redesign cache to allow 2 accesses per cycle (slow, expensive)
 - Stall pipeline

32

2. Data hazard : Also called a pipeline data hazard. When a planned instruction cannot execute in the proper clock cycle because data that is needed to execute the instruction is not yet available.

In a computer pipeline, data hazards arise from the dependence of one instruction on an earlier one that is still in the pipeline (a relationship that does not really exist when doing laundry). For example, suppose we have an add instruction followed immediately by a subtract instruction that uses the sum (\$s0):

Eg:

```
add $s0, $t0, $t1
sub $t2, $s0, $t3
```

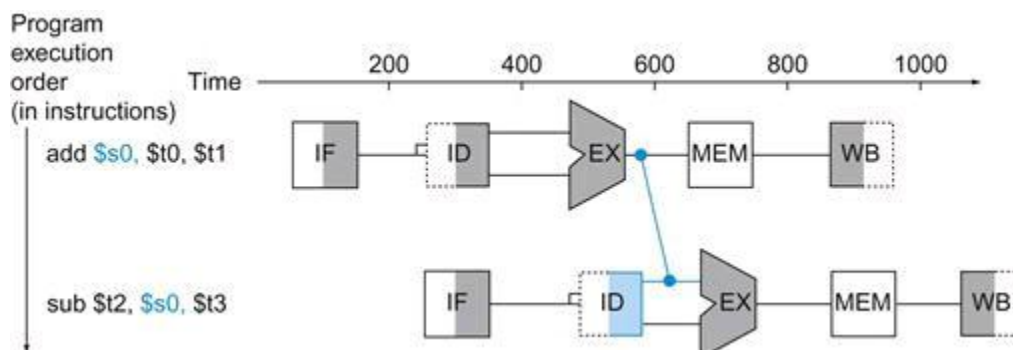
Here one of the source operand in the sub instruction is \$s0 which is the destination operand of add instruction. Hence there exist data dependency between those two instructions. The add instruction does not write its result until the fifth stage, hence the sub instruction cannot be executed till the result of add instruction is available meaning that we would have to waste three clock cycles in the pipeline.

Without intervention, a data hazard could severely stall the pipeline..

The **primary solution** for data hazard is **data forwarding**.

Data forwarding : Also called bypassing. A method of resolving a data hazard by retrieving the missing data element from internal buffers rather than waiting for it to arrive from programmer-visible registers or memory.

Graphical representation of data forwarding:



The connection shows the forwarding path from the output of the EX stage of add to the input of the EX stage for sub, replacing the value from register \$s0 read in the second stage of sub.

Forwarding works very well, it cannot prevent all pipeline stalls, however. For example, suppose the first instruction was a load of \$s0 instead of an add. The desired data would be available only after the fourth stage of the first instruction in the dependence, which is too late for the input of the third stage of sub. Hence, even with forwarding, it would have to stall one stage for a load-use data hazard.

There are several other ways to handle such hard cases such as:

- ⤴ Hardware detection
- ⤴ Stalls or software that reorders code

Reordering code to Avoid Pipeline Stalls:

Example

Consider the following code segment in C:

a = b + e;

c = b + f;

Here is the generated MIPS code for this segment, assuming all variables are in memory and are addressable as offsets from \$t0:

```
lw $t1, 0($t0)
lw $t2, 4($t0)
add $t3, $t1,$t2
sw $t3, 12($t0)
lw $t4, 8($t0)
add $t5, $t1,$t4
sw $t5, 16($t0)
```

Find the hazards in the preceding code segment and reorder the instructions to avoid any pipeline stalls.

Answer

Both add instructions have a hazard because of their respective dependence on the immediately preceding lw instruction. Notice that bypassing eliminates several other potential hazards, including the dependence of the first add on the first lw and any hazards for store instructions. Moving up the third lw instruction to become the third instruction eliminates both hazards:

```
lw    $t1, 0($t0)
lw    $t2, 4($t0)
lw    $t4, 8($t0)
add   $t3, $t1,$t2
sw    $t3, 12($t0)
add   $t5, $t1,$t4
sw    $t5, 16($t0)
```

On a pipelined processor with forwarding, the reordered sequence will complete in two fewer cycles than the original version.

Load-use data hazard

A specific form of data hazard in which the data being loaded by a load instruction has not yet become available when it is needed by another instruction.

Pipeline stall

Also called bubble. A stall initiated in order to resolve a hazard.

DATA FORWARDING TO AVOID PIPE LINE STALL

Note that when we detect hazards, half of the problem is solved – but still we must forward the proper data. Figure 3.53 shows the dependences between the pipeline registers and the inputs to the ALU for the same code sequence. The change is that the dependence begins from a pipeline register, rather than waiting for the WB stage to write the register file. Thus, the required data exists in time for later instructions, with the pipeline registers holding the data to be forwarded.

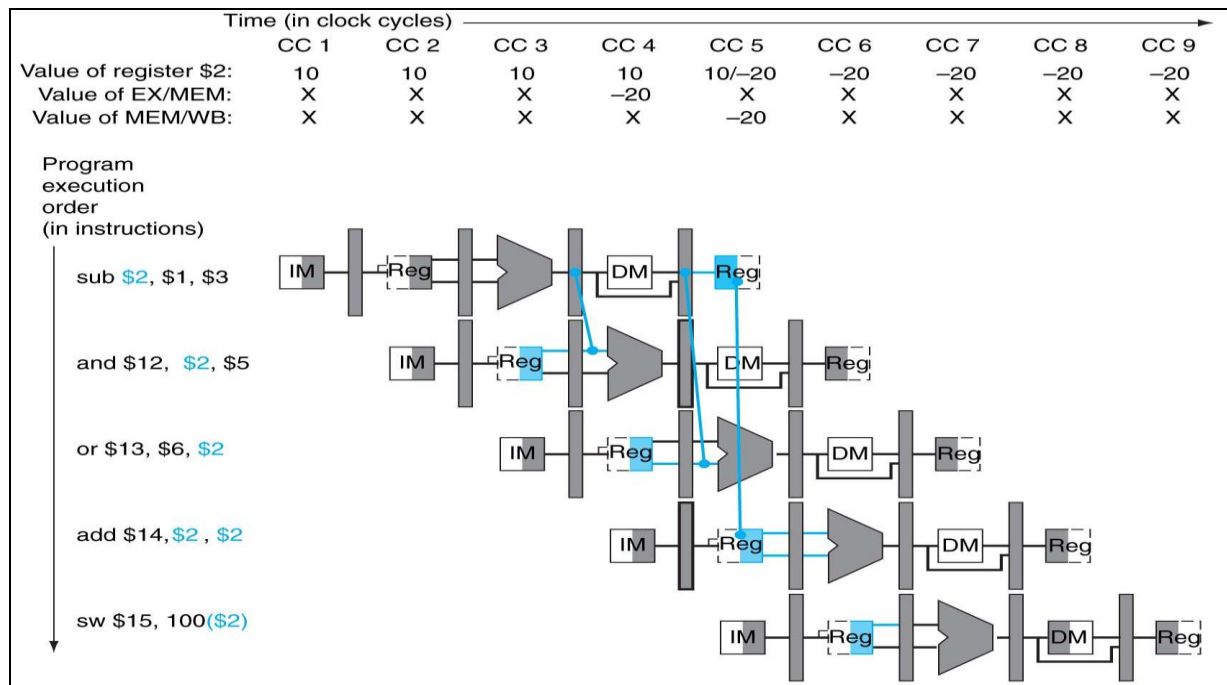
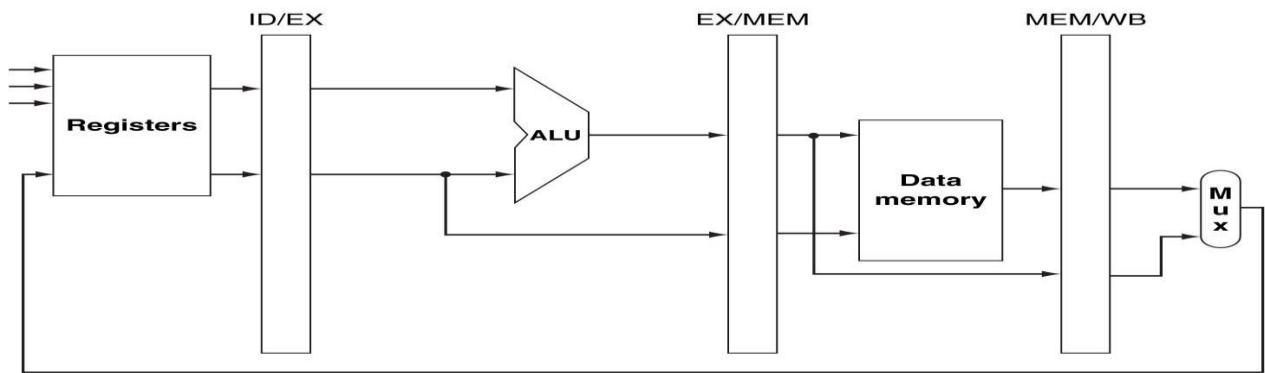
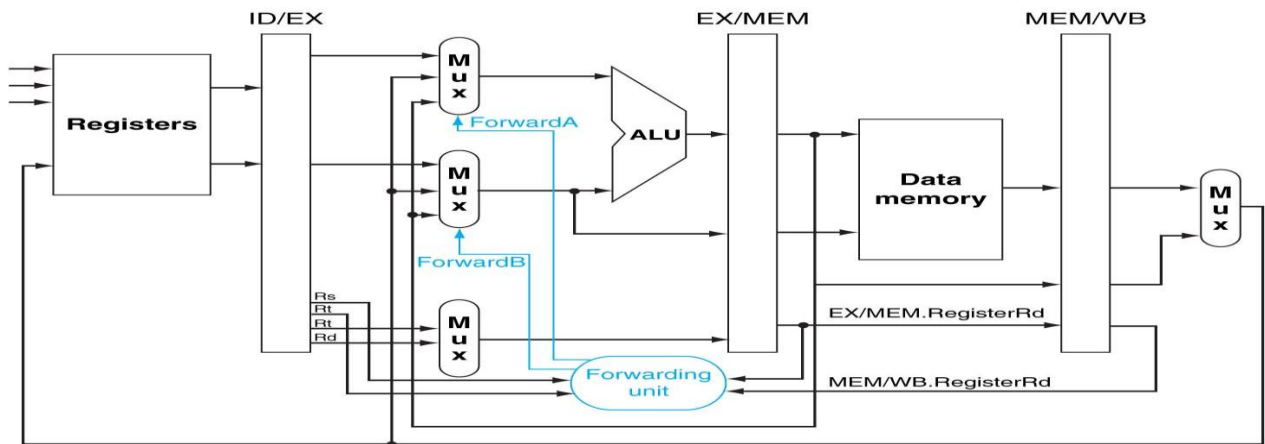


Figure 3.53 The dependences between the pipeline registers move forward in time, so it is possible to supply the inputs to the ALU needed by the AND instruction and OR instruction by forwarding the results found in the pipeline registers

Figure 3.54 shows a close-up of the ALU and pipeline register before and after adding forwarding. This forwarding control will be in the EX stage, because the ALU forwarding multiplexors are found in that stage. Thus, we must pass the operand register numbers from the ID stage via the ID/EX pipeline register to determine whether to forward values. We already have the rt field. Before forwarding, the ID/EX register had no need to include space to hold the rs field. Hence, rs is added to ID/EX.



a. No forwarding



b. With forwarding

Figure 3.54 on the top are the ALU and pipeline registers before adding forwarding. Let's now write both the conditions for detecting hazards and the control signals to resolve them:

1. EX Hazard:

if (EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq$ 0)
 and (EX/MEM.RegisterRd = ID/EX.RegisterRs)) ForwardA = 10
 if (EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq$ 0)
 and (EX/MEM.RegisterRd = ID/EX.RegisterRt)) ForwardB = 10

Note that the EX/MEM.RegisterRd field is the register destination for either an ALU instruction or a load instruction. This case forwards the result from the previous instruction to either input of the ALU. If the previous instruction is going to write to the register file, and the write register number matches the read register number of ALU inputs A or B, provided it is not register 0, then the multiplexor to pick the value instead from the pipeline register EX/MEM.

Figure 3.55 shows the values of the control lines for the ALU multiplexors that select either the register file values or one of the forwarded values.

Mux control	Source	Explanation
ForwardA = 00	ID/EX	The first ALU operand comes from the register file.
ForwardA = 10	EX/MEM	The first ALU operand is forwarded from the prior ALU result.
ForwardA = 01	MEM/WB	The first ALU operand is forwarded from data memory or an earlier ALU result.
ForwardB = 00	ID/EX	The second ALU operand comes from the register file.
ForwardB = 10	EX/MEM	The second ALU operand is forwarded from the prior ALU result.
ForwardB = 01	MEM/WB	The second ALU operand is forwarded from data memory or an earlier ALU result.

Figure 3.55 The control values for the forwarding multiplexors in Figure 3.54

2. MEM Hazard:

if (MEM/WB.RegWrite and (MEM/WB.RegisterRd $\neq$ 0)
and (MEM/WB.RegisterRd = ID/EX.RegisterRs)) ForwardA = 01
if (MEM/WB.RegWrite and (MEM/WB.RegisterRd $\neq$ 0)
and (MEM/WB.RegisterRd = ID/EX.RegisterRt)) ForwardB = 01

As mentioned above, there is no hazard in the WB stage, because we assume that the register file supplies the correct result if the instruction in the ID stage reads the same register written by the instruction in the WB stage. Such a register file performs another form of forwarding, but it occurs within the register file.

One complication is potential data hazards between the result of the instruction in the WB stage, the result of the instruction in the MEM stage, and the source operand of the instruction in the ALU stage.

For example, when summing a vector of numbers in a single register, a sequence of instructions will all read and write to the same register:

Add \$1, \$1, \$2
Add \$1, \$1, \$3
Add \$1, \$1, \$4
.....

In this case, the result is forwarded from the MEM stage because the result in the MEM stage is the more recent result. Thus, the control for the MEM hazard would be :

if (MEM/WB.RegWrite
and (MEM/WB.RegisterRd $\neq$ 0)
and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq$ 0)
and (EX/MEM.RegisterRd = ID/EX.RegisterRs))
and (MEM/WB.RegisterRd = ID/EX.RegisterRs)) ForwardA = 01
if (MEM/WB.RegWrite
and (MEM/WB.RegisterRd $\neq$ 0)
and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq$ 0)
and (EX/MEM.RegisterRd = ID/EX.RegisterRt))
and (MEM/WB.RegisterRd = ID/EX.RegisterRt)) ForwardB = 01

Figure 3.56 shows the hardware necessary to support forwarding for operations that use results during the EX stage.

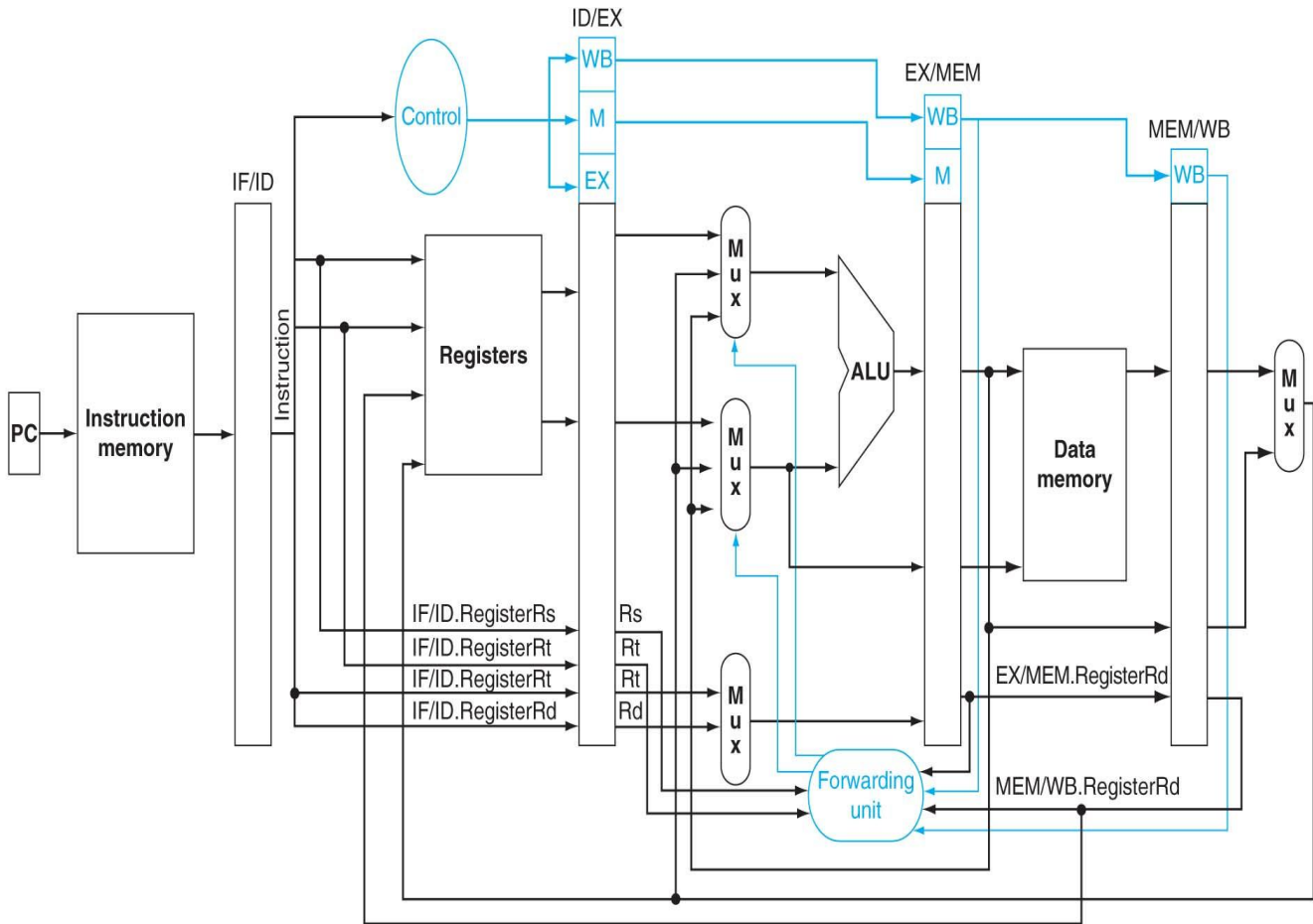


Figure 3.56 The datapath modified to resolve hazards via forwarding

Data Hazards and Stalls

One case where forwarding cannot work is when an instruction tries to read a register following a load instruction that writes the same register. Figure 3.57 illustrates the problem. The data is still being read from memory in clock cycle 4 while the ALU is performing the operation for the following instruction. Something must stall the pipeline for the combination of load followed by an instruction that reads its result.

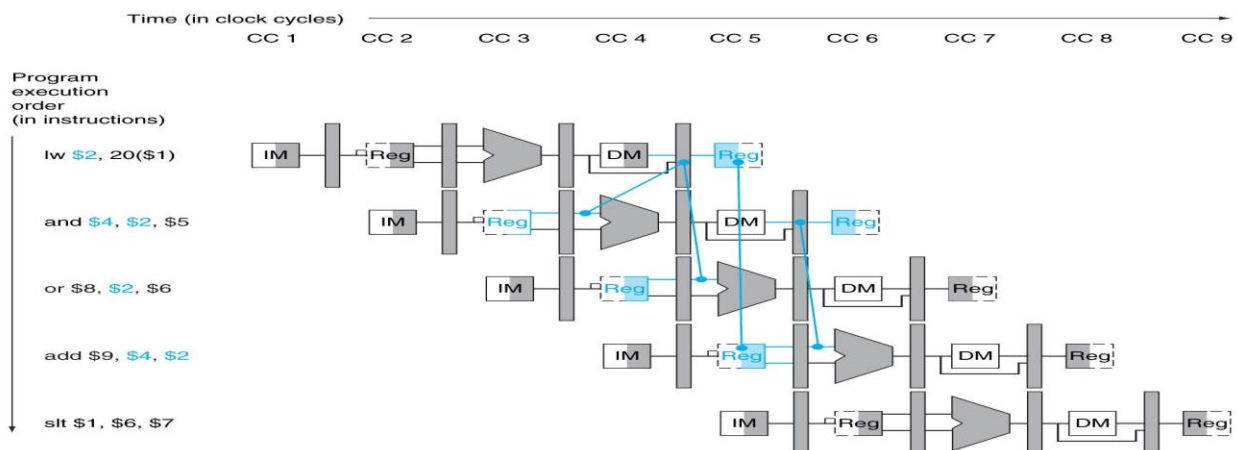


Figure 3.57 A pipelined sequence of instructions

ID/EX.MemRead		and
((ID/EX.RegisterRt = IF/ID.RegisterRs)	=	or
(ID/EX.RegisterRt = IF/ID.RegisterRt))		
If detected, stall and insert bubble		

If the instruction in the ID stage is stalled, then the instruction in the IF stage must also be stalled; otherwise, we would lose the fetched instruction. Preventing these two instructions from making progress is accomplished simply by preventing the PC register and the IF/ID pipeline register from changing. Provided these registers are preserved, the instruction in the IF stage will continue to read using the same PC, and the registers in the ID stage will continue to be read using the same instruction fields in the IF/ID pipeline register.

Figure 3.58 shows what really happens in the hardware: the pipeline execution slot associated with the AND instruction is turned into a nop and all instructions beginning with the AND instructions are delayed one clock cycle. Like an air bubble in a water pipe, a stall bubble delays everything behind it and proceeds down the instruction pipe one stage each cycle until it exits at the end.

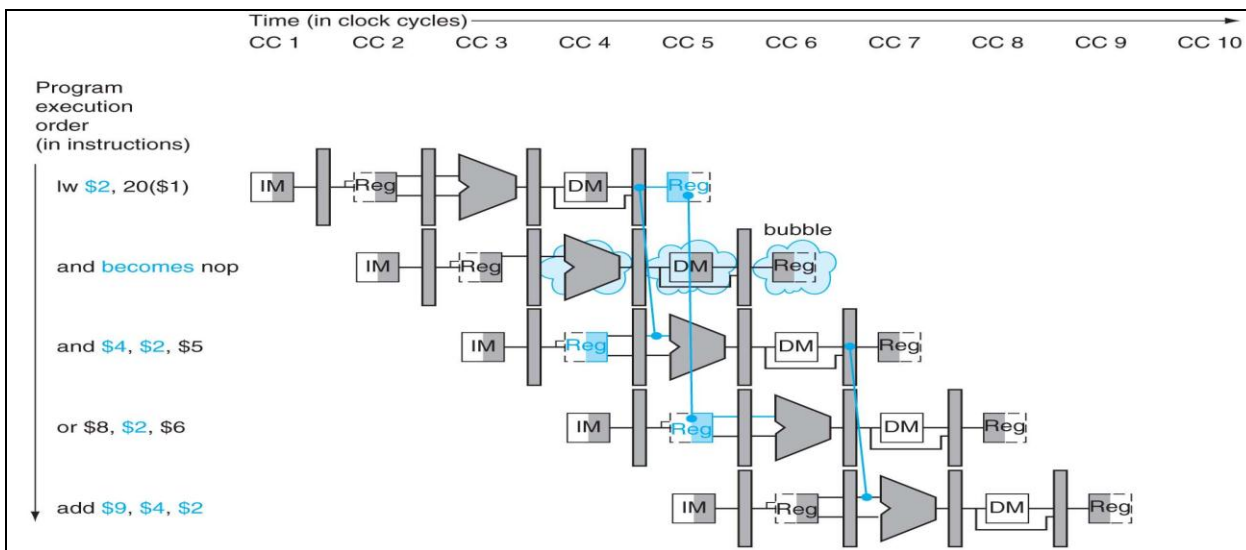


Figure 3.58 The way stalls are really inserted into the pipeline

Figure 3.59 highlights the pipeline connections for both the hazard detection unit and the forwarding unit. As before, the forwarding unit controls the ALU multiplexors to replace the value from a general-purpose register with the value from the proper pipeline register. The hazard detection unit controls the writing of the PC and IF/ID registers plus the multiplexor that chooses between the real control values and all 0's. The hazard detection unit stalls and deasserts the control fields if the load-use hazard test above is true.

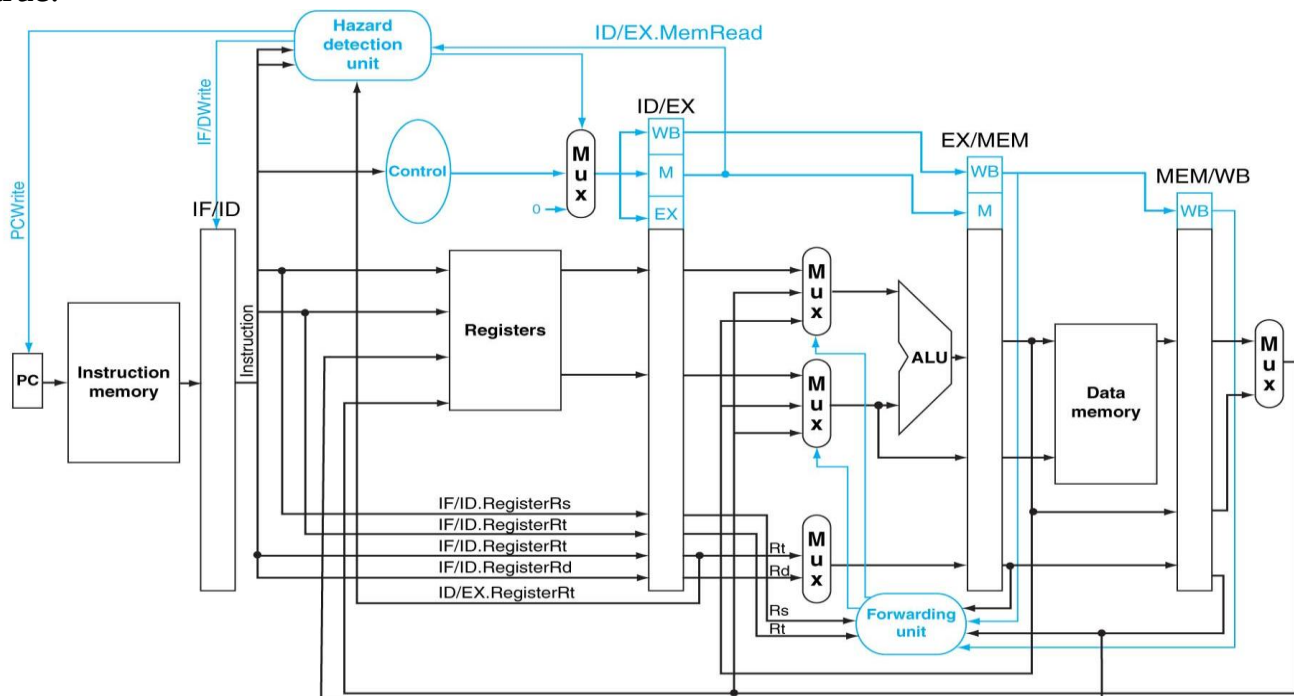
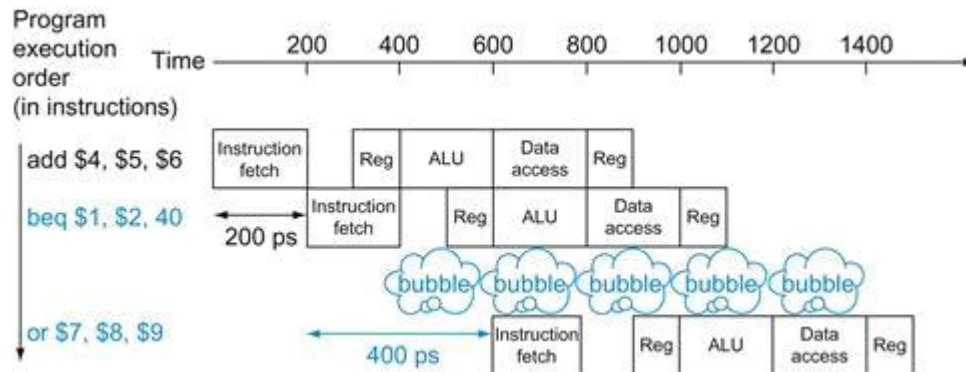


Figure 3.59 Pipelined control overview, showing the two multiplexors for forwarding, the hazard detection unit, and the forwarding unit

3. Control hazard : Also called branch hazard. When the proper instruction cannot execute in the proper pipeline clock cycle because the instruction that was fetched is not the one that is needed; that is, the flow of instruction addresses is not what the

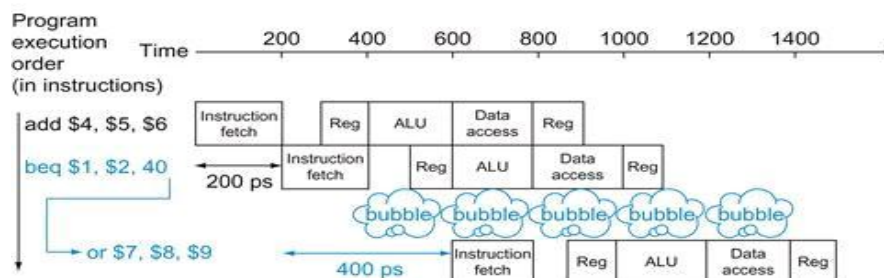
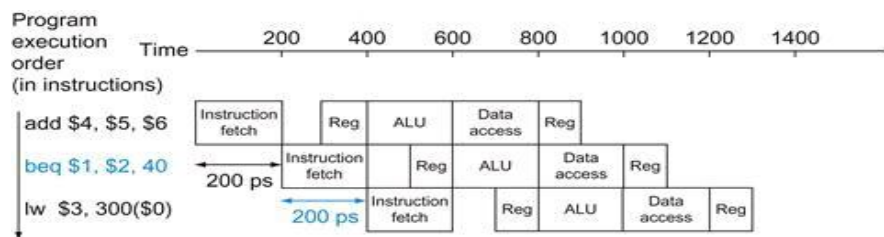
pipeline expected.

Pipeline showing stalling on every conditional branch as solution to control hazards.



This example assumes the conditional branch is taken, and the instruction at the destination of the branch is the OR instruction. There is a one-stage pipeline stall, or bubble, after the branch.

In the case of longer pipelines, the slowdown would be even larger if we stall on branches. The cost of this option is too high for most computers to use and motivates a second solution to the control hazard.



The top drawing shows the pipeline when the branch is not taken. The bottom drawing shows the pipeline when the branch is taken. As it is shown in Figure above the insertion of a bubble in this fashion simplifies what actually happens, at least during the first clock cycle immediately following the branch.

Figure 3.60 shows a sequence of instructions and indicates when the branch would occur in this pipeline. An instruction must be fetched at every clock cycle to

sustain the pipeline, yet in our design the decision about whether to branch doesn't occur until the MEM pipeline stage. This delay in determining the proper instruction to fetch is called a **control hazard or branch hazard or instruction hazard**.

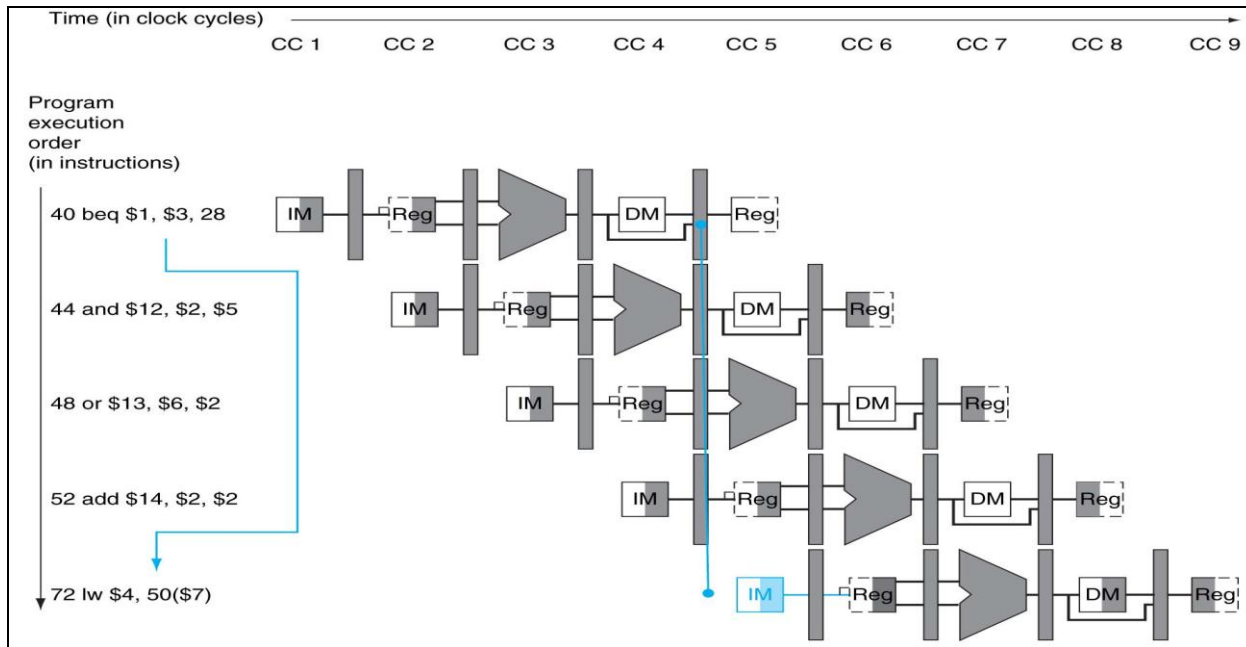


Figure 3.60 The impact of the pipeline on the branch instruction

Control hazards are relatively simple to understand, they occur less frequently than data hazards, and there is nothing as effective against control hazards as forwarding is against data hazards. We look at two schemes for resolving control hazards and one optimization to improve these schemes.

Branch Not Taken:

Stalling until the branch is complete is too slow. A common improvement over branch stalling is to assume that the branch will not be taken and thus continue execution down the sequential instruction stream. If the branch is taken, the instructions that are being fetched and decoded must be discarded. Execution continues at the branch target. If branches are untaken half the time, and if it costs little to discard the instructions, this optimization halves the cost of control hazards.

To discard instructions, we merely change the original control values to 0s, similar to the one we did to stall for a load-use data hazard. The difference is that we must also change the three instructions in the IF, ID, and EX stages when the branch reaches the MEM stage. Discarding instructions, means we must be able to flush instructions in the IF, ID, and EX stages of the pipeline.

Reducing the Delay of Branches:

One way to improve branch performance is to reduce the cost of the taken branch. So far, we have assumed the next PC for a branch is selected in the MEM stage, but if we move the branch execution earlier in the pipeline, then fewer instructions need be flushed. The MIPS architecture was designed to support fast single-cycle branches that could be pipelined with a small branch penalty.

Moving the branch decision up requires two actions to occur earlier:

1. Computing the Branch Target Address and
2. Evaluating the Branch Decision.

Computing the Branch Target Address:

The easy part of this change is to move up the branch address calculation. We already have the PC value and the immediate field in the IF/ID pipeline register, so we just move the branch adder from the EX stage to the ID stage; of course, the branch target address calculation will be performed for all instructions, but only used when needed.

Evaluating the Branch Decision:

The harder part is the branch decision itself. For branch equal, we would compare the two registers read during the ID stage to see if they are equal. Equality can be tested by first exclusive ORing their respective bits and then ORing all the results. Moving the branch test to the ID stage implies additional forwarding and hazard detection hardware, since a branch dependent on a result still in the pipeline must still work properly with this optimization.

For example, to implement branch on equal, we will need to forward results to the equality test logic that operates during ID. There are two complicating factors:

1. During ID, we must decode the instruction, decide whether a bypass to the equality unit is needed, and complete the equality comparison so that if the instruction is a branch, we can set the PC to the branch target address. Forwarding for the operands of branches was formerly handled by the ALU forwarding logic, but the introduction of the equality test unit in ID will require new forwarding logic.
2. Because the values in a branch comparison are needed during ID but may be produced later in time, it is possible that a data hazard can occur and a stall will be needed.

Despite these difficulties, moving the branch execution to the ID stage is an improvement, because it reduces the penalty of a branch to only one instruction if the branch is taken, namely, the one currently being fetched.

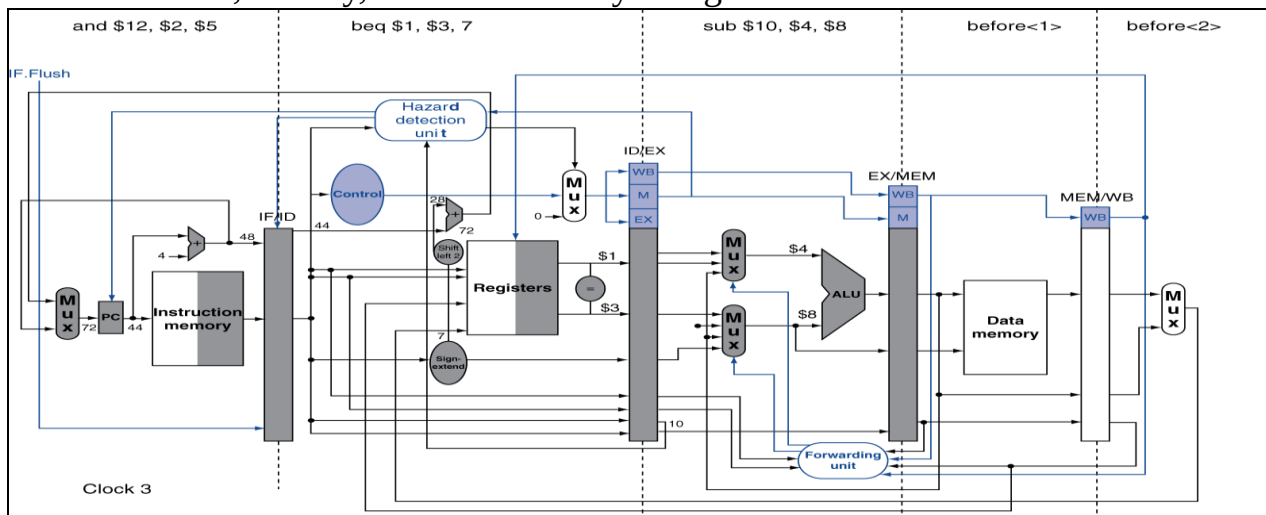


Figure 3.61 The ID stage of clock cycle 3 determines that a branch must be taken, so it selects 72 as the next PC address and zeros the instruction fetched for the next clock cycle

To flush the instructions in the IF stage, we add a control line, called IF.Flush, that zeros the instruction field of the IF/ID pipeline register. Clearing the register transforms the fetched instruction into a nop, an instruction that has no action and changes no state.

Branch Prediction:

Branch prediction is a technique for predicting branch penalty associated with conditional branches is to attempt to predict whether or not a particular branch will be taken or not. There are two forms of branch prediction techniques. They are

1. Static Branch Prediction
2. Dynamic Branch Prediction

Static Branch Prediction

The simplest form of branch prediction is to assume that the branch will not take place and to continue to fetch instructions in sequential address order, until the branch condition is evaluated, instruction execution along the predicted path must be done on a speculative basis.

Assuming a branch is not taken is one simple form of branch prediction. In that case, we predict that branches are untaken, flushing the pipeline when we are wrong. For the simple five-stage pipeline, possibly coupled compiler-based prediction is probably adequate. With deeper pipelines, the branch penalty increases when measured in clock cycles. Similarly, with multiple issue, the branch penalty increases in terms of instructions lost. This combination means that in an aggressive pipeline, a simple static prediction scheme will probably waste too much performance.

Dynamic Branch Prediction

With more hardware it is possible to try to predict branch behavior during program execution. One approach is to look up the address of the instruction to see if a branch was taken the last time this instruction was executed, and if so, to begin fetching new instructions from the same place as the last time. This technique is called dynamic branch prediction.

One-bit Prediction Scheme

One implementation of that approach is a branch prediction buffer or branch history table. A branch prediction buffer is a small memory indexed by the lower portion of the address of the branch instruction. The memory contains a bit that says whether the branch was recently taken or not.

This is the simplest form of buffer, in fact, if the prediction is the right one – it may have been put there by another branch that has the same low-order address bits. However, this doesn't affect correctness.

Prediction is just a hint that we hope is correct, so fetching begins in the predicted direction. If the hint turns out to be wrong, the incorrectly predicted instructions are deleted, the prediction bit is inverted and stored back, and the proper sequence is fetched and executed.

This simple 1-bit prediction scheme has a performance shortcoming: even if a branch is almost always taken, we can predict incorrectly twice, rather than once, when it is not taken.

Two-bit Prediction Scheme

The accuracy of the predictor would match the taken branch frequency for these highly regular branches. To remedy this weakness, the 2-bit prediction schemes are often used. In a 2-bit scheme, a prediction must be wrong twice before it is changed. Figure 3.62 shows the finite-state machine for a 2-bit prediction scheme.

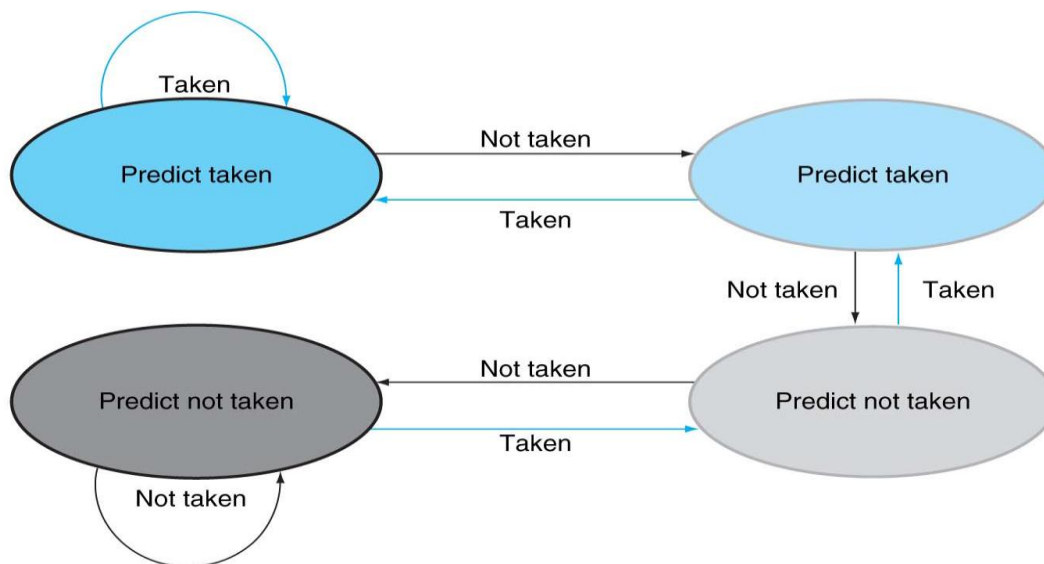


Figure 3.62 The states in a 2-bit prediction scheme

A branch prediction buffer can be implemented as a small, special buffer accessed with the instruction address during the IF pipe stage. If the instruction is predicted as taken, fetching begins from the target as soon as the PC is known; it can be as early as the ID stage. Otherwise, sequential fetching and executing continue. If the prediction turns out to be wrong, the prediction bits are changed.

Delayed Branching Technique

In a five-stage pipeline we can make the control hazard a feature by redefining the branch. A delayed branch always executes the following instruction, but the second instruction following the branch will be affected by the branch.

The location following a branch instruction is called a branch delay slot. Compilers and assemblers try to place an instruction that always executes after the branch in the branch delay slot. The job of software is to make the successor instructions valid and useful. Figure 3.63 shows the three ways in which the branch delay slots can be scheduled.

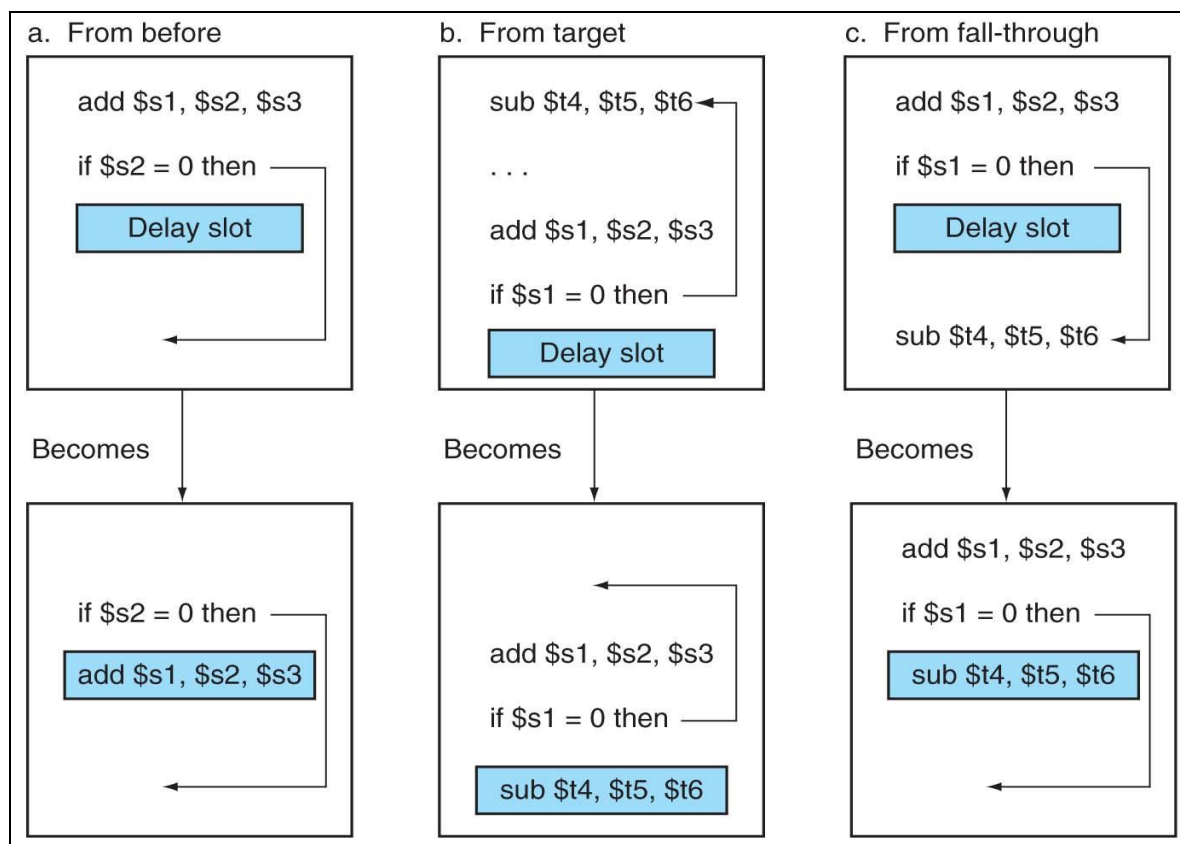


Figure 3.63 Scheduling the branch delay slot

The limitations on delayed branch scheduling arise from the restrictions on the instructions that are scheduled into the delay slots and our ability to predict at compile time whether a branch is likely to be taken or not.

Delayed branching was a simple and effective solution for a five-stage pipeline issuing one instruction each clock cycle. As processors go to both longer pipelines and issuing multiple instructions per clock cycle, the branch delay becomes longer, and a single delay slot is insufficient. Hence, delayed branching has lost popularity compared to more expensive but more flexible dynamic approaches. Simultaneously, the growth in available transistors per chip has made dynamic prediction relatively cheaper.

EXCEPTIONS

Control is the most challenging aspect of processor design: it is both the hardest part to get right and the hardest part to make fast.

One of the hardest parts of control is implementing exceptions and interrupts – events other than branches or jumps that change the normal flow of instruction execution.

They were initially created to handle unexpected events from within the processor, like arithmetic overflow. Many of the requirements to support exceptions come from the specific situation that causes an exception to occur.

Detecting exceptional conditions and taking the appropriate action is often on the critical timing path of a processor, which determines the clock cycle timing and thus performance.

Types of Exceptions

The two types of exceptions that our current implementation can generate are

- a. Execution of an undefined instruction and
- b. Arithmetic overflow.

a. Undefined Instruction Exception

The basic action that the processor must perform when an exception occurs is to save the address of the offending instruction in the exception program counter (EPC) and then transfer control to the operating system at some specified address.

The OS can then take the appropriate action, which may involve providing some service to the user program, taking some predefined action in response to an overflow, or stopping the execution of the program and reporting an error. After performing whatever action is required because of the exception, the operating system can terminate the program or may continue its execution, using the EPC to determine where to restart the execution of the program.

For the OS to handle the exception, it must know the reason for the exception, in addition to the instruction that caused it. There are two main methods used to communicate the reason for an exception.

1. The method used in the MIPS architecture is to include a status register (called the cause register), which holds a field that indicates the reason for the exception.
2. A second method is to use vectored interrupts. In a vectored interrupt, the address to which control is transferred is determined by the cause of the exception.

For example, to accommodate the two exception types listed above, we might define the following two exception vector addresses:

Exception Type	Exception Vector Address (in hex)
Undefined Instruction	8000 0000
Arithmetic Overflow	8000 0180

The OS knows the reason for the exception by the address at which it is initiated. The addresses are separated by 32 bytes or eight instructions, and the OS must record the reason for the exception and may perform some limited processing in this sequence. When the exception is not vectored, a single entry point for all exceptions can be used, and the OS system decodes the status register to find the cause.

Let's assume that we are implementing the exception system used in the MIPS architecture, with the single entry point being the address 8000 0180. We will need to add two additional registers to the MIPS implementation:

1. EPC: A 32-bit register used to hold the address of the affected instruction. Such a register is needed even when exceptions are vectored.
2. Cause Register: A register used to record the cause of the exception. In the MIPS architecture, this register is 32-bits, although some bits are currently unused. Assume there is a five-bit field that encodes the two possible exception sources mentioned above, with 10 representing an unused instruction and 12 representing arithmetic overflow.

b. Arithmetic Overflow

A pipelined implementation treats exceptions as another form of control hazard. Suppose there is an arithmetic overflow in an add instruction. We must flush the instructions that follow the add instruction from the pipeline and begin fetching instructions from the new address. We will use the same mechanism we used for taken branches, but this time the exception causes the deasserting of control lines.

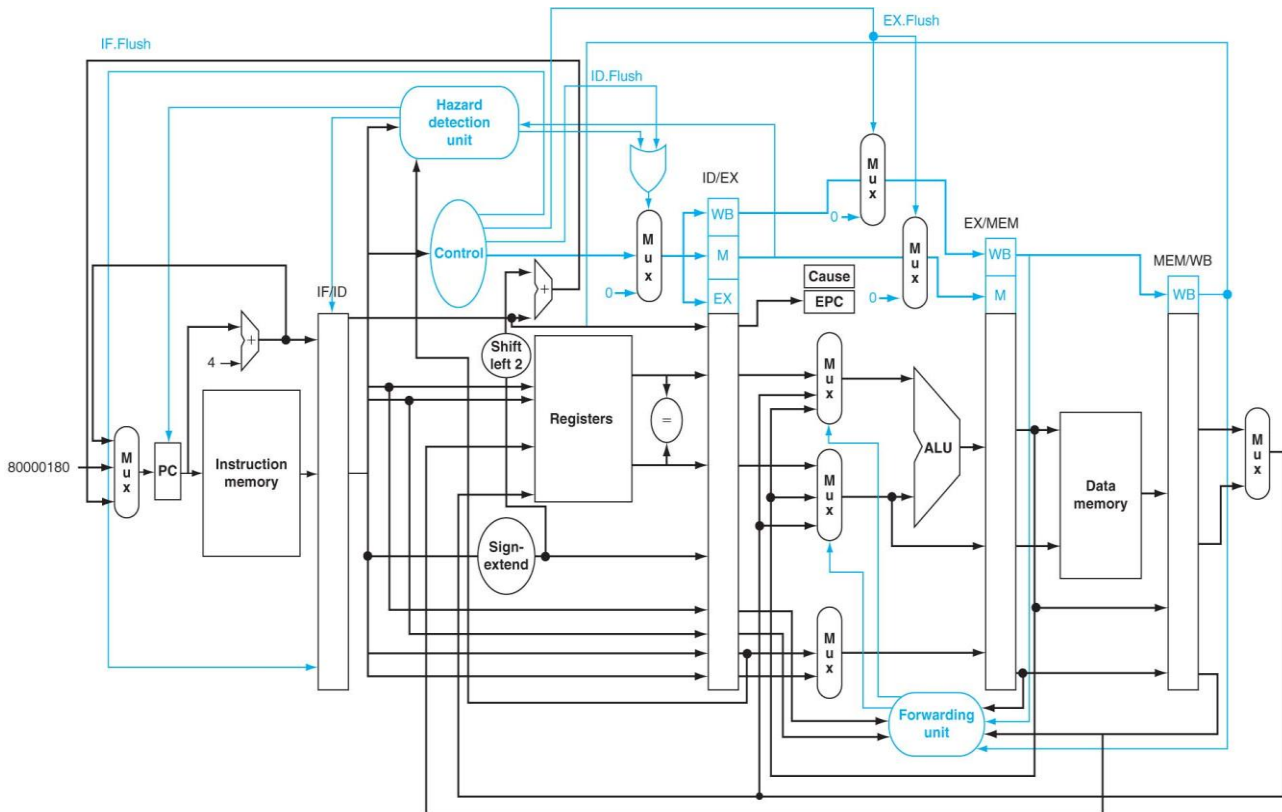


Figure 3.64 The datapath with controls to handle exceptions

When we dealt with branch misprediction, we saw how to flush the instruction in the IF stage by turning it into a nop. To flush instructions in the ID stage, we use the multiplexor already in the ID stage that zeros control signals for stalls. A new control signal, called ID.Flush, is ORed with the stall signal from the hazard detection unit to flush during ID. To flush the instruction in the EX phase, we use a new signal called EX.Flush to cause new multiplexors to zero the control

lines. To start fetching instructions from location 8000 0180, which is the MIPS exception address, we simply add an additional input to the PC multiplexor that sends 8000 0180 to the PC.

This example points out a problem with exceptions: if we do not stop execution in the middle of the instruction, the programmer will not be able to see the original value of register that helped cause the overflow because it will be clobbered as the destination register of the add instruction. Because of careful planning, the overflow exception is detected during the EX stage; hence, we can use the EX.Flush signal to prevent the instruction in the EX stage from writing its result in the WB stage. Many exceptions require that we eventually complete the instruction that caused the exception as if it executed normally. The easiest way to do this is to flush the instruction and restart it from the beginning after the exception is handled.

The final step is to save the address of the offending instruction in the exception program counter (EPC). In reality, we save the address+4, so the exception handling routine must first subtract 4 from the saved value.

Figure 3.65 shows the events, starting with the add instruction in the EX stage. The overflow is detected during that phase, and 8000 0180 is forced into the PC. Clock cycle 7 shows that the add and following instructions are flushed, and the first instruction of the exception code is fetched.

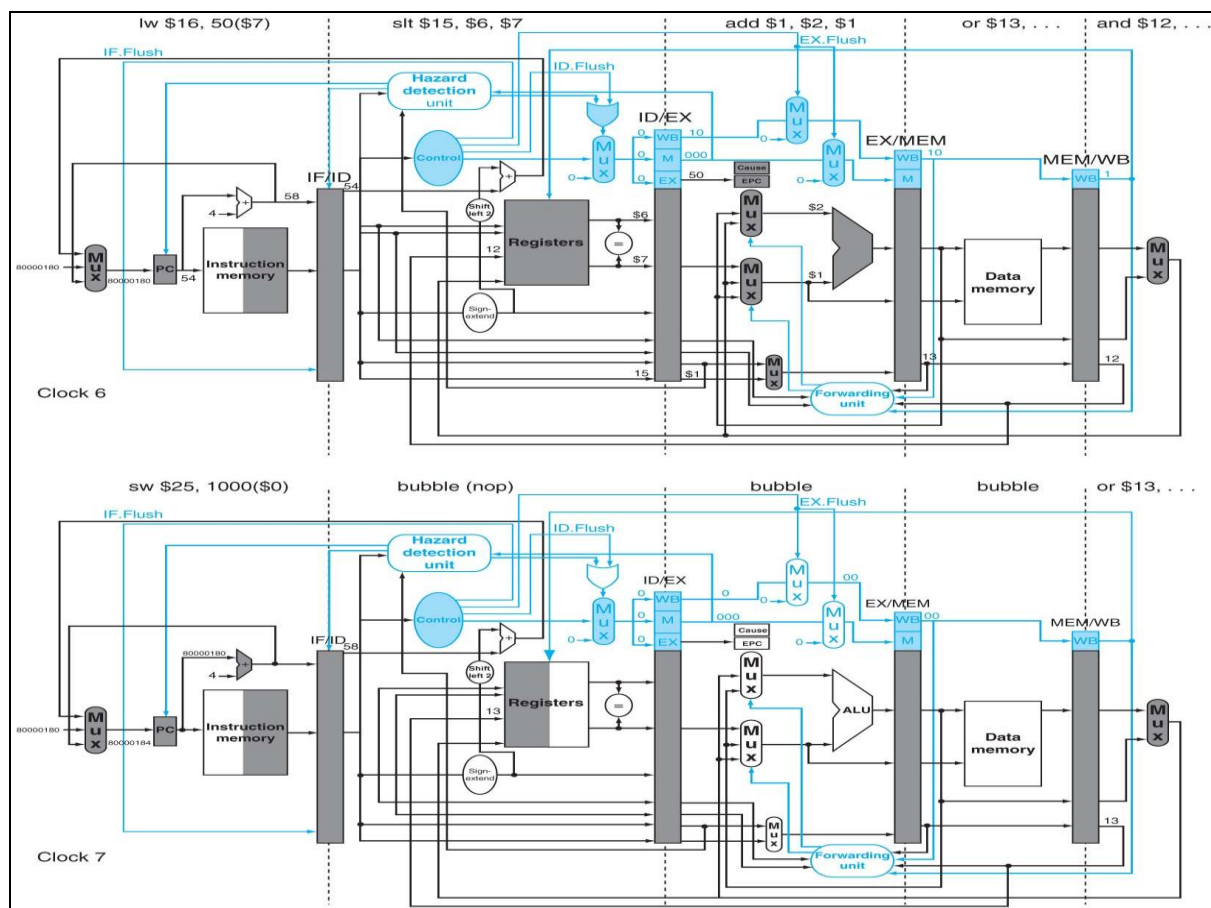


Figure 3.65 The result of an exception due to arithmetic overflow in the add instruction

With five instructions active in any clock cycle, the challenge is to associate an exception with the appropriate instruction. Moreover, multiple exceptions can occur simultaneously in a single clock cycle. The solution is to prioritize the exceptions so that it is easy to determine which is serviced first. In most MIPS implementations, the hardware sorts exceptions so that the earliest instruction is interrupted.

I/O device requests and hardware malfunctions are not associated with a specific instruction, so the implementation has some flexibility as to when to interrupt the pipeline. Hence, the mechanism used for other exceptions works just fine.

The EPC captures the address of the interrupted instructions, and the MIPS cause register records all possible exceptions in a clock cycle, so the exception software must match the exception to the instruction. An important clue is knowing in which pipeline stage a type of exception can occur. For example, an undefined instruction is discovered in the ID stage, and invoking the operating system occurs in the EX stage. Exceptions are collected in the Cause register in a pending exception field so that the hardware can interrupt based on later exceptions, once the earliest one has been serviced.

Imprecise Exceptions

It is also called as imprecise interrupt. Interrupts or exceptions in pipelined computers those are not associated with the exact instruction that was the cause of the interrupt or exception.

Precise Exceptions

It is also called as precise interrupt. An interrupt or exception that is always associated with the correct instruction in pipelined computers.